1 A Recap of OpenMP 2.5

In [2], OpenMP is covered at great length. At the time that [2] was published, the OpenMP 2.5 specifications were released and that is what that book focused on. Since then, four more specifications have been released. The enhancements and new features they introduced are the major topic of this book. For completeness, this chapter presents a brief recap of OpenMP 2.5. In many cases, the syntax description is informal and not every aspect is covered. For such details, refer to the specifications [20].

1.1 OpenMP Directives and Syntax

OpenMP is an Application Programming Interface (API) for shared memory parallelization, using C, C++, or Fortran. The API consists of compiler directives to specify and control parallelization, augmented with runtime functions, and environment variables. It is incumbent upon the user to identify the parallelism and insert the appropriate control structures in the program. These controls are called *directives*.

In C/C++, the directive is based on the `#pragma omp` construct. This is a regular language pragma and is ignored if it is not recognized by the compiler. This may happen for several reasons: 1) the compiler does not support OpenMP, although that is rare these days, 2) if a typo is made (e.g. `#pragma openmp`), or if the source is not compiled with an option to enable recognition of the directives.

In Fortran, the directive starts with either `!$omp` or `*$omp/c$omp` in old-style Fortran programs. To the compiler, this is a regular comment string, unless through the use of a specific option, the compiler is instructed to recognize the special `omp` keyword and generate the corresponding OpenMP code, but also here a typo gives unexpected results.

In C/C++ and Fortran, portability of the code is not violated when using the directives. The source compiles with, or without, OpenMP enabled. If the OpenMP-specific runtime functions are used, but no option to support OpenMP is specified, unresolved references to those functions are reported by the linker. Luckily, there is an easy way around this.

If a source is compiled with OpenMP enabled, the `_OPENMP` macro is guaranteed to be set in C/C++. It is set to the release date of the OpenMP version the compiler supports. Regardless of the value, this macro may be used in conjunction with an `#ifdef` construct to support the runtime functions.

For example, the code fragment below handles the use of the `omp_get_num_threads()` function and compiles both with and without OpenMP enabled. In the case of the former, the include file `omp.h` is used, exposing all OpenMP function definitions to the compiler. Outside an OpenMP environment, the function is defined to always return `1` for the number of threads.

```
1 #ifdef _OPENMP
2   #include <omp.h>
3 #else
4   #define omp_get_num_threads() 1
5 #endif
```

In Fortran, conditional compilation may be used to handle the use of runtime functions. With this mechanism, the character string `!$` (or `*$` and `c$`) is replaced by two spaces if the source is compiled with OpenMP enabled. Otherwise, it is considered to be a language comment and ignored. This allows for the compile-time activation of executable statements, as demonstrated below:

```
1 !$    use omp_lib
2          ....
3       nthreads = 1
4 !$    nthreads = omp_get_num_threads()
```

If not compiled for OpenMP, the first and third lines in the source snippet are ignored, because they are treated as language comments. If OpenMP is enabled, the `!$` string is replaced by two spaces. This results in the use of module `omp_lib` and the second assignment to variable `nthreads` is activated. The second assignment overrides the first one, and at runtime, the number of active threads is returned.[1]

An additional benefit of the directive-based approach is that compilers have full visibility of the (parallel) code and typically perform many optimizations, as well as issue warning messages. This is much more difficult to do with a programming model based upon function calls.

1.2 Creating a Parallel Program with OpenMP

The user must identify the code part(s) that may be executed in parallel. The parallelism is defined through the addition of the appropriate directives to the

[1] Optimizing compilers recognize that the first assignment is redundant and eliminate it.

#pragma omp parallel *[clause[[,] clause]...] new-line* *structured block*
!$omp parallel *[clause[[,] clause]...]* *structured block* **!$omp end parallel**

Figure 1.1: **Syntax of the parallel construct in C/C++ and Fortran** – The statements within the parallel region are executed by all threads. In C/C++, the parallel region implicitly ends at the end of the structured block.

source code. It is the responsibility of the user to identify which part(s) are selected to run in parallel and use the various constructs to ensure correct results.

The nature (private or shared), or "scoping," of the variables must also be specified, but everything else is handled by the compiler and OpenMP runtime system. This completely eliminates the book-keeping and handling of details found in many other parallel programming models.

In many cases, more control than simply the execution of a block of code in parallel is needed. Extensive additional functionality is available and easy to add to an application. Subsequent sections in this chapter describe the type of building blocks that are available to implement additional functionality.

1.2.1 The Parallel Region

A parallel program in OpenMP starts and ends with the *parallel region*. It is the cornerstone of OpenMP. This is where parallel execution takes place and multiple threads are put to work. The syntax in C/C++ and Fortran is given in Figure 1.1.

There is no limit on the number of parallel regions, but for performance reasons it is best to keep the number of regions to a minimum and make them as large as possible. In C/C++, it is good practice to use a comment string to mark the end of the parallel region. This is less of an issue in Fortran, because it has the mandatory terminating `!$omp end parallel` construct.

The thread that encounters the parallel region is called the *master thread*. It creates the additional threads and is in charge of the overall execution. The threads that are active within a parallel region are referred to as a *thread team*, or "team" for short. Multiple teams may be active simultaneously.

The specifications also distinguish between an *active* and *inactive* parallel region. In the case of the former, the team consists of more than the master thread. This

is in contrast with an inactive parallel region, which is executed only by the master thread.

The statements within the parallel region are executed by all threads, unless the optional `if` clause evaluates to `false`. In this case, only one thread executes the region. This implies that it is an inactive parallel region. Outside of the parallel region(s), the master thread executes the serial portions of the application.

All OpenMP directives are to be placed in the *dynamic* extent of a parallel region in order to have an effect. This means that they need not be visible from the parallel region, but must be in the execution path that starts at a parallel region.

An example is a function with OpenMP constructs that is called from within a parallel region. If the same function is called outside of a parallel region, the directives are ignored and only one thread executes the code.

The situation is somewhat different for the OpenMP runtime functions. These may be used anywhere in the program, but some return a meaningful result, only if executed from within a parallel region. Other functions *must* be executed outside of a parallel region. For example, the `omp_set_num_threads()` function is used to set the number of threads. It is illegal to use it within a parallel region, and the runtime behavior is undefined if this rule is violated.

Before discussing the major OpenMP constructs, the execution and memory model are presented. Every user must have a basic understanding of these topics.

1.2.2 The OpenMP Execution Model

The way an OpenMP program executes is defined by the execution model. The OpenMP model is illustrated in Figure 1.2.

An OpenMP program always starts in serial mode. The thread executing the serial parts of the application, that is, the code outside the parallel regions, is called the *master thread*. This thread runs throughout the lifetime of the program.

At some point during the execution, the additional threads are created by the runtime system. When and how this happens is implementation-defined, but the threads are guaranteed to be available when the parallel regions are encountered.[2]

The master thread is included in the total number of active threads in the parallel region. For example, in Figure 1.2, four additional threads are created when the first parallel region is encountered.

[2]This assumes nothing unusual has happened, causing one or more threads to be unavailable.

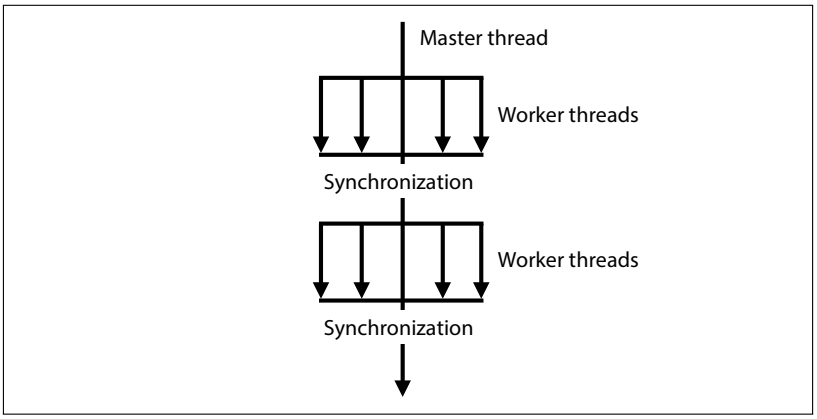

Figure 1.2: **The OpenMP execution model** – The master thread runs from start to finish. When a parallel region is encountered, the additional threads are engaged to perform the work that needs to be done. A barrier at the end of each parallel region synchronizes all threads. Execution continues only after the last thread has arrived at the barrier. After the parallel region exits, the master thread continues, until the next parallel region is encountered.

After this first thread-creation phase, multiple threads are active and within the parallel regions, the program executes in parallel. At the termination of a parallel region, the master thread continues until the next parallel region is encountered. This is called the *fork-join model*.

Until OpenMP 3.0, the user had no control over the behavior of the idle threads in-between the parallel regions, but the newly introduced environment variable `OMP_WAIT_POLICY` sets the behavior of idle threads. For more information on this variable, refer to Section 2.2 on page 53.

There are various ways to control the number of threads executing the parallel region. By default, this value is defined through environment variable `OMP_NUM_THREADS`. This sets the initial number of threads, but if not set by the user, a system-dependent default is used. Without any other action taken, it remains constant throughout the execution of the program.

If the number of threads needs to be more dynamic, the `omp_set_num_threads()` runtime function may be used to change the number of threads, while the application executes. This function affects the number of threads used in the next parallel region encountered, as well as all subsequent regions. This function

should only be executed *prior* to a parallel region. It is illegal to call it within a parallel region.

An alternative is to use the `num_threads(<nt>)` clause on the directive that defines the parallel region. The OpenMP runtime system uses the value of variable `nt` as the number of threads for the current parallel region, plus all subsequent parallel regions.

A useful feature is that the number of threads may increase, or decrease, on a per-parallel region basis. How this is implemented is transparent to the user and the responsibility of the runtime system. If the thread count shrinks, it is determined by the implementation how to handle the threads no longer needed. A common choice is to keep these threads around and avoid the relatively expensive setup cost of recreating threads in case the number increases again.

Thread synchronization occurs in the implied barrier at the end of the parallel region. The `nowait` clause is not supported on a parallel region. In other words, this barrier is always present and cannot be removed.

For a single-level parallel region this is a not a problem. It makes sense for all threads to finish their work before the master thread continues. With nested parallelism (see also section 1.5), this is not so obvious and the additional barriers may negatively impact performance.

1.2.3 The OpenMP Memory Model

Until now, a very important aspect of shared memory parallel programming has been glossed over: the memory model. It describes what types of memory there are and how threads interact with each of them. It also defines in what way the potentially different views on memory maintain consistency and, most importantly, when.

Until support for heterogeneous computing was introduced in OpenMP 4.0, there was a single address space only. This is still the case for the (host) system where the OpenMP program starts execution, but if accelerators are used, multiple address spaces may exist.

As illustrated in Figure 1.3, an OpenMP program has two different elementary types of memory: *private* and *shared*. To which memory type a variable belongs is defined by default rules, but may also be explicitly controlled through the appropriate clause on a construct.

In the case of the latter, the user must assign the appropriate memory type to variables used in the parallel region. Especially if the `default(none)` clause is

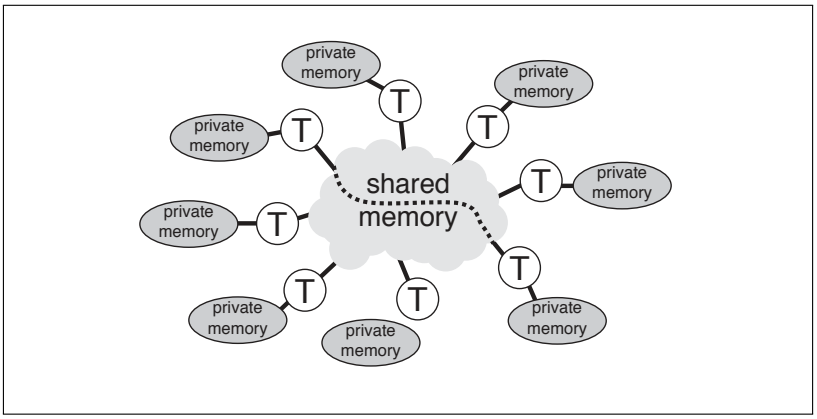

Figure 1.3: **The OpenMP memory model** – There are two different elementary types of memory: private and shared. Each thread has a private memory. A thread may access, or modify, variables in its own private memory, but not in another thread's private memory. In addition to this, there is a single shared memory. All threads have read and write access to variables stored there.

used, all variables need to be specified. Although more work upfront, it is strongly recommended not to rely on the default rules and explicitly label, or "scope," the variables.

The reason is that the default rules are clearly defined, but not always what one might expect. Knowing the subtleties is one option, but it is still easy to make a mistake. It is also not as difficult as it may seem at first sight to explicitly scope the variables.

There is one exception to this recommendation. In C/C++, variables declared within a code block are automatically made private. This is very convenient, and the next section has more information on this.

Private variables Each thread has unique access to its private memory and no other thread may interfere. There is never the risk of an access conflict with another thread. Although multiple threads may use the same name for a private variable, these variables are stored in different locations in memory. Modifications apply to the local copy only and a thread can never change the value of a private variable owned by another thread.

```
 1 x = 5;
 2 y = 20;
 3 #pragma omp parallel private(x) firstprivate(y)
 4 {
 5    x = 10;        // x is undefined on entry, but now set to 10
 6    int z = x + y; // y was pre-initialized to a value of 20
 7          ....
 8    y = 30;        // (first)private variables may be modified
 9          ....
10 } // End of parallel region
```

Figure 1.4: **Example of private variables** – Private variables are undefined on entry to (and exit from) the parallel region. The `firstprivate` clause at line 3 is used to pre-initialize variable `y` to 20 and use this value at line 6.

A loop variable is a simple example. If this variable is in private memory, as it must be, multiple threads may use the *same* variable name to execute a loop. It is impossible for one thread to modify the loop variable in the private memory of another thread.

If the same variable name is used outside, and within a parallel region, there is no risk of a conflict either. The reason is that the variable inside the region is treated as a different variable (even though it has the same name). For example, in the code fragment in Figure 1.4, variable `x` appears both outside, and inside the parallel region, but there is no conflict.

Although not yet available in OpenMP 2.5, more recent versions of the specifications describe many features in terms of tasks. In Chapter 3, tasking is discussed in great detail, but ahead of this, we need to introduce the concept of an *implicit task*. This is a task, generated by an implicit parallel region, or generated when a parallel construct is encountered during execution. In the context of a parallel region, or a worksharing construct, an "execution instance" is an implicit task. When a variable is private, each execution instance refers to its own instance of that variable. This topic also comes up with the support for SIMD instructions. See Section 5.2.1 for the definition of an execution instance with `simd` loops.

In OpenMP, the lifetime of a private variable is restricted. It only exists within a parallel region and by default, is undefined on entry to and exit from a construct. If needed, there are ways around this, but this must be explicitly handled through the appropriate constructs.

This is also illustrated in the example. Although variable `x` has been initialized to 5 by the master thread, its value is undefined within the parallel region. This is not a problem, because it is explicitly set to 10.

How about variable `y`? The value is needed within the parallel region, but it is not defined. The solution is to declare this variable to be `firstprivate`, instead of `private`. This not only guarantees that all threads have a private copy of this variable, but each copy is pre-initialized to the value `y` had prior to entering the parallel region.

This is also illustrated in the example. Upon executing the parallel region, variable `y` inherits the original value of 20. This is the value used at line 6. At line 8, it is reset to 30. This variable may actually have a different value within each thread, because it is a private variable.

The `firstprivate` clause is also supported on some constructs introduced after OpenMP 2.5. This is covered in the respective sections in the remainder of this book.

Variable `z` is declared within the parallel region. Per the rules of OpenMP, this makes it a private variable. This is a convenient feature and the exception to the recommendation not to rely on the default scoping rules.

In some cases, there is a need to use the value of a private variable after the parallel construct. For this situation, the worksharing `loop` and `sections` construct support the `lastprivate` clause.[3] This makes the value of such variable(s) accessible after the construct.

In a parallel application, it is not always clear what "last" means. For these constructs, this is defined such that the sequential semantics are preserved. For example, on a loop, it is the value as computed for the sequentially last iteration. This convenience comes at a (small) cost in performance, so use it with care.

There is one more thing worth mentioning here. What happens with the value of the original variable if it is also declared to be a private variable within the parallel region? The answer is that it depends on the version of OpenMP used. In OpenMP 2.5 the value is undefined upon exit of the parallel region. This was a

[3]This clause is also supported on `simd` loops, covered in Chapter 5.

highly undesirable limitation and has been lifted. As of OpenMP 3.0, the initial value of this variable is preserved.

Shared variables In addition to its private memory, each thread has access to the same single shared memory. There are two substantial differences with private memory.

In contrast with a private variable, only one instance of a shared variable exists. All threads "see" the same shared variable(s). In addition to this, each thread may read, as well as write, any shared variable. As long as the variable is within the dynamic extent, there is no constraint as to when this is allowed to occur. It is the responsibility of the user to correctly handle this situation

There is one case that requires extra attention and even warrants a specific directive: global, or static, variables. They are shared by default.

For example, consider the situation where a function uses global data as scratch space that is modified on each function call. If this function is called in parallel, a data race occurs, because multiple threads may modify the same memory address at the same time. A solution is to give each invocation of the function a private copy of the global data. In that way, even if multiple threads modify the data simultaneously, the updates are performed on different memory locations.

This is such a common scenario that, through the `threadprivate` directive, OpenMP provides elegant support for this situation. Each thread uses its private storage for the data that was originally shared. In case such privatized data must be pre-initialized, the `copyin` clause may be used.

The reason to be judicious when using shared variables, especially when they are modified, is performance-related. If multiple threads update the same cache line simultaneously and do this frequently, *false sharing* occurs and this has a negative impact on performance. In the worst case, this may lead to a substantial performance degradation. With private data, this is not an issue, so the recommendation is to use private data as much as possible. Shared variables are indispensable, but only use them if it is necessary to do so.

Memory Consistency Models The question is what happens when a modification has been made to a shared variable. The thread that made the change has immediate access to the new value, but what about the other threads? Intuitively, one might expect that all the threads see the same change at the same time as the thread that made the modification, but that is not the case.

Consider the following example. A thread initializes, or modifies, the value of two independent shared variables `x` and `y` in the order `x = ..., y = ....` The compiler may decide to interchange the statements and perform the assignment to `y` first. Unfortunately, such a valid optimization could be an issue if *another* thread assumes the value of `x` to be available *before* `y`. That is, the original order should not have been changed. The compiler has no way of knowing this though. It sees only the sequence of statements as executed by one thread and does not know how these statements may interact with other threads.[4]

When do the results of independent store instructions arrive in the main memory system and commit the values? It is not guaranteed that all the bytes of a variable are stored in one atomic transaction and actually may arrive piecemeal. Also, in what order do the variables arrive? Can it be guaranteed that they arrive in the order issued?

The fundamental question is how does one ensure that different threads see the same, consistent, value of a shared variable, and most importantly, when?

This is governed by the *memory consistency model*, or *consistency model* for short [13]. Such a model sets the rules regarding the handling of loads and stores. It defines *when* a thread is guaranteed to read the correct value of a variable after an update.

This is not a new issue. Over time, quite a number of consistency models have been defined, but for our purposes, only two cases are briefly covered. It is important to realize there is no right or wrong when it comes to the choice of a specific model. Performance and ease of use are the main differentiators and trade-offs.

Sequential Memory Consistency - This is defined by Lamport [16] and is intuitively easiest to understand. Each thread performs its loads and stores in the original sequential order, and stores are atomic. The memory operations may be thought of as a set, consisting of the loads and stores performed by this thread in the original sequence. Each of these sets is processed sequentially, but the order in which this happens is arbitrary.

In the code fragment in Figure 1.5, a classical example is shown. There are two threads and two shared variables `x` and `y`, both initialized to zero. Because of sequential consistency, either {`x,y`} = {`1,0`}, or {`x,y`} = {`0,1`}. This means only one thread executes the function call. Without sequential consistency, this is not

[4] At the hardware level, there is a potential issue too. Memory operations (among others) could be asynchronous because of caches, store buffers, buffer bypasses, and so on.

```
<Shared variables x and y are initialized to zero>

     Thread 0                Thread 1

     x = 1;                  y = 1;
     if ( y == 0 )           if ( x == 0 )
       function_call();        function_call();
```

Figure 1.5: **An example of the effect of sequential consistency** – Only one of the two threads executes the function call.

guaranteed. For example, if the compiler knows the function call has no side effects regarding `x` and `y`, it may decide to move the assignments after the if-statement. If that is the case, both threads call the function.

Unfortunately, excluding these kinds of optimizations and the imposed severe restrictions on the hardware regarding the handling of memory operations, reduces performance significantly. This is why many alternatives to sequential consistency have been proposed.

Relaxed Memory Consistency - This is not one single model, but a class of models, with a common goal to improve performance by relaxing the consistency requirements. The models differ in how far they go to relax the rules. The degrees of freedom are the order of the loads and stores, plus the atomicity.

Some examples of relaxed models are Total Store Order (TSO), Processor Consistency (PC), Relaxed Memory Ordering (RMO), and Release Consistency (RC). Covering these here is beyond the scope of this book. However, the details of the underlying memory consistency model need not be known in order to write a correct and efficient OpenMP program.

Memory Consistency and Cache Coherence - While a consistency model is needed for shared memory programming, cache coherence is not a necessary feature to implement OpenMP.

Cache coherence deals with the problem of *how* other cores see memory updates and does this by tracking changes in cache lines. A software implementation may do this, but hardware support is needed to make this efficient.[5] This is why hardware cache coherence has been implemented in shared memory systems for a long time.

[5]There are exceptions to this, but mostly in the area of special-purpose processors.

The confusion many users of OpenMP have is that, while cache coherence provides a single system image on a multi-core system, it does not prevent data races. There is no guarantee that a load instruction automatically returns the most recent value of a shared variable. This is defined by the consistency model and for performance reasons, most modern systems use a relaxed model.

The OpenMP View on Memory OpenMP assumes a relaxed memory consistency model, but does not require a specific implementation. Each thread is allowed to have its own *temporary view* of memory.[6]

As with any relaxed memory model, only at well-defined points, are threads guaranteed to have the same, consistent, view on the value of shared variables. In between such points, the temporary view may be different across the threads. OpenMP defines its own consistency points. At such a point, the temporary views are made consistent again. The question is how that impacts writing OpenMP code.

Read-only shared data is easy, because there are no concerns about consistency. Things get (much) more complicated when shared data is modified. Without precautions, any thread may modify shared data at any time. It is the responsibility of the user to ensure this is handled in the correct way.

As long as different threads write to a different memory location, for example, different elements of the same vector, there is no reason to worry. Problems arise if they simultaneously write to the *same* address in memory. Then, threads may step on each other and generate incorrect results. This is a bug in the code and is called a "data race."

A classical case is the accumulation of partial contributions into a shared variable. This occurs in reduction operations, for example. Each thread computes a contribution and adds it to the final solution, which is stored in a shared variable. This scenario is shown in the code fragment in Figure 1.6.

At line 1, shared variable `sum` is initialized to zero. This is outside the parallel region and performed by one thread only, the master thread. A local variable called `my_contribution` is declared at line 5 and per the rules of OpenMP, it is a private variable. Each thread performs its computations, and at line 7 the result is stored into variable `my_contribution`. In the next step, each thread reads the current

[6]The exception is the `seq_cst` clause on the `atomic` construct, which is covered in detail in Section 2.1.6, starting on page 47.

```
 1 sum = 0;
 2 #pragma omp parallel shared(sum)
 3 {
 4
 5   int my_contribution; // Contains the per-thread partial sum
 6     ....
 7   my_contribution = ....  // Thread computes a value
 8
 9   // Without the critical region this code has a data race
10
11   #pragma omp critical
12   {
13       sum += my_contribution;
14   } // End of critical region
15      ....
16 } // End of parallel region
```

Figure 1.6: **Example of the update of a shared variable** – To avoid multiple threads updating shared variable `sum` at the same time, a critical region is used.

value of variable `sum`, adds its contribution `my_contribution` to it, and stores the updated value of `sum`. This is the update statement at line 13, but without additional controls, this update has a data race on `sum` and the result is undefined.

The solution is to prevent another thread from reading `sum`, before the previous update has completed. OpenMP provides constructs like a critical section, or atomic update, to handle this situation. In Figure 1.6, the former is used to guard the update of variable `sum`. A thread is not allowed to enter a critical region, as long as another thread executes it. As a result, a thread cannot perform the update, while another thread is inside the critical region.

This is, however, not the end of the story. A relaxed memory consistency model is assumed, but what guarantee is there that a thread reads the correct value of `sum`, as it was recently updated by another thread? The answer is that, without additional measures, there is no such guarantee. Not even the initial value of zero is guaranteed to be available when the first thread performs the update. Unless this happens to be the master thread, but this cannot be counted upon.

This raises a very important question: *how does OpenMP ensure each thread reads the updated value?* This is where the `flush` construct comes in.

#pragma omp flush [*(flush-set)*] *new-line*
!$omp flush [*(flush-set)*]

Figure 1.7: **Syntax of the flush directive in C/C++ and Fortran** – This directive guarantees that all threads have the same consistent view of shared data.

The Flush Construct A key element of a relaxed consistency model is that the user needs to know when a shared variable may be read reliably.[7]

To enforce such global consistency, OpenMP provides the `flush` directive. After this directive, and before the next update of shared variables, all threads are guaranteed to have the same global view of all thread-visible variables. The syntax of the `flush` construct in C/C++ and Fortran is given in Figure 1.7.

The *flush-set* consists of a list with all the variables affected by the `flush` directive. Without this optional list, all thread-visible variables are affected. When a list is used, the flush-set consists of the variables in this list only. The recommendation is to *not* use the flush-set feature. It may be extremely difficult to employ correctly. If used incorrectly, optimizing compilers may apply valid code transformations, such that the `flush` does not have the intended effect.

Although compilers are not allowed to move operations on variables in the flush-set, beyond the `flush` directive, they are allowed to move flush constructs relative to each other in case the flush-sets are disjoint. For example, if the list contains variable `a` in one `flush` directive, and variable `b` in another one, both directives may be interchanged. This could result in incorrect values being read. To avoid this, both variables may be put in the same list, but this is also discouraged.

The `flush` directive affects only the thread executing it. The temporary view of other threads is not automatically updated. This implies that, in many cases, *all* threads must execute the same `flush` directive(s) to guarantee consistency.

The `flush` directive may be used explicitly in applications, but for convenience and as a safety net, a flush region is implied in the following situations:

- During a `barrier` region.
- At entry to, and exit from, the `parallel`, `critical`, `target`, and `target data` region.
- At exit from worksharing constructs, unless a `nowait` clause is present.

[7] The details depend on the model. Refer to Section 1.2.3 for an explanation of this.

- At entry to, and exit from, an `ordered` region, if a `threads` or `depend` clause is present, or if no clauses are present.
- Immediately before and after every task-scheduling point.
- During the following runtime functions: `omp_set_lock()` and `omp_unset_lock()`.
- During the following runtime functions, if the region causes the lock to be set or unset:
  - `omp_set_lock()`, `omp_unset_lock()`
  - `omp_test_lock()`, `omp_test_nest_lock()`
- At entry to, and exit from, the atomic operations (`read`, `write`, `update`, or `capture`), performed in a sequentially or non-sequentially consistent atomic region. In the case of the latter, the flush-set must contain the object updated in the construct.
- During a `cancel` or `cancellation point` region, in case cancellation has been enabled and activated.
- At entry to a `target update` region, whose corresponding construct has a `to` clause.
- At exit from a `target update` region, whose corresponding construct has a `from` clause.
- At entry to a `target enter data` region.
- At exit from a `target exit data` region.

The example in Figure 1.6 works correctly, because of the implied flush on the `critical` region. This ensures that the correct value is read before the update. The new value is synchronized, prior to exiting the region. In this case, the latter is not required for correctness, but it won't hurt either.

A flush region is *not* implied at the following locations:

- At entry to a `worksharing` region.
- At entry to, or exit from, a `master` region.

```
1 #pragma omp flush
2
3 while ( execution_state[i] != READ_FINISHED ) {
4
5    <wait for a short while>
6
7    #pragma omp flush
8
9 } // End of while-loop
```

Figure 1.8: **Example of the use of the flush directive** – The purpose of this while-loop is to wait for element `execution_stats[i]` to be modified.

The `master` construct is different from the `single` construct. Often, such a single thread region is used to initialize shared data. If the `master` construct is used for this, care needs to be taken that the other threads read the correct value(s). Without a `flush` region prior to accessing these variables, this is not guaranteed.

In many situations, the built-in flush operation avoids the need to explicitly use a flush directive. Even when the `nowait` clause is used, a barrier further down the execution path ensures a synchronized view on the shared data again.

This is why most OpenMP applications do not need to use an explicit `flush` construct, but in some cases, it is required for correct execution. An example of this is shown in Figure 1.8.

In this example, it is assumed that the master thread is in control of the value of variable `execution_state[i]`. The threads executing the loop, wait for the value to change. Without the two `flush` directives, there is no guarantee that changes in the value are propagated to the other threads. As a result, they might wait forever to actually see the change, so the program hangs.

The `flush` directive at line 1 is needed to ensure that the first time `execution_state[i]` is read, the correct value is returned at line 3. The second `flush` directive at line 7 guarantees this variable is updated, before it is read again at line 3. Although not shown here, the master thread must use a `flush` directive after setting and changing `execution_state[i]`.

A related, and commonly asked question is about pointers. If a pointer variable is part of a flush-set, *only* the pointer is synchronized, not the block of memory it points to.

The code fragment in Figure 1.8 is taken from the pipeline example presented and discussed in Section 1.3.2. The arrays used there implement dependences between activities. A much more robust and elegant solution, without the need for the `flush` directive, is provided by task dependences. This is discussed extensively in Chapter 3, where the same example is shown, but without explicit flush operations.

1.3 The Worksharing Constructs

A worksharing construct must be placed within a parallel region. Upon encountering a worksharing construct, the runtime system distributes the work to be performed over the threads active in the parallel region.

There are three worksharing constructs in C/C++/Fortran and a fourth one in Fortran. None of the worksharing constructs have a barrier on entry. There is one on exit, but it is omitted when using the `nowait` clause, which also means that there is no implied flush operation.

There are two important restrictions regarding worksharing constructs. First of all, either all threads in the team, or none at all, encounter the construct. As a consequence of this rule, threads are not allowed to skip a worksharing construct. The second rule is that the sequence of worksharing constructs and barriers encountered must be the same for all threads. If an OpenMP program does not comply with these rules, the behavior is undefined.

1.3.1 The Loop Construct

The loop construct provides for a straightforward way to assign the work associated with loop iterations to threads. The syntax for C/C++ and Fortran is shown in Figure 1.9.

The `schedule` clause is optional and may be used to explicitly specify the mapping of loop iterations onto threads. In OpenMP 2.5, there are three different scheduling types: `static`, `dynamic`, and `guided`. All three take an optional `chunk` parameter to control how many iterations are to be processed each time a thread gets assigned new work.

From a runtime overhead point of view, `static` is most efficient. The assignment of work to threads is pre-defined and very easy to determine at runtime. It works best with a regular workload, where each thread roughly gets assigned the same amount of work.

#pragma omp for *[clause[[,] clause]...] new-line* *for-loop(s)*
!$omp do *[clause[[,] clause]...]* *do-loop(s)* **[!$omp end do** *[nowait]***]**

Figure 1.9: **Syntax of the parallel loop construct in C/C++ and Fortran** – The loop iterations are distributed over the threads and executed in parallel. There are no curly braces in C/C++. The terminating `!$omp end do` on the Fortran directive is optional, but is recommended to clearly mark the end of the construct.

The other two types are more suitable in the case of a load imbalance, but they are more expensive in terms of overhead. The work assignment is handled at runtime and requires some locking. This is why these scheduling algorithms may not be the best choice if the imbalance is modest only.

A fourth keyword supported on this clause is `runtime`. As suggested by the name, the distribution is decided at runtime and controlled through the `OMP_SCHEDULE` environment variable. It is ideally suited to experiment with various policies and may even make this choice dependent upon the problem size and other characteristics.

If the schedule clause is not specified, the choice is made by the compiler and is therefore system-dependent.

OpenMP 3.0 and subsequent releases added more options and features to this construct. This includes an optional modifier on the `schedule` clause. For this and more details on the new features, refer to Section 2.1.1, starting on page 41.

1.3.2 The Sections Construct

Parallel sections provide for a very different structure to express and implement parallelism. Although generally applicable, this feature targets a higher level of parallelism. In Figure 1.10, the syntax in C/C++ and Fortran is shown for this directive.

Parallel sections are ideally suited to call different functions in parallel. In [2], it is explained how this is used to set up a pipeline to overlap I/O with computations. The assumption is that not all I/O needs to be completed before processing may start. In Figure 1.11, a code fragment based on this example is given. Three sections are used to implement the functionality needed.

```
#pragma omp sections [clause[[,] clause]...] new-line
{
    [#pragma omp section ]
        structured block
    [#pragma omp section
        structured block ]
    ...
}
```

```
!$omp sections [clause[[,] clause]...]
    [!$omp section ]
        structured block
    [!$omp section
        structured block ]
    ...
!$omp end sections [nowait]
```

Figure 1.10: **Syntax of the parallel sections construct in C/C++ and Fortran** – The number of sections both controls and limits the amount of parallelism. If there are "n" of these code blocks, at most "n" threads execute in parallel, but if nested parallelism is used, additional threads may be active.

The first section reads a block of data for each loop iteration. After completing the read for iteration `i`, a notification is posted. This is implemented in function `signal_read()`. Meanwhile, the second thread executes function `wait_read()`. It waits for a ready signal to be set. As soon as this is posted, the computations are performed. Once these have completed, another signal is set through function `signal_processed()`. The thread waiting in the third section picks this up, and starts the post-processing phase.

The threads communicate through flags set in memory and explicit flushes are needed to ensure threads see the modified values of the flags. This is illustrated in the code fragment shown earlier in Figure 1.8, which has been derived from the same pipeline example program.

Parallel sections are straightforward and easy to use, but they are most effective if there are only a relatively small number of sections. In cases where it is unknown upfront how many activities there are, or if the work performed is less regular, OpenMP tasks are more suitable. Tasks are covered extensively in Chapter 3, where this example is revisited. In Section 3.4, starting on page 128, it is shown that in

```
 1 #pragma omp parallel sections
 2 {
 3   #pragma omp section
 4   {
 5     for (int i=0; i<n; i++) {
 6        (void) read_input(i);
 7        (void) signal_read(i);
 8     }
 9   }
10   #pragma omp section
11   {
12     for (int i=0; i<n; i++) {
13        (void) wait_read(i);
14        (void) process_data(i);
15        (void) signal_processed(i);
16     }
17   }
18   #pragma omp section
19   {
20     for (int i=0; i<n; i++) {
21        (void) wait_processed(i);
22        (void) post_process_results(i);
23     }
24   }
25 } // End of parallel sections
```

Figure 1.11: **Example of the use of parallel sections** – A pipeline structure is set up to overlap I/O, computations, and post-processing.

this case, OpenMP tasks provide a more natural and elegant solution, because the arrays used for the signaling are not needed when using tasks. This eliminates the need for the explicit use of the `flush` construct.

1.3.3 The Single Construct

Sometimes there is a need within a parallel region to perform an activity by one thread only, such as when performing I/O for example.

#pragma omp single *[clause[[,] clause]...] new-line* *structured block*
!$omp single *[clause[[,] clause]...]* *structured block* **!$omp end single** *[nowait,[copyprivate]]*

Figure 1.12: **Syntax of the single construct in C/C++ and Fortran** – Only one thread executes the structured block. Unlike with the `master` construct, there is an implied barrier at the end.

The parallel region could be split into two separate regions and the master thread assigned to handle this single thread task, but this is not ideal for performance and requires more coding. The `single` construct is ideally suited for situations like this. It is not known which thread performs the work, which may vary from run to run. This is not an issue because there is no need to distinguish which thread does the work. The syntax in C/C++ and Fortran for this directive is given in Figure 1.12.

What is important, is that there is an implied barrier at the end of this construct. Regardless of which thread executes the code, all other threads wait for this thread. This is why the `nowait` clause may be useful, but be careful not to introduce a data race. If the `single` region is used to initialize data or to read from a file and the `nowait` clause is used, care needs to be taken that the other threads do not use this data before it has been initialized. There is also no implied `flush` construct. If the `nowait` clause is used, an explicit barrier, placed prior to the point where the data is used, guarantees the operations are completed and the data synchronized before execution continues.

This construct has a specific and unique clause called `copyprivate (list)`. This clause ensures all threads have a copy of the variables specified in the list. The values are copied into the private instance of the same variable. All threads wait in the single region until all copies have completed. This is why this clause is not allowed in combination with the `nowait` clause.

Often, the same may be achieved using shared variables. Each thread automatically has access to these, but this clause is more than a convenience. In the case of a recursion, it is more complicated to handle this explicitly, and the `copyprivate` clause provides a convenient solution.

```
!$omp workshare
      structured block
!$omp end workshare [nowait]
```

Figure 1.13: **Syntax of the workshare construct in Fortran** – This construct is used to parallelize (blocks of) statements using array-syntax.

1.3.4 The Fortran Workshare Construct

The fourth and last worksharing construct is available in Fortran only. This is because it has been designed to parallelize array syntax, a language feature not supported in C/C++. The syntax is shown in Figure 1.13. The only clause supported is the `nowait` clause.

1.3.5 The Combined Worksharing Constructs

The combined constructs are shortcuts and useful in the event there is only a single worksharing construct in a parallel region. The semantics are identical to the version with one worksharing construct within a parallel region. The syntax in C/C++ and Fortran is given in Figures 1.14 and 1.15.

Clauses from both directives may be used, but only if there is no conflict.[8] The application is an illegal OpenMP program if the behavior depends on whether the clause was applied to either the parallel construct, or the worksharing construct.

The combined constructs are not only convenient and easier to read, there is also a (potential) performance difference. To start with, there is only one implied barrier at the end. The compiler may also generate more efficient code. A general parallel region provides significant flexibility, but this may cost some performance. The combined constructs are fairly simple from an OpenMP point of view and this might be exploited by compilers.

Although it may not seem meaningful to embed a single threaded code block in a parallel region, and there is indeed no combined construct for this, this combination actually does have an interpretation in the context of OpenMP tasking. This is explained in detail in Chapter 3.

[8] The `if` clause has been extended to support the name of the directive. See also Section 2.1.2.

Full version	Combined construct

#pragma omp parallel
{
  #pragma omp for
  for-loop
}
	#pragma omp parallel for
	  for-loop

#pragma omp parallel
{
  #pragma omp sections
  {
  [**#pragma omp section**]
  structured block
  [**#pragma omp section**
  structured block]
  ...
  }
}
	#pragma omp parallel sections
	{
	  [**#pragma omp section**]
	  structured block
	  [**#pragma omp section**
	  structured block]
	  ...
	}

!$omp parallel
  !$omp do
  do-loop
  [**!$omp end do**]
!$omp end parallel
	!$omp parallel do
	  do-loop
	!$omp end parallel do

!$omp parallel
  !$omp sections
  [**!$omp section**]
  structured block
  [**!$omp section**
  structured block]
  ...
  !$omp end sections
!$omp end parallel
	!$omp parallel sections
	  [**!$omp section**]
	  structured block
	  [**!$omp section**
	  structured block]
	  ...
	!$omp end parallel sections

Figure 1.14: **Syntax of the combined worksharing constructs in C/C++ and Fortran** – The combined constructs may have a performance advantage over the more general parallel region with only a single worksharing construct embedded. This is because a parallel region has some overhead the compiler may now eliminate. The Fortran combined workshare construct is listed in Figure 1.15.

Full version	Combined construct

!$omp parallel
  !$omp workshare
  structured block
  !$omp end workshare
!$omp end parallel
	!$omp parallel workshare
	  structured block
	!$omp end parallel workshare

Figure 1.15: **Syntax of the combined workshare construct in Fortran** – The combined construct may have a performance advantage.

#pragma omp master *new-line*   *structured block*
!$omp master   *structured block* **!$omp end master**

Figure 1.16: **Syntax of the master construct in C/C++ and Fortran** – The master thread is guaranteed to execute the structured block.

1.4 The Master Construct

Although not a worksharing construct, the `master` construct is very closely related to the `single` worksharing construct. The syntax in C/C++ and Fortran is shown in Figure 1.16.

As suggested by the name, the master thread is guaranteed to execute this block of code. It may be seen as a special case of the `single` construct, but there is one important difference. In contrast with the `single` construct, the `master` construct does *not* have an implied barrier on exit.

Depending on the operations performed in the master region, this may give rise to a data race. If the master thread initializes or reads shared data, and there is no barrier before the other threads use this data, a data race occurs. This construct does not have an implied `flush` construct either.

1.5 Nested Parallelism

For certain algorithms, it is natural to use *nested parallelism*. With this, each thread within a parallel region initiates a second parallel region. If needed, this repeats to the nesting level desired.

```
1 #pragma omp parallel num_threads(3)
2 {
3    <code in here is executed by 3 threads>
4    #pragma omp parallel num_threads(2)
5    {
6        <code executed by 3x2 = 6 threads>
7    } // End of second level parallel region
8 } // End of first level parallel region
```

Figure 1.17: **Code fragment for a two-level nested parallel region** – There are two parallel regions. Each region may have a different number of threads. In this example, they are set through the `num_threads` clause.

Nested parallelism is achieved by nesting the `parallel` constructs. At runtime, each thread that encounters the next parallel region creates a new parallel region. This happens as often as there are nested parallel region constructs.

In Figure 1.17, the code fragment for a two-level nested parallel region in C/C++ is shown. The dynamic behavior is illustrated in Figure 1.18.

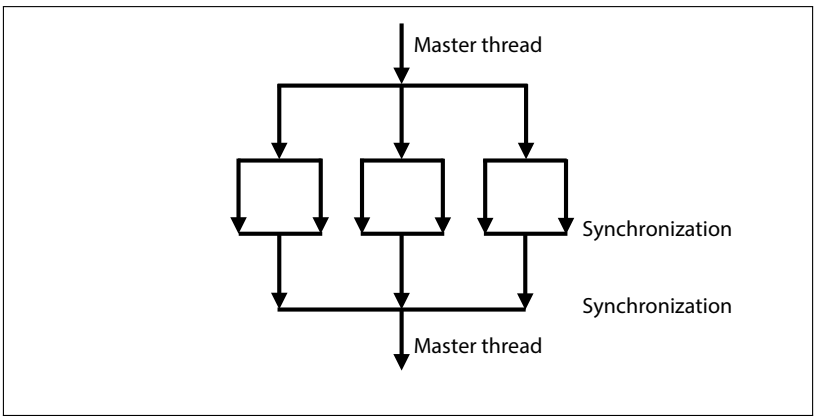

Figure 1.18: **An example of nested parallelism** – The master thread starts the program. Three threads are active in the first parallel region. Each one of these starts a separate parallel region. Because of the `num_threads(2)` clause on the second level parallel region, a total of 6 threads is active at this level.

In the example, three threads are active in the first parallel region. Because the `num_threads(2)` clause is used on the second parallel region, each of these threads creates a parallel region with two threads. At that point, a total of six threads are active.

This also demonstrates that thread teams do not need to be of the same size at every nesting level. A thread initiating another parallel region may set the number of threads through the `omp_set_num_threads()` function, or the `num_threads` clause. In case an `if` clause is used, and it evaluates to `false`, only one thread executes this parallel region.

At each nesting level, all parallel regions have their own master thread. As a consequence, the thread numbers are the same for all parallel regions at the same level. In this example, the second level threads in a team are numbered zero and one.

This is usually not a problem, but if a unique thread number is needed, the new runtime functions related to nested parallelism may be used to construct a thread ID that is unique across all nesting levels. These functions were introduced in OpenMP 3.0 and are discussed in Section 2.3.1 of Chapter 2. In Section 2.3.5, under "Nested Parallelism Revisited," it is shown how to use some of these functions to construct a unique thread ID across all nesting levels.

A common mistake made with nested parallelism is to try to nest *worksharing* constructs, which is not allowed. The nesting is at the level of the parallel region. It is valid to use the combined parallel worksharing constructs and nest them, because these are parallel regions after all. In Figure 1.19, this is illustrated with a parallel section that includes a parallel `for`-loop.

The three parallel sections execute simultaneously, assuming sufficient threads are available. If nested parallelism is enabled, the parallel for-loop at lines $8 - 13$ is executed by four threads and in total, six threads are active then.

Nested parallelism is an optional feature. An implementation is free to not support this, effectively ignoring the nested parallel regions. This feature may also be enabled, or disabled, either from within the application through the `omp_set_nested()` function, or by setting the `OMP_NESTED` environment variable. Keep in mind that the default setting for this variable is implementation-dependent. This means that nested parallelism may be turned off by default.

Nested parallelism definitely has its value, but it is rather rigid and the cost of the implied barrier that comes with each parallel region quickly adds up. In many cases, OpenMP tasks, covered extensively in Chapter 3, provide a much more convenient, flexible, and efficient mechanism.

```
 1 pragma omp parallel sections
 2 {
 3   #pragma omp section
 4   { printf("I am section 1\n"); }
 5   #pragma omp section
 6   {
 7      printf("I am section 2\n");
 8      #pragma omp parallel for shared(n) num_threads(4)
 9      for (int i=0; i<n; i++)
10      {
11          printf("Section 2:\tIteration = %d Thread ID = %d\n",
12                  i, omp_get_thread_num());
13      } // End of parallel for loop
14   }
15   #pragma omp section
16      { printf("I am section 3\n"); }
17
18 } // End of parallel sections
```

Figure 1.19: **Example of nested combined parallel worksharing constructs** – Three parallel sections are used. If sufficient threads are available, they are executed simultaneously. The second section includes another parallel region, consisting of a single `for`-loop. This loop is executed in parallel, resulting in nested parallelism. In this case the work in the loop is distributed over four threads.

1.6 Synchronization Constructs

In this section, several synchronization constructs are presented and discussed. They are used to control the behavior of the threads and it is quite likely an application needs at least one of them.

1.6.1 The Barrier Construct

The `barrier` construct forces all threads to wait until all of them have reached the barrier region in the program. Program execution continues once the last thread has arrived. The syntax in C/C++ and Fortran is given in Figure 1.20.

The barrier is used, for example, to avoid that a thread prematurely accesses data that is defined by another thread. This avoids data races and is illustrated in

#pragma omp barrier *new-line*
!$omp barrier

Figure 1.20: **Syntax of the barrier construct in C/C++ and Fortran** – All threads wait in the barrier until the last one has arrived.

```
 1 #pragma parallel shared(n,a,b)
 2 {
 3    #pragma omp for
 4    for (int i=0; i<n; i++)
 5        a[i] = i;
 6
 7    // The implied barrier on the for-loop above is needed there
 8
 9    #pragma omp for nowait
10    for (int i=0; i<n; i++)
11        b[i] += a[i];
12 } // End of parallel region
```

Figure 1.21: **Example to demonstrate the need for a barrier** – The implied barrier on the first loop is needed to avoid a data race on array element `a[i]`.

the code fragment in Figure 1.21. The implied barrier on the first parallel loop is needed to avoid a data race when reading array element `a[i]` at line 11. Without the implied barrier on the first for-loop, there is no guarantee that the value of `a[i]`, read in the second loop, has been updated.

The parallel region has an implied barrier upon exit. This is why the `nowait` clause at line 9 may be used to avoid two back-to-back barriers. Although the time spent in the second barrier should be short, there is still an additional cost that is best to avoid.

The implied flush operation that comes with the barrier is essential here. Without this, there is no guarantee that the correct value of `a[i]` is read in the second loop.

There is another possibility to fine-tune this code. The iteration space of both loops is identical, and the same elements of array `a` are accessed. Unfortunately in OpenMP 2.5, there is no guarantee that, even with static scheduling on loops with the same iteration space, the same thread operates on the same set of iterations across the loops. A data race may still occur if a `nowait` clause is used on the first loop.

```
1 #pragma parallel for shared(n,a,b)
2 for (int i=0; i<n; i++)
3 {
4     a[i] = i;
5     b[i] += a[i];
6 } // End of the parallel region
```

Figure 1.22: **Reworked example to eliminate a barrier** – By fusing (or merging) the two loops, one barrier is eliminated. On top of that, a combined worksharing construct is used.

This undesirable situation has been changed as of OpenMP 3.0. The `static` clause may now be used, because the mapping of iterations onto threads is preserved when using this scheduling type. This eliminates the need for the barrier, and a `nowait` clause may be used on the first loop. This is already more efficient, but in this case, another solution is preferred.

In addition to entirely avoid the barrier inside the parallel region, loop fusing creates more work per loop iteration, and cache line reuse improves. By using a combined worksharing construct, the execution time is reduced further. This approach is shown in Figure 1.22.

1.6.2 The Critical Construct

This is probably one of the most commonly used synchronization constructs, because it is ideally suited to avoid a data race when updating a shared variable.

A critical region is a block of code executed by all threads, but it is guaranteed that only one thread at a time may be active in the region. There is no implied

#pragma omp critical *[(name)] new-line* *structured block*
!$omp critical *[(name)]* *structured block* **!$omp end critical** *[(name)]*

Figure 1.23: **Syntax of the critical construct in C/C++ and Fortran** – The structured block is executed by all threads, but only one thread at a time executes the block. The construct may have an optional name.

```
 1 #pragma parallel
 2 {
 3              ...
 4    #pragma omp critical (c_region1)
 5    {
 6       sum1 += ...
 7    }
 8              ...
 9    #pragma omp critical (c_region2)
10    {
11       sum2 += ...
12    }
13                 ...
14 } // End of the parallel region
```

Figure 1.24: **An example using named critical regions** – Different threads may be in either one of the two different critical regions at the same time.

barrier on exit from the region, but there is an implied flush construct on entry and exit. The syntax in C/C++ and Fortran is given in Figure 1.23.

The main use of a critical region is to update a shared variable, because with this construct, it can never happen that two or more threads simultaneously update the same variable. This prevents a data race when performing an update.

All critical regions without a name are considered to have the same internal name and are dynamically ordered. Critical regions with different names are allowed to execute in parallel. This allows for advanced synchronization, but care needs to be taken that there are no unwanted side effects when updating shared data. In Figure 1.24, an example with two different critical regions is given.

If multiple named critical regions are used, different threads may simultaneously execute different regions. If a lexically later region uses a variable that is updated in an earlier region, a data race is introduced.

This example produces correct results as long as variable `sum1` is not used in the second critical region. If so, both regions should have the same name or no name at all.

In OpenMP 4.5, an optional *hint* is supported. This may be used on the `critical` construct to allow the implementation to tune the underlying code to the specific

#pragma omp atomic *new-line* *statement*
!$omp atomic *statement*

Figure 1.25: **Syntax of the atomic construct in C/C++ and Fortran** – The statement is executed by all threads, but the loads and stores of the updated variable are atomic. There are constraints regarding the type of expression.

access pattern of the critical region. More details are found in Sections 2.1.5 and 2.3.3 on pages 46 and 67, respectively.

1.6.3 The Atomic Construct

The `atomic` construct has been in the specifications from the very first OpenMP 1.0 release. The name is inspired by particle physics and chosen to reflect that an atomic operation cannot be further broken down into sub-operations. It is uninterruptible. The syntax in C/C++ and Fortran is given in Figure 1.25.

The atomic construct can be used to guarantee mutually exclusive access to a specific memory location, represented through a variable. The access to this location is guaranteed to be atomic.

Conceptually, the atomic construct is somewhat similar to a critical region, but the level of atomicity is finer grained: only the load and store instructions are atomic. By restricting the functionality, compilers may generate more efficient code using low-level atomic instructions supported by many processors.

A simple, but typical, example is given in Figure 1.26. Variable `x` is incremented by `1`. While a thread is performing this operation, no other thread can access `x`. The load and store instructions involved in the update are guaranteed to be atomic. This is to avoid a thread reading variable `x` before the thread that currently executes the update has completed the write.

In C/C++, the `atomic` construct may be used together with an expression statement to initialize, or update, a single variable, possibly through a simple binary operation. The supported operations are: `+, *, -, /, &, ^, |, <<, >>`.

In Fortran, the statement must also take the form of an update to a variable. The operator used for the update may be only one of `+, *, -, /, .AND., .OR., .EQV., .NEQV.`, and the intrinsic procedure `MAX`, `MIN`, `IAND`, `IOR`, or `IEOR`.

```
1 #pragma omp parallel
2 {
3      ......
4    #pragma omp atomic
5        x += 1;
6      ......
7 } // End of parallel region
```

Figure 1.26: **Example of the atomic construct** – The `atomic` construct at line 4 guarantees that the load and store instructions related to the update of variable `x` at line 5 are atomic.

There are several restrictions on the form that the expression may take: for example, it must not involve the variable on the left-hand side of the assignment statement. If the right-hand side includes a function call that is not thread-safe, the result is undefined.

Since OpenMP 3.1, this construct has been extended over the various releases and is quite comprehensive by now. Refer to Section 2.1.6 in Chapter 2 for an overview and more details.

1.6.4 The Ordered Construct

The ordered construct is applicable to parallel loops only. The construct must be placed within the parallel loop. The syntax in C/C++ and Fortran is given in Figure 1.27.

In addition to this construct, the `ordered` clause must be specified on the loop directive as well. The ordered region within the loop body is guaranteed to be executed in the order of the loop iterations, essentially serializing and ordering execution.

Valid use might be to enforce output to be printed in the original sequential order, or to debug a particular block of code. Because parallel processing changes the order of computations, floating-point round-off behavior could be different compared to the serial code. To verify whether this is the case, the `ordered` construct may be used. This construct may also be used to isolate a data race by narrowing down the region where this may occur. This construct is expensive and therefore should be used only if no suitable alternative is available.

#pragma omp ordered *new-line* *structured block*
!$omp ordered *structured block* **!$omp end ordered**

Figure 1.27: **Syntax of the OpenMP 2.5 ordered construct in C/C++ and Fortran** – This construct is placed within a parallel loop. The structured block is executed in the sequential order of the loop iterations. As of OpenMP 4.5, this construct has been enhanced to support additional features.

Function name	Description
`OMP_NUM_THREADS`	Set the number of threads used.
`OMP_SCHEDULE`	Set the runtime scheduling type and chunk size.
`OMP_DYNAMIC`	Enable/disable dynamic thread adjustment.
`OMP_NESTED`	Enable/disable support for nested parallelism.

Figure 1.28: **The OpenMP 2.5 environment variables** – These are the four variables one may set prior to program start up.

As of the OpenMP 4.5 specifications, the `ordered` construct has been enhanced to support `doacross` and `SIMD` loops. For details refer, to Sections 2.4.4 and 5.2.7, respectively.

1.7 The OpenMP 2.5 Environment Variables

In OpenMP 2.5, four environment variables are available to initialize the corresponding runtime settings. They are listed in Figure 1.28. If not set by the user, an implementation-dependent default is used. The exception is nested parallelism. It is turned off by default and may be activated by setting the `OMP_NESTED` variable to `true`.

As a general rule, a runtime function with the same functionality as an environment variable takes precedence. For example, by using the `omp_set_num_threads()` function, the initial value set by `OMP_NUM_THREADS` is overruled.

Since OpenMP 2.5, many new environment variables have been added. These are introduced and discussed in Section 2.2, starting on page 53. As explained there, variables `OMP_NUM_THREADS` and `OMP_SCHEDULE` are augmented to better support nested parallelism and additional workload scheduling features, respectively.

1.8 The OpenMP 2.5 Runtime Functions

Runtime functions may be used to query settings and also override initial values, either set by default or specified through environment variables defined prior to program start-up. Some of the runtime functions may also be used to modify settings while the application executes.

An example is the number of threads used to execute a parallel region. The initial value is implementation-dependent, but through the `OMP_NUM_THREADS` environment variable, this number may be set explicitly before the program starts. During program execution, function `omp_set_num_threads()` may be used to increase, or decrease, the number of threads to be used in the next parallel region(s).

When using the runtime functions in C/C++, file `omp.h` must be included. Fortran programs require either file `omp_lib.h` to be included or a module called `omp_lib` to be used. Either one of these, or both, are guaranteed to be available in Fortran.

Here, and in Section 2.3, the syntax is given for C/C++ only, but the usage in Fortran is very similar. Functions returning, or using, `true` or `false` as a function argument in C/C++, require the `LOGICAL` data type in Fortran. Many C/C++ functions are functions in Fortran as well, but in some cases they are subroutines instead. Refer to the specifications for details.

In Figure 1.29, the functions related to the execution environment are listed. Only the names and a short description are given. Most of these functions are straightforward to use, but there are a few things worth pointing out:

- When the `omp_get_num_threads()` function is used outside of a (nested) parallel region, a value of 1 is returned.
- The value returned by `omp_get_thread_num()` is relative to the parallel region that the thread executing this function is part of. As a result, it always starts at zero upon the execution of a new parallel region or when nested parallelism is used.
- The `omp_get_max_threads()` function returns the number of threads used in the next parallel region (unless the `num_threads` clause is used). This, for example, allows for the allocation of storage based on the number of threads in the parallel region.

Function name	Description
`omp_set_num_threads`	Set the number of threads.
`omp_get_num_threads`	Number of threads in the current team.
`omp_get_max_threads`	Number of threads in the next parallel region.
`omp_get_num_procs`	Number of processors available to the program.
`omp_get_thread_num`	Thread number within the parallel region.
`omp_in_parallel`	Check if within a parallel region.
`omp_get_dynamic`	Check if thread adjustment is enabled.
`omp_set_dynamic`	Enable, or disable, thread adjustment.
`omp_get_nested`	Check if nested parallelism is enabled.
`omp_set_nested`	Enable, or disable, nested parallelism.

Figure 1.29: **The OpenMP 2.5 runtime functions** – These are the functions to query or change settings related to threads, processors/cores, and the parallel execution environment. The names are the same in C/C++ and Fortran, but the usage follows the language syntax.

Function name	Description
`omp_get_wtime`	Absolute wall-clock time in seconds.
`omp_get_wtick`	The number of seconds between successive clock ticks.

Figure 1.30: **The two OpenMP 2.5 timing functions** – The first function is used to time specific portions of the program. The second function returns the clock resolution. It is used to determine what kind of time interval may be measured accurately.

In addition to this functionality, there are two functions to support measuring the execution time. The rationale to include these functions is to provide an accurate, as well as portable, way to measure the execution time of specific parts of the program. Although the specifications do not require this, the expectation is that the OpenMP implementation provides the most accurate timer available on a specific platform.

The two timer functions are listed in Figure 1.30 and are quite straightforward to use. The one thing to remember is that `omp_get_wtime()` returns the number of wall-clock or "elapsed" seconds. In other words, it returns an absolute value. A meaningful timing value is therefore obtained by taking the difference between two calls. This is illustrated in Figure 1.31.

The third set of runtime functions is related to locks. An overview of the locking functions is given in Figure 1.32. Locks provide a way to avoid that a block of code is executed by multiple threads at the same time. Conceptually this is very simple.

```
1 t_start = omp_get_wtime();
2    <code block to be timed>
3 t_wall_clock = omp_get_wtime() - t_start;
```

Figure 1.31: **An example how to use function omp_get_wtime()** – The value returned in variable `t_wall_clock` is the wall-clock time of the code block enclosed between the two calls to this timer function. To obtain accurate measurements, it is highly recommended for this time to significantly exceed the value returned by the `omp_get_wtick()` function.

A thread tests for the lock to be available. If this is not the case, the application needs to handle this. In most cases, it waits for the lock to become available but may also perform some other work and check again later.

Once the lock is owned by the thread, no other thread may access it. This allows the thread owning the lock to perform its work, without the risk that other threads execute the same block of code at the same time. When the thread has finished, it needs to release the lock in order for another thread to gain access to it. Failure to do so results in a deadlock.

Locking functionality is by no means unique to OpenMP, but by including it in the specifications, the user has the guarantee that it is available and a natural part of the OpenMP environment.

Many more runtime functions have been added since OpenMP 2.5. These may be found in Section 2.3, starting on page 60.

1.9 Internal Control Variables in OpenMP

Internal to the implementation, OpenMP uses a set of *Internal Control Variables* (*ICVs*) to access and control runtime settings. An example of this is the number of threads used in the next parallel region. The ICVs are an implementation detail, but must be mentioned, because in the specifications, some behavior is explained in terms of ICVs.

The names of the ICVs are given in the specifications, for example `nthreads-var`, but these are suggestions only. An implementation is free to deviate from this and by design there is no impact on the user, because an OpenMP application never uses ICVs directly. Instead, the interaction is through environment variables and runtime functions.

Function name	Description
`omp_init_lock`	Initialize a lock variable.
`omp_init_nest_lock`	Similar, but for a nestable lock.
`omp_set_lock`	Blocking request to acquire the lock.
`omp_set_nest_lock`	Similar, but for a nestable lock.
`omp_unset_lock`	Release the lock.
`omp_unset_nest_lock`	Similar, but for a nestable lock.
`omp_destroy_lock`	Change the state of the lock to be unitialized.
`omp_destroy_nest_lock`	Similar, but for a nestable lock.
`omp_test_lock`	Non-blocking request to acquire the lock.
`omp_test_nest_lock`	Similar, but for a nestable lock.

Figure 1.32: **The OpenMP 2.5 locking functions** – These functions define, acquire, and release a lock. A nestable lock may be set multiple times by the same thread.

Initial values are often system-dependent and pre-defined. They may, however, be set by the user through OpenMP environment variables. The corresponding ICV is initialized to this value by the implementation.

The number of threads is again a good example. The default is system-specific and the implementation assigns an initial value to the corresponding ICV. The value set for environment variable `OMP_NUM_THREADS` overrides this ICV upon program start up.

At runtime, OpenMP runtime functions are available to query the value of a specific ICV (for example, `omp_get_num_threads()`) and if needed, may be used to change the value. For example, function `omp_set_num_threads()` changes the number of threads at the user level. The implementation handles this by changing the value of the corresponding ICV.

1.10 Concluding Remarks

This chapter has provided a brief summary of OpenMP 2.5 and has set the foundation for the remainder of this book. At the time this release of OpenMP came out, it was a compact, yet very powerful, parallel programming model for the SMP systems of those days.

Since then, shared memory computer systems have evolved to contain hundreds of cores with multiple threads per core. Most of these systems use a cc-NUMA architecture and optionally support hardware accelerators.

Through the subsequent four releases of the specifications that came out after OpenMP 2.5, these very complex and extremely powerful systems are fully supported in OpenMP. In the remaining chapters, all of the enhancements and new features added are covered in great detail.

2 New Features in OpenMP

In May 2008, the specifications of OpenMP 3.0 were released. This was the first update since the 2.5 release and a major improvement. Many existing features were enhanced, support for C++ was substantially improved, and the concept of tasks was introduced.

Since then, OpenMP has continued to evolve. OpenMP 3.1 was released in July of 2011. This was an update release. Several constructs and features were enhanced, including tasking. The specifications were clarified where needed, but no significant new functionality was added.

New functionality was implemented in OpenMP 4.0. When it was released in July of 2013, OpenMP made a huge step forward. For the first time in the history of OpenMP, support for cc-NUMA architectures became available. Heterogeneous computing support was added also. This integrated very smoothly with the rest of the language and preserved the ease of use of OpenMP. Another noticeable addition was support for handling a failure in a parallel region, or task. In addition, tasking was expanded, vectorization through SIMD instructions was added and User-Defined Reductions (UDRs) were now supported as well.

A little over two years later, in November 2015, OpenMP 4.5 was released. It is another significant step forward with many enhancements to the existing functionality.

In this chapter, most of the features and enhancements, added since the release of OpenMP 2.5, are presented and discussed. The exceptions are tasking, thread affinity, SIMD, and heterogeneous architectures. These are major topics, covered in great detail in the chapters to follow.

2.1 Enhancements to Existing Constructs

In this section, the enhancements to the constructs supported in OpenMP 2.5 and earlier releases are introduced and discussed. Since this release, entirely new concepts and constructs have been introduced as well. An overview of these can be found in Section 2.4.

2.1.1 The Schedule Clause

The `schedule` clause has been extended to support additional functionality. The updated syntax is shown in Figure 2.1.

schedule *([modifier [, modifier]:] kind [, chunk_size])*

Figure 2.1: **Syntax of the extended schedule clause in C/C++ and Fortran** – In addition to the already supported types, the `auto` type for the workload distribution schedule has been added. Another new feature is the support for an optional modifier, or even two, to provide additional details in which way the chunks of work, if applicable, are distributed over the threads.

Modifier	Description
`monotonic`	Each thread executes its chunks in increasing logical iteration order. This is the order the iterations would be executed in if the loop was executed sequentially.
`nonmonotonic`	Chunks are assigned to threads in any order.
`simd`	The new chunk size is set to *chunk_size/simd_width*, with *simd_width* an implementation-defined value.

Figure 2.2: **The three modifiers supported on the schedule clause** – These modifiers specify how the chunks should be distributed over the threads.

As of OpenMP 3.0, in addition to the `static`, `dynamic`, and `guided` options for the workload distribution schedule for parallel loops, a fourth choice is supported. Through the `auto` keyword, the selection of the schedule is determined by the implementation.

In OpenMP 4.5, an optional keyword on the `schedule` clause was introduced to support one, or even two, *modifiers* to specify in which way the chunks should be distributed over the threads. This feature was introduced to resolve an ambiguity in the earlier specifications that could cause applications to make an incorrect assumption of how the work was distributed over the threads. With the introduction of the modifiers, this problem is no longer an issue.

The three new keywords to select the modifier, plus a brief description, are listed in Figure 2.2. If more details are needed, we recommend checking the specifications.

There are several restrictions when using these modifiers:

- The `nonmonotonic` modifier may be used only in combination with the `guided` or `dynamic` scheduling types.
- If the `static` schedule type, or the `ordered` clause, is specified without a modifier, the `monotonic` modifier is assumed. In other words, in these cases, `monotonic` is the default.

Important - With the next release of the OpenMP specifications, this default will change to `nonmonotonic`. Any application relying on the monotonic behavior in these cases should set this explicitly.

- The `monotonic` and `nonmonotonic` modifiers are mutually exclusive. This implies that, when applicable, either one of these may be used in combination with the `simd` modifier only.
- The `nonmonotonic` modifier may not be specified in combination with the `ordered` clause.

2.1.2 The If Clause

With combined constructs, it is not always clear to which part(s) the `if` clause applies. This is why the clause has been extended with an optional name, formally called the *directive-name-modifier*, to specify to which construct it applies.

For example, when using tasking in a combined construct, the following makes it clear the clause applies to the tasking part of the construct: `if(task: n < 10)`.

2.1.3 The Collapse Clause

A short loop has a downside in a serial program, because the "loop overhead" dominates the performance if there is not much work performed per loop iteration. In a parallel program, the overhead is even worse.

A short loop not only limits the number of threads that may be used, it also easily leads to a load imbalance if the loop trip count is not a multiple of the number of threads. Depending on the amount of work performed per iteration, the parallel overhead may dominate as well.

Consider the serial code fragment in Figure 2.3, where a linearized 2D mxn array `a` is initialized. Both loops can be parallelized, but unless nested parallelism is used, only one loop may be selected for parallelization. The inner loop is longest, but for performance reasons it is not a good idea to select it for parallelization, because the parallel loop overhead is incurred "m" times. If the outer loop is parallelized, only two threads may be used.

There does not seem to be an easy way out of this dilemma. The only solution is to merge or "collapse" the two loops into one single and longer loop. This new loop has the length of both loops combined and may be parallelized. This transformation comes at the price of some additional computations to reconstruct the original loop

```
1 int m = 2;
2 int n = 5;
3      ........
4 for (int i=0; i<m; i++)
5   for (int j=0; j<n; j++)
6      a[i*n+j] = i+j+1;
```

Figure 2.3: **Example of a nested loop with short loops** – Both loops can be parallelized, but neither of the choices results in efficient code or substantial speed ups.

```
1 for (int k=0; k<m*n; k++)
2 {
3    int i = (k/n) % m;
4    int j = k % n;
5    a[i*n+j] = i+j+1;
6 }
```

Figure 2.4: **An example of explicit loop collapsing** – At the cost of some additional book-keeping operations, the two short loops are combined into a single longer loop that can be parallelized and executed more efficiently.

iteration values from the single loop variable. The code fragment that implements this idea is shown in Figure 2.4.

This optimization is not only beneficial in a serial program. Both issues when parallelizing the original loop nest have been eliminated as well. In general, and as demonstrated in this example, loop collapsing provides a good way to deal with short nested parallel loops, but it does not make the code any easier to read and maintain.

This is why the `collapse` clause is useful. It is ideally suited in the event a perfectly nested loop has more than one parallelizable loop, but all of these loops are short. The syntax of the clause is given in Figure 2.5.

Figure 2.6 shows how to use the clause in this example. The compiler first generates code similar to what is shown in Figure 2.4. The resulting single loop is then parallelized.

To demonstrate how this works, we parallelized the outer loop in Figure 2.3 and executed both versions using five threads. The loops were instrumented to print diagnostic information. The results are listed in Figure 2.7.

#pragma omp for collapse *(n) [clause[[,] clause]...] new-line* *for loop(s)*
!$omp do collapse *(n) [clause[[,] clause]...]* *do loop(s)* **!$omp end do** *[nowait]*

Figure 2.5: **Syntax of the loop collapse clause in C/C++ and Fortran** – This clause combines multiple loops into a single loop. The length of the resulting parallel loop is the combined length of the loops affected. The parameter `n` is used to specify how many loops should be collapsed.

```
1 #pragma omp parallel for default(none) shared(a,m,n)\
2                          collapse(2)
3 for (int i=0; i<m; i++)
4   for (int j=0; j<n; j++)
5      a[i*n+j] = i+j+1;
```

Figure 2.6: **Example of the `collapse` clause** – The compiler generates a single loop of length mxn and parallelizes it.

The output lines marked "Outer loop" are from the original loop nest, where the outer loop was parallelized. In this case only two threads may be used, although we requested five threads. This limitation has been lifted for the version that uses the `collapse` clause. The output lines are marked "Collapse" and clearly all five threads are used now.

There are some things to remember when using the `collapse` clause:

- This clause is supported on the `worksharing` loop construct, the `simd` construct, and the `distribute` construct.
- The collapsed loops must be perfectly nested.
- The collapsed loops must form a rectangular iteration space.
- The bounds and stride of each loop must be invariant over all the loops.
- Only one `collapse` clause may be used per loop nest.
- The iterations of the collapsed loop are scheduled according to the `schedule` clause in effect.

```
Outer loop: (0,0) has been initialized to 1 by thread 0
Outer loop: (0,1) has been initialized to 2 by thread 0
Outer loop: (0,2) has been initialized to 3 by thread 0
Outer loop: (0,3) has been initialized to 4 by thread 0
Outer loop: (0,4) has been initialized to 5 by thread 0
Outer loop: (1,0) has been initialized to 2 by thread 1
Outer loop: (1,1) has been initialized to 3 by thread 1
Outer loop: (1,2) has been initialized to 4 by thread 1
Outer loop: (1,3) has been initialized to 5 by thread 1
Outer loop: (1,4) has been initialized to 6 by thread 1

Collapse:   (0,2) has been initialized to 3 by thread 1
Collapse:   (0,3) has been initialized to 4 by thread 1
Collapse:   (0,0) has been initialized to 1 by thread 0
Collapse:   (0,1) has been initialized to 2 by thread 0
Collapse:   (1,3) has been initialized to 5 by thread 4
Collapse:   (1,4) has been initialized to 6 by thread 4
Collapse:   (0,4) has been initialized to 5 by thread 2
Collapse:   (1,0) has been initialized to 2 by thread 2
Collapse:   (1,1) has been initialized to 3 by thread 3
Collapse:   (1,2) has been initialized to 4 by thread 3
```

Figure 2.7: **Output of the original code and the version with the collapse clause** – This example was executed using five threads, but in the first version only two threads may be used. The second version uses all five threads.

2.1.4 The Linear Clause

The `linear` clause is mentioned here only for the sake of completeness. It is part of the support for SIMD instructions and covered in detail in Section 5.2.4 on page 230.

2.1.5 The Critical Construct

The `critical` construct has been enhanced with an optional hint. The syntax is given in Figure 2.8.

If a hint is used, the construct *must* have a name and the expression for the hint must evaluate to the same value for all critical regions with the same name.

#pragma omp critical *[(name) [* `hint` *(hint-expression)]] new-line* *structured block*
!$omp critical *[(name) [* `hint` *(hint-expression)]]* *structured block* **!$omp end critical** *[(name)]*

Figure 2.8: **Syntax of the extended critical construct in C/C++ and Fortran** – The hint is optional, but if used, the critical construct must have a name and the expression for the hint must evaluate to the same value for all critical regions with the same name.

The purpose and use of the hint is the same as described in Section 2.3.3 on page 67 and allows an implementation to tailor the code to the expected access pattern of the critical region.

The expression for the hint must be of type `omp_lock_hit_kind` and must evaluate to a scalar value that is a valid lock hint. The supported values are listed in Figure 2.20 on page 68. Without a hint, the effect is as if `hint(omp_lock_hint_none)` was specified.

Because locks and critical regions potentially limit scalability, choosing the correct value for the hint can make a noticeable difference and is certainly worth considering.

2.1.6 The Atomic Construct

In earlier versions of the specifications, the `atomic` construct applied to a single update statement only (see also Section 1.6.3 on page 32), but as of OpenMP 3.1 it has been refined to support several additional access and update patterns. It is now also possible to capture the value of the target variable before, or after an atomic update. The syntax of the extended construct is given in Figure 2.9.

Through the optional *atomic-clause*, the type of modification performed is specified. In absence of a clause, the `update` clause is implied. This is consistent with the earlier definition.

The names of the clauses allude to their respective functionality:

- **Read** - This is an atomic read. The input variable is read atomically and stored into the output variable. This is guaranteed regardless of the size of the variable.

#pragma omp atomic *[*`seq_cst`*[,]]* *atomic-clause* *[[,]*`seq_cst`*]* *new-line*
expr. statement
#pragma omp atomic *[*`seq_cst`*]* *new-line*
expr. statement
#pragma omp atomic *[*`seq_cst`*[,]]* `capture` *[[,]*`seq_cst`*]* *new-line*
structured block

!$omp atomic *[*`seq_cst`*[,]]* `[read|write|update]` *[[,]*`seq_cst`*]*
capture, write, or update statement respectively
!$omp end atomic
!$omp atomic *[*`seq_cst`*]*
update statement
!$omp atomic *[*`seq_cst`*[,]]* `capture` *[[,]*`seq_cst`*]*
update+capture, capture+update or capture+write statement
!$omp end atomic

Figure 2.9: **Syntax of the extended atomic construct in C/C++ and Fortran** – The type of atomic operation performed can be specified. If necessary, sequential consistency may be enforced by using the `seq_cst` keyword.

- **Write** - This is an atomic write. The output variable is written atomically. This is guaranteed regardless of the size of the variable.
- **Update** - The same variable is both read and written. As before, only the read/write operations are guaranteed to be atomic. If, for example, an expression is used, the evaluation of this need not be atomic. The consequence is that any possible side effect(s) of such an evaluation could lead to an erroneous result. An example is a call to a function that updates a shared variable. If no precautions are taken, a data race may occur. Task scheduling pointss are not allowed between the read and write of the variable. See also page 144 in Section 3.7 for more information on task scheduling points.
- **Capture** - In addition to an update, the value of the modified variable is saved *atomically* either before or after the update.

  There is a subtle, but important, distinction between the two forms of the `capture` clause in C/C++. The first form supports a single statement only, while the second form is somewhat more general and not only supports the atomic update, but also a (simple) statement to capture the value of the variable modified. The details for this are given in Figure 2.11.

Clause	Expr. STMT (C/C++)	Expr. STMT (Fortran)
`read`	`v = x;`	`v = x`
`write`	`x = expr;`	`x = expr`

`update`	`x++;`
	`++x;`
	`x--;`
	`--x;`
	`x binop = expr;`
	`x = x binop expr;`
	`x = expr binop x;`
		`x = x operator expr`
		`x = expr operator x`
		`x = intrinsic_func(x,expr_list)`
		`x = intrinsic_func(expr_list,x)`

Figure 2.10: **Clauses and expression statements supported on the atomic construct** – This table lists the various combinations of clauses and statements one can use with the `read`, `write`, and `update` clauses. The `capture` clause is covered separately.

In Figure 2.10, the expression statements in C/C++ and Fortran supported with the `read`, `write` and `update` clauses are given. In this Figure, *binop* is one of the following operators: `+`,`-`,`*`,`/`, `&`,`^`, `|`, `<<`, or `>>`.

In C++, `binop`, `binop =`, `++`, and `--` cannot be overloaded operators. In Fortran, the operator is `+`, `-`, `*`, `/`, `.AND.`, `.OR.`, `.EQV.`, or `.NEQV.`. The intrinsic function is one of `MAX`, `MIN`, `IAND`, `IOR`, or `IEOR`.

The fine-print in the specifications on the combined use of an operator and an expression is not always easy to digest. With the following simple observation in mind, it is however hopefully easier to use this construct correctly.

All one needs to remember is that the loads and stores are executed atomically and whatever is contributed to the target variable should have no side effects regarding this variable. Whether it is a function call or an expression, evaluating this should not have an impact on the value of the variable to be updated.

Either the standard precedence rules apply, or parentheses may be used to enforce that the expression is evaluated prior to the update. For example, a statement of the form `x = x binop expr` must be equivalent to `x = x binop (expr)`.

The following example illustrates this: `x = x + y*y`. Because the multiplication is guaranteed to be performed prior to the addition, there is no parentheses. If this is not the case, or when in doubt, parentheses may be used to eliminate any ambiguity. For example, in an update of this type, the parentheses enforce the precedence: `x = x + (z + y*y)`.

Clause	Expr. STMT (C/C++)	Structured Block (C/C++)

`capture`	`v = x++;`
	`v = ++x;`
	`v = x--;`
	`v = --x;`
	`v = x` *`binop`* `= expr;`
	`v = x = x` *`binop expr`*`;`
	`v = x =` *`expr binop`* `x;`
		{`v = x; x` *`binop`* `= expr;`}
		{`x` *`binop`* `= expr; v = x;`}
		{`v = x; x = x` *`binop expr`*`;`}
		{`v = x; x =` *`expr binop`* `x;`}
		{`x = x` *`binop expr`*`; v = x;`}
		{`x =` *`expr binop`* `x; v = x;`}
		{`v = x; x =` *`expr`*`;`}
		{`v = x; x++;`}
		{`v = x; ++x;`}
		{`++x; v = x;`}
		{`x++; v = x;`}
		{`v = x; x--;`}
		{`v = x; --x;`}
		{`--x; v = x;`}
		{`x--; v = x;`}

Figure 2.11: **Expression statements and structured blocks supported on the capture clause in C/C++** – With this construct, the value of the target variable is saved into another variable, either before or after the update.

While the other three clauses are refinements on the `atomic` construct prior to the OpenMP 3.1 specifications, the `capture` clause *extends* the functionality. With this clause, the value of the target variable may be saved into another variable before, or after, the update.

Due to syntactical differences between C/C++ and Fortran, the overview for the `capture` clause is split. In Figure 2.11, the supported statements and structured blocks for C/C++ are given. In Figures 2.12 and 2.13, the Fortran case is covered.

Why was there a need for the `capture` clause? Prior to the introduction of this feature there was no way to *atomically* intercept the value of the target variable. This could easily lead to a data race. If one thread accesses this variable outside the atomic construct, while another thread performs the atomic update, the result is undefined.

The example in Figure 2.14 shows such a case and demonstrates how to use the `capture` clause to avoid this problem. In this program, function `update` adds a certain contribution to a shared variable. We want to preserve the previous value and use it as the return value of this function.

Clause	Update STMT (Fortran)

`capture`	`x = x` *`operator expr`*
	`x =` *`expr operator`* `x`
	`x =` *`intrinsic_function`*`(x,`*`expr_list`*`)`
	`x =` *`intrinsic_function`*`(`*`expr_list`*`,x)`

Figure 2.12: **The update statements supported on the capture clause in Fortran** – The intrinsic function is one of the following: `min`, `max`, `iand`, `ior`, or `ieor`.

Clause	Capture STMT (Fortran)	Write STMT (Fortran)
`capture`	v = x	x = *expr*

Figure 2.13: **The capture and write statement supported on the capture clause in Fortran** – These are the various possibilities to choose from when using this clause.

The main program calls this function from within a parallel region and sets the contribution to the thread ID plus 1. The addition of 1 makes the output in Figure 2.15 easier to follow. The final result should be $1+2+\ldots+NT+(NT+1) = NT*(NT+1)/2$ if `NT` threads are used.

The output in Figure 2.15 demonstrates how the value is carried over from one thread to another.

Note there is one issue here. Printing variable `new_value` within the parallel region introduces a data race, because the value as modified by one thread could have been modified by another thread before the print statement is executed. There is no such risk with the old value, because it is stored in a private variable and performs an atomic read of the new value.

There is one feature that has not been discussed so far: the `seq_cst` clause. This has mainly been introduced to provide the same semantics as the `memory_order_seq_cst` and `memory_order_relaxed` atomic operations in the C11 and C++11 standards [4, 5].

In OpenMP, adding the `seq_cst` clause to the `atomic` construct implies a `flush` operation without a list. This guarantees sequential consistency but comes at a price. As discussed in section 1.2.3 on page 11, this consistency model limits potentially significant compiler optimizations and most likely negatively impacts performance.

```
 1  int main(int argc, char *argv[])
 2  {
 3    int new_value = 0;
 4
 5    #pragma omp parallel shared(new_value)
 6    {
 7       int TID       = omp_get_thread_num();
 8       int old_value = update(&new_value, TID+1);
 9
10       printf("TID = %4d: old_value = %4d\n",TID,old_value);
11
12    } // End of parallel region
13
14    printf("Final value is %4d\n",new_value);
15
16    return(0);
17  }
18  int update (int *new_value, int contribution)
19  {
20     int return_value;
21
22     #pragma omp atomic capture
23     {
24        return_value = *new_value;
25        *new_value  += contribution;
26     } // End of atomic capture
27
28     return(return_value);
29  }
```

Figure 2.14: **Example of the capture clause on the atomic construct** – Prior to OpenMP 3.1 it was not possible to reliably save the old value before the update. The `capture` clause used at line 22 addresses this issue. Before variable `new_value` is updated at line 25, its value is stored into variable `return_value` (line 24). These are atomic operations, so no other thread may interfere. The final result of this program is `NT*(NT+1)/2` if `NT` threads are used.

```
TID =    0: old_value =    5
TID =    1: old_value =    0
TID =    2: old_value =    2
TID =    3: old_value =   11
TID =    4: old_value =    6
TID =    5: old_value =   15
TID =    6: old_value =   29
TID =    7: old_value =   21
Final value is   36
```

Figure 2.15: **Sorted output of the update function that uses the atomic capture** – This result was obtained using 8 threads. Thread 1 happens to be the first one to perform the update. It adds 2 to it, and this new value is then picked up by thread 2, which adds 3 to it. The value is then subsequently updated by threads $0, 4, 3, 5, 7$ and 6. The final result is indeed $8 * 9/2 = 36$.

Hence the recommendation to use this clause only if really needed and no suitable alternative is available.

Many aspects of the `atomic` construct have been covered here, but there are some things glossed over. We recommend checking the specifications for all details.

2.2 New Environment Variables

In OpenMP 2.5 there were only four environment variables. Since then, the list given in Figure 1.28 on page 34 has been significantly expanded. These new variables standardize the typical extensions that most compiler vendors have provided already and also support the new functionality introduced since OpenMP 2.5.

In addition to this, the already existing variables `OMP_NUM_THREADS` and `OMP_SCHEDULE` are extended to support additional functionality.

These two, plus all additional environment variables supported in OpenMP 4.5, are listed in Figure 2.16.

Below, these new environment variables are covered in more detail. As before, the names must be in uppercase, but the value(s) assigned are case-insensitive. In our examples, we use lowercase for the values.

Please keep in mind that, in all cases, the default is system-dependent. Also, as with any environment variable, the way to set these depends on the Operating System used and/or the specific (Unix) shell.

Function name	**Description**
OMP_DISPLAY_ENV	If set, displays the OpenMP version number and ICV values. In verbose mode, more information is shown.
OMP_NUM_THREADS	Set the number of threads for (nested) parallel regions.
OMP_SCHEDULE	Set the workload distribution schedule for parallel loops.
OMP_STACKSIZE	Set the size of the stack for the threads.
OMP_WAIT_POLICY	Specify the behavior of idle threads.
OMP_MAX_ACTIVE_LEVELS	Set the maximum number of nested active parallel regions.
OMP_THREAD_LIMIT	Set the maximum number of threads to be created in a contention group.
OMP_MAX_TASK_PRIORITY	Set the maximum value that can be used in the `priority` clause on the `task` construct.
OMP_PROC_BIND	Set and control thread affinity.
OMP_PLACES	Define where threads are allowed to execute.
OMP_CANCELLATION	Enable or disable cancellation.
OMP_DEFAULT_DEVICE	Set the default device number.

Figure 2.16: **The additional OpenMP 4.5 environment variables** – These are the environment variables changed or added since OpenMP 2.5.

- `OMP_DISPLAY_ENV` - This is a very useful variable to verify the settings and check the defaults. Each environment variable and macro is printed in the format `ENV_VAR_NAME = 'value'`. Optionally a line starts with `[device-id]`.

  The information is printed after the environment variables have been processed, but before any of the corresponding ICVs are modified. The information is enclosed between the strings `OPENMP DISPLAY ENVIRONMENT BEGIN` and `OPENMP DISPLAY ENVIRONMENT END`.

  There are three choices for this variable:

  - `false` - No information is displayed.
  - `true` - Print the OpenMP version number and the values of all environment variables. These are either the defaults, or the value(s) set by the user prior to program start up.

  - `verbose` - In addition to the standard environment variables, vendor-specific extensions may be printed as well. It is up to the implementor to select which environment variables to include.

  Below is an example after setting `OMP_DISPLAY_ENV` to `true`. This is printed immediately after program start up:

```
OPENMP DISPLAY ENVIRONMENT BEGIN
  _OPENMP='201307'
  OMP_CANCELLATION='FALSE'
  OMP_DISPLAY_ENV='true'
  OMP_DYNAMIC='TRUE'
  OMP_MAX_ACTIVE_LEVELS='4'
  OMP_NESTED='FALSE'
  OMP_NUM_THREADS='16'
  OMP_PLACES='N/A'
  OMP_PROC_BIND='FALSE'
  OMP_SCHEDULE='static'
  OMP_STACKSIZE='8388608B'
  OMP_THREAD_LIMIT='1024'
  OMP_WAIT_POLICY='PASSIVE'
OPENMP DISPLAY ENVIRONMENT END
```

  One of the things to observe in this output is that the settings for `OMP_PLACES` are "N/A," which stands for "Not Applicable." This is because processor binding is apparently disabled (`OMP_PROC_BIND` has been set to `false`). This illustrates how useful this environment variable is to perform a sanity check on the settings and to ensure the values are as expected.

- `OMP_NUM_THREADS` - This is probably the most often used OpenMP environment variable, but before OpenMP 3.1 it was not working well in conjunction with nested parallelism.

  This has since then been addressed, and it now supports a set of values, which are set by a list of positive integer values. These values are used to define the number of threads at the corresponding nesting level of parallel regions, starting at level 1.

For example, setting `OMP_NUM_THREADS` to "`3,2`" implies that a top-level parallel region uses 3 threads. The team size of the next nested parallel region is 2.

If the nesting level is deeper than the number of entries in the list, the last value in the list is used to set the number of threads in subsequent nested parallel regions. In the above example, this means that 2 threads are used in every parallel region at nesting level 3 or more.

- `OMP_SCHEDULE` - As before, this variable is used to set the type of workload scheduling for parallel loops in the event the `runtime` option is used on the `schedule` clause.

  In addition to `static`, `dynamic`, and `guided`, as of OpenMP 3.0, the `auto` schedule type is supported as well. With this, the choice of the schedule is determined by the implementation. This new scheduling type does not support a chunk size.

- `OMP_STACKSIZE` - Each OpenMP thread needs a certain amount of storage space to store information, such as private variables. This part of memory is referred to as "the stack."

  An OpenMP runtime system sets aside a certain default amount of storage space for each thread. This is controlled by the `OMP_STACKSIZE` environment variable. The setting can be checked in the documentation or through environment variable `OMP_DISPLAY_ENV`.

  Unfortunately an application is likely to crash if enough (stack) space is not available. In some cases, this problem can be determined at compile time and the compiler may issue a warning, but more often than not, this is not possible and the problem only shows up at runtime.

  That is when this variable needs to be used to increase the thread stack size.

  The syntax supports a case-insensitive unit qualifier that is appended (possibly with whitespace in between) to the number: `B` for bytes, `K` for 1024 bytes, `M` for 1024 Kbytes and `G` for 1024 Mbytes.

  For example, `export OMP_STACKSIZE=2M` or `export OMP_STACKSIZE="2 m"` both set the thread stack size to 2 Mbyte using Bash shell syntax.

  There are two more things worth noting:

  - In the absence of a unit qualifier (`B`, `K`, `M` or `G`), the default unit is *Kbytes*, not bytes. This is easily overlooked, resulting in very high memory requirements that may cause the application to fail.
  - The application may require its own *main* stack size to be set if the OS does not provide sufficient stack space by default. This has no relation to OpenMP and is *not* affected by the `OMP_STACKSIZE` variable. Typically this stack is set through an OS-specific command (e.g. `limit` or `ulimit` on Unix systems) and is most likely specified in bytes, but one is advised to check the documentation when in doubt.

- `OMP_WAIT_POLICY` - In the optimal scenario, all threads are busy all the time doing useful work, but in practice there are often shorter or longer periods of inactivity for one or more threads.

  The question is what to do with such threads. Putting them to sleep saves processor cycles and other resources, but waking up such a thread is relatively costly. Simply keeping the processor busy doing nothing wastes cycles.

  Until the `OMP_WAIT_POLICY` variable was introduced in OpenMP 3.0, there was no portable solution to specify the desired behavior.

  There are currently two settings for `OMP_WAIT_POLICY` to specify the behavior of such waiting threads, also called *idle threads*:

  - `active` - Idle threads should mostly be kept active. This is good for the performance of the application, because an idle thread is ready to execute as soon as it is needed again.

    The word *mostly* is important here. It is not good for the system throughput to keep idle threads in a busy loop for an extended period of time. It only wastes cycles.

    This is why, by default, an implementation may decide to put an idle thread to sleep after a certain time out. Obviously this choice, and the length of the time out, is implementation-, or even system-dependent.

    By setting the idle mode to `active`, this is overruled and threads are ready to go any time they are needed.
  - `passive` - Idle threads should mostly do nothing and not use any processor cycles. This favors overall system throughput, but activating an idle thread may take longer compared to the active setting.

Remember that the setting is merely a suggestion. The implementation is free to ignore it.

If more explicit control is needed, we recommend checking the documentation of the specific runtime environment used. Several OpenMP implementations support more control than provided by this environment variable.

- `OMP_MAX_ACTIVE_LEVELS` - The `OMP_MAX_ACTIVE_LEVELS` variable is set to a non-negative integer that defines the upper limit on the number of active parallel regions that may be nested.

  If the value is negative, not an integer at all, or exceeds the limit supported by the implementation, the behavior is undefined.

- `OMP_THREAD_LIMIT` - The `OMP_THREAD_LIMIT` variable is set to a non-negative integer that defines the maximum number of threads to be used in a *contention group*, where a contention group consists of an initial thread and its descendant threads.

  Its main use is to prevent an application from generating too many threads. Especially with recursive types of algorithms, this can easily happen. As a result, the system may become unresponsive. This limit can be used to avoid such a situation.

  If the value is negative, not an integer at all, or exceeds the limit supported by the implementation, the behavior is undefined.

- `OMP_MAX_TASK_PRIORITY` - The `OMP_MAX_TASK_PRIORITY` variable is set to a non-negative integer that defines the maximum priority level to be used in the `priority` clause on the `task` construct.

  If this variable is not set explicitly, the default value is zero, and all priority settings in the application are effectively ignored.

- `OMP_PLACES` - The `OMP_PLACES` variable is extensively covered in Chapter 4, starting on page 151. This is why there is only a very brief discussion here.

  The `OMP_PLACES` variable defines the OpenMP place list to be used by the runtime environment.

  A single place is defined through an *unordered* set of comma separated non-negative numbers enclosed in curly braces ({ and }). For example {`8,0,4`} defines a place with three entries, and the order is irrelevant.

Places are the building blocks of the *place list*, which consists of an *ordered* list with one or more comma-separated places.

In other words, the order within a place definition does not matter, but the order in which the places are specified in the place list does matter.

The integer numbers are system-specific and may need to be adapted in case a different system is used. This is not very convenient and that is why an alternative to such lists is supported.

Three *abstract names* to cover several common scenarios are available. The names are `sockets`, `cores`, and `threads`. They cannot be combined with a list specification.

As with most OpenMP environment variables, if `OMP_PLACES` has not been set, a *system-dependent default place list* is used. It is important to note that this means there is *always* a place list.

- `OMP_PROC_BIND` - The `OMP_PROC_BIND` variable sets the thread affinity policy, or policies, to be used for parallel regions.

  The settings control if and how OpenMP threads are bound to computational resources, such as sockets, cores and/or hardware threads.

  This is achieved through a tight coupling between binding and *OpenMP places*, a new concept introduced in OpenMP 4.0 and briefly covered under the description of the `OMP_PLACES` environment variable.

  The `OMP_PROC_BIND` variable was already introduced in OpenMP 3.1, but could be set to only `true` or `false`. With the introduction of thread affinity support in OpenMP 4.0, the functionality has been greatly enhanced and provides very fine-grained control over thread placement.

  There are three policies regarding how to place the threads: `master`, `close`, and `spread`.

  In the case of nested parallel regions, multiple and potentially different placement types may be defined by using a comma-separated list. For example `OMP_PROC_BIND="spread,close"`.

  If the nesting level exceeds the number of policies in the list, the last policy is applied to the remaining levels.

  If binding is requested, but no place list is defined, a default place list is used.

Note that if `OMP_PROC_BIND` is not set, the default may be `false`. In this case, or if it is explicitly set to `false`, the settings for the place list are *ignored*.

Much more detailed coverage of binding and places can be found in Chapter 4, which is dedicated to thread affinity, binding, and the places concept.

- `OMP_CANCELLATION` - If `OMP_CANCELLATION` variable is set to `true`, cancellation is activated. The effects of the `cancel` construct and of cancellation points are enabled. If set to `false`, cancellation is disabled.

  If this variable is not set explicitly, the default value is `false`.

- `OMP_DEFAULT_DEVICE` - The `OMP_DEFAULT_DEVICE` variable may be used to define the default device number to be used in device constructs. The value must be a non-negative integer.

2.3 New Runtime Functions

Since OpenMP 2.5, quite a number of runtime functions have been added to the functions listed in Figure 1.29 on page 36. Existing functionality has been expanded and newly introduced features have been augmented by a set of functions specific to the feature.

As before, these functions may be used to query settings and also change them, but in the case of heterogeneous systems, they go a step further. The device memory functions may be used to manage memory on the target device(s).

A new element in the specifications is that some functions require an OpenMP-specific data type. They are defined in include file `omp.h` in C/C++. Fortran programs require either file `omp_lib.h` to be included or a module called `omp_lib` to be used.

In Fortran, these new data types are specified using the `KIND` type selector.

For example, function "`omp_proc_bind_t omp_get_proc_bind()`" in C/C++ is "`integer (kind=omp_proc_bind_kind) function omp_get_proc_bind()`" in Fortran.

In the remainder of this section, functions added since OpenMP 2.5 are listed and discussed. To avoid lengthy tables, the list has been split into three parts, each covered in its own section. The syntax given is for C/C++ only, but the usage in Fortran is very similar. The exception are the device memory functions. There are

Function name	Description
`omp_get_thread_limit`	Return the maximum number of threads in the contention group.
`omp_get_schedule`	Return the loop schedule (and chunk size) in case the `runtime` clause is used.
`omp_set_schedule`	Change the loop schedule (and chunk size) in case the `runtime` clause is used.
`omp_get_max_active_levels`	Return the maximum number of nested active parallel regions.
`omp_set_max_active_levels`	Change the maximum number of nested active parallel regions.
`omp_get_level`	Return the number of nested parallel regions enclosing the current task.
`omp_get_active_level`	Return the number of nested *active* parallel regions enclosing the current task.
`omp_get_ancestor_thread_num`	Return the thread number of the ancestor of the current thread.
`omp_get_team_size`	Return the size of the thread team to which the current thread or ancestor belongs.

Figure 2.17: **Runtime functions for thread management, scheduling, and nested parallelism** – These functions provide support for thread management, workload scheduling and nested parallelism. The names are the same in C/C++ and Fortran, but the usage follows the language syntax.

no native interfaces for Fortran. One may be able to call the C/C++ versions from Fortran, but there is no such guarantee.

2.3.1 Runtime Functions for Thread Management, Thread Scheduling, and Nested Parallelism

The first set of additional runtime functions consists of functions related to thread management, loop iteration scheduling, and nested parallelism. They are listed in Figure 2.17. These functions augment the constructs already supported in OpenMP 2.5. The next two sections cover the functions related to the new features introduced since then.

- `int omp_get_thread_limit()` - This function returns the maximum number of OpenMP threads that the application is allowed to create in the current contention group.

  While the number of hardware scheduling resources is fixed for a given system configuration, this upper limit on the number of threads is controlled by the OpenMP implementation. This function returns the limit. Environment variable `OMP_THREAD_LIMIT` may be used to change this limit.[1]

  The difference between this function and the older `omp_get_num_procs()` function is that the latter returns the number of "processors" available on the system. The notion of what a processor is exactly, can be somewhat ambiguous, so it is up to the OS to decide. It could be a core, or a hardware thread, for example.

- `void omp_get_schedule(omp_sched_t *kind, int *chunk_size)` - The first argument of this function returns a pointer of type `omp_sched_t` to identify the schedule used to assign loop iterations to threads. Where relevant, the second argument returns an additional qualifier, such as the chunk size.

  There are four pre-defined schedule types: `static`, `dynamic`, `guided`, and `auto`. The corresponding codes are 1, 2, 3, and 4, respectively. Note that the first value is 1, not 0. This may require care if used as an index into an array in C/C++.

  An implementation has the freedom to support additional scheduling types. If so, the definition of the second argument for that type is implementation-dependent and the reason we also refer to it as a "qualifier."

- `void omp_set_schedule(omp_sched_t kind, int chunk_size)` - Function to change the scheduling type and (optionally) the chunk size. The first argument is an integer identifier (or a specific reserved variable name) to specify the schedule to be used.

[1] There may also be an OS or installation-specific upper limit. Getting this changed depends on the specific environment and local situation.

The currently supported reserved variable names include the name of the schedule:

  - `omp_sched_static = 1`
  - `omp_sched_dynamic = 2`
  - `omp_sched_guided = 3`
  - `omp_sched_auto = 4`

If supported by the scheduling type, the second argument may be used to pass on the chunk size. Any value less than one implies the *default* chunk size is used.

This function may be used to adapt the loop schedule on a per-loop basis. In Figures 2.22 and 2.23 the usage of the functions to get and set the schedule is demonstrated.

Note that this function has an effect only in combination with the `runtime` clause.

- `int omp_get_max_active_levels()` - This function returns how deep parallel regions may be nested.
- `void omp_set_max_active_levels(int max_levels)` - Environment variable `OMP_MAX_ACTIVE_LEVELS` may be used to initially set the maximum level of parallel regions that are allowed to be nested. This function increases or decreases this limit at runtime to adapt to specific needs.
- `int omp_get_level()` - This function returns the number of nested parallel regions enclosing the current task, regardless of whether the region is active or not. Recall that *active* in an OpenMP context means that more than one thread executes the region.

  This function is used to determine at which level a (nested) parallel region is. The first-level region is assigned a level of 1, the one nested within this region has a level of 2, and so on.

  One of the uses of this functionality is to determine a unique thread ID at any level. Figure 2.26 on page 83 shows a source code example of how to do this.

- `int omp_get_active_level()` - This function provides the same functionality as the function `omp_get_level()` described above, but it is restricted to active parallel regions only.

  Both functions may be used to determine at which level a (nested) parallel region is. The first level region is assigned a level of 1, the one nested within this region has a level of 2, and so on.

- `int omp_get_ancestor_thread_num(int level)` - The thread ID of the ancestor thread is returned by this function. The function argument is the nesting level this applies to.

  There is one important thing to point out. This is the level of the *ancestor*, not the level of the thread calling this function. For example, if at level 3, the function argument should be 2, not 3, to get the thread ID of the ancestor.

  If this function is called with the nesting level of the caller, the regular thread ID is returned. This is the same value as returned by function `omp_get_thread_num()`.

- `int omp_get_team_size(int level)` - This function returns the number of threads in the team executing the parallel region at the nesting level specified by the argument.

  If this function is called with the nesting level of the caller, the number of threads in the caller's current team is returned. This is the same value as returned by function `omp_get_num_threads()`.

2.3.2 Runtime Functions for Tasking, Cancellation, and Thread Affinity

This section presents and discusses the functions related to tasking, thread cancellation, and thread affinity. These functions return a value, or setting, only. There are no functions to change a setting.

The functions are listed in Figure 2.18. Note that for the affinity-related functions, we do not use the word "processor" as the specifications do. Instead, we use "(hardware) resource number" or simply "resource" for short. This terminology is explained in detail in Section 4.5.1 in Chapter 4.

- `int omp_in_final()` - This function returns `true` if called from a task that is final. Otherwise it returns `false`.

Function name	Description
`omp_in_final`	Return `true` if executed in a final task region; otherwise return `false`
`omp_get_max_task_priority`	Return the maximum value that may be specified in the `priority` clause.
`omp_get_cancellation`	Return the status of cancellation.
`omp_get_proc_bind`	Return the thread affinity policy to be used for the *subsequent* parallel region that does not use a `proc_bind` clause.
`omp_get_num_places`	Return the number of places in the place list.
`omp_get_place_num_procs`	Return the number of resources associated with the argument, which must be a place number.
`omp_get_place_proc_ids`	Return the numerical identifiers of each resource associated with the first argument, which must be a place number.
`omp_get_place_num`	Return the place number to which the encountering thread is bound.
`omp_get_partition_num_places`	Return the number of places in the subpartition.
`omp_get_partition_place_nums`	Return the list of place numbers corresponding to the places in the subpartition.

Figure 2.18: **Runtime functions for tasking, cancellation, and thread affinity** – These functions provide support for tasking, cancellation, and thread affinity. The names are the same in C/C++ and Fortran, but the usage follows the language syntax.

- `int omp_get_max_task_priority()` - This function returns the maximum value that may be used on the `priority` clause supported on the `task` construct.
- `int omp_get_cancellation()` - This function returns `true` if cancellation is enabled. Otherwise it returns `false`.
- `omp_proc_bind_t omp_get_proc_bind()` - This function uses the setting for

environment variable `OMP_PROC_BIND` to return the policy for the *subsequent* parallel region.

The return value has data type `omp_proc_bind_t` and may take one of the following values:

- `omp_proc_bind_false = 0`
- `omp_proc_bind_true = 1`
- `omp_proc_bind_master = 2`
- `omp_proc_bind_close = 3`
- `omp_proc_bind_spread = 4`

This function does not return a meaningful result if the `proc_bind` clause is used on the subsequent parallel region, because this takes precedence over the policy set through `OMP_PROC_BIND`.

Figure 2.28 on page 85 shows an example of how to use this function.

More on the choices for these thread affinity settings may be found under the description of the `OMP_PROC_BIND` environment variable.

- `int omp_get_num_places()` - This function returns the number of places in the place list.

- `int omp_get_place_num_procs(int place_num)` - This function returns the number of hardware resources, for example, hardware threads or cores, in the place identified by `place_num`.

  A value of zero is returned if `place_num` is either less than zero or is equal to or larger than the value as returned by `omp_get_num_places()`.

- `void omp_get_place_proc_ids(int place_num, int *ids)` - This function returns a list in array `ids`. This list contains the (hardware) resource numbers associated with the place identified by `place_num`. This place identifier, as well as the resource numbers, are implementation-dependent numbers to identify a place and its contents. Refer to Section 4.5.1 on page 161 for the definition of a resource number.

  Be sure to allocate sufficient storage for array `ids`. At least `omp_get_place_num_procs(place_num)` locations must be available.

This function has no effect if `place_num` is less than zero, or is equal to, or larger than the value as returned by `omp_get_num_places()`.

- `int omp_get_place_num()` - If the encountering thread is bound to a place, the place number associated with this thread is returned and the value is in the interval $[0, \texttt{omp\_get\_num\_places()} - 1]$.

  If the thread is not bound to a place, a value of -1 is returned.

- `int omp_get_partition_num_places()` - This function returns the number of places in the subpartition of the encountering thread.

- `void omp_get_partition_place_nums(int *place_nums)` - This function returns the list of place numbers corresponding to the places in the subpartition of the encountering thread.

  Array `place_nums` must be sufficiently large to contain the number of integers returned by `omp_get_partition_num_places()`. Otherwise, the behavior is undefined.

2.3.3 Runtime Functions for Locking

Before diving deeper into this, an important warning first: the ownership of locks has changed. In most cases, this has no impact on the application, but we recommend reading Section 2.4.1 on page 86 for the details.

The set of locking functions has been extended to support a *hint* the user may provide. This hint is used to specify the expected behavior of the lock. This allows the implementation to select a locking mechanism tuned for this specific purpose.

The syntax of the new functions is given in Figure 2.19. The hint does not change the mutual exclusion semantics of the lock, and the implementation is free to ignore it. The application should also not rely on the hint in order to work correctly.

The choices for the hint are listed in Figure 2.20. As usual, such values are defined in header file `omp.h` for C/C++ and the Fortran `omp_lib.h` include file and/or module file `omp_lib`.

At first sight, the support for `omp_lock_hint_none` seems somewhat peculiar. This informs the implementation there is no specific preference.

```
void omp_init_lock_with_hint (omp_lock_t *t, omp_lock_hint_t hint);

void omp_init_nest_lock_with_hint (omp_lock_t *t, omp_lock_hint_t hint);
```

```
subroutine omp_init_lock_with_hint (svar, hint)
 integer (kind=omp_lock_kind) svar
 integer (kind=omp_lock_kind_hint) hint

subroutine omp_init_nest_lock_with_hint (svar, hint)
 integer (kind=omp_lock_kind) svar
 integer (kind=omp_lock_kind_hint) hint
```

Figure 2.19: **Syntax of the new locking functions in C/C++ and Fortran** – These functions support an optional hint. This gives the implementation the opportunity to select the most suitable solution, but it is also free to ignore the hint. Both single-level, as well as, nested locks are available.

```
typedef enum omp_lock_hint_t {
  omp_lock_hint_none = 0,
  omp_lock_hint_uncontended = 1,
  omp_lock_hint_contended = 2,
  omp_lock_hint_nonspeculative = 4,
  omp_lock_hint_speculative = 8
} omp_lock_hint_t;
```

```
integer (kind=omp_lock_hint_kind), &
  parameter::omp_lock_hint_none = 0
integer (kind=omp_lock_hint_kind), &
  parameter::omp_lock_hint_uncontended = 1
integer (kind=omp_lock_hint_kind), &
  parameter::omp_lock_hint_contended = 2
integer (kind=omp_lock_hint_kind), &
  parameter::omp_lock_hint_nonspeculative = 4
integer (kind=omp_lock_hint_kind), &
  parameter::omp_lock_hint_speculative = 8
```

Figure 2.20: **Definition of the hint data type in C/C++ and Fortran** – These are the various choices supported for the hint. An implementation may add to this list. For an explanation of these hints, refer to the main text.

Why would one use a function that supports a hint and then effectively say it doesn't matter? This choice is available to accommodate flexibility in an application. The same locking function may be called for various values of the hint, including the lack of a preference, leaving it up to the system to decide.

The next two choices are about *lock contention*. The lock is said to be "contended," if multiple threads want to access a lock at the same time. If this is the case, a more advanced mechanism to acquire the lock could be beneficial. For example, by using an exponential backoff algorithm in case a lock cannot be acquired right away.

The last two types are included for systems with support for *Transactional Memory*. When using a *speculative lock*, the assumption is that there is no contention and that the code within the locked region can be executed unconditionally. Only afterward is it verified that there was indeed no contention. If there had been contention, a rollback strategy is applied, if necessary, to guarantee the correct results.

This highly improves the efficiency of the locking mechanism if there is little or no contention, but it is not without a potential cost. The rollback part is relatively expensive and if this is executed too often, the performance could be worse than without the speculation.

If the user knows upfront that the lock is only lightly contended, using this hint may improve performance.

There are several restrictions when using hints:

- Hints may be combined through wither the `+` or `|` operators in C/C++ or the `+` operator in Fortran, but the behavior is implementation-defined and may even be ignored.
- Combining `omp_lock_hint_none` with any other hint is equivalent to specifying the other hint only.
- As one might expect, when combining conflicting settings, for example "contended" with "uncontended," the behavior is undefined.
- An implementation is free to support additional choices for the hint, but be aware that these may not be portable to other implementations.

2.3.4 Runtime Functions for Heterogeneous Systems

One of the main additions to OpenMP is the support for heterogeneous systems, typically consisting of a host system, plus one or more accelerators. In OpenMP these are referred to as "devices."

In this section, only the runtime functions are summarized. For extensive coverage of this topic, plus an introduction to the device-specific terminology, refer to Chapter 6 starting on page 253.

The set of runtime functions supporting devices are used to query the settings, as well to set the default device. OpenMP 4.5 adds functions to explicitly manage the memory between the host and the device(s).

The full list can be found in Figure 2.21. Below is a more extensive description of each function.

- `int omp_get_default_device()` - This function returns the value of the *default-device-var* ICV, which is the default device number. If it is called from within a `target` region, the result is undefined.
- `void omp_set_default_device(int device_num)` - This function assigns the value of the integer argument `device_num` to the *default-device-var* ICV. If it is called from within a `target` region, the behavior is undefined.
- `int omp_get_initial_device()` - This function returns the host device number. If the return value is in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$, then it may be used in a `device` clause and in all device memory functions. If the value is outside that range, then it may be only in device memory functions only. Calling this function from within a `target` region results in undefined behavior.
- `int omp_is_initial_device()` - This function returns `true` if the thread that calls it is executing on the host device. Otherwise, it returns `false`.
- `int omp_get_num_devices()` - This function returns the number of available devices. Calling it from within a `target` region results in undefined behavior. Whether or not the host device is included in the number of available devices is implementation-defined.
- `int omp_get_num_teams()` - This function returns the number of teams in the current teams region. See Section 6.5 starting at page 275 for more details

Function name	Description
`omp_get_default_device`	Return the default device number.
`omp_set_default_device`	Set the default device number.
`omp_get_initial_device`	Return the host device number.
`omp_is_initial_device`	Return `true` if the current task is executing on the host device; otherwise return `false`.
`omp_get_num_devices`	Return the number of devices.
`omp_get_num_teams`	Return the number of teams in the current `teams` region.
`omp_get_team_num`	Return the team number of the calling thread.
`omp_target_alloc`	Allocate a block of memory on a device.
`omp_target_free`	Free memory allocated on a device by the `omp_target_alloc` function.
`omp_target_is_present`	Check whether a host address is mapped to a device.
`omp_target_memcpy`	Copy memory between any combination of host and device pointers.
`omp_target_memcpy_rect`	The same as `omp_target_memcpy`, but for entire sub-volumes of multi-dimensional arrays.
`omp_target_associate_ptr`	Associate a device address with a host address.
`omp_target_disassociate_ptr`	Remove the association between a device address and a host address.

Figure 2.21: **Runtime functions for heterogeneous computing** – These functions provide support for heterogeneous computing, including memory management between the host and the device(s). The last seven functions, the "device memory functions," are not supported in Fortran.

on the `teams` construct. A value of 1 is returned if called from outside a teams region.

- `int omp_get_team_num()` - This function returns the calling thread's team number. The return value is an integer number in the interval $[0, \texttt{omp\_get\_num\_teams()} - 1]$. If it is called from outside a `teams` region, a value of zero is returned.

- `void* omp_target_alloc(size_t size, int device_num)` - This function returns the device address of a memory block of `size` bytes dynamically allocated in the address space of the device number specified by `device_num`. If the allocation fails, then it returns `NULL`. The device number should be either the value returned by `omp_get_initial_device()` or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$.

  Upon a successful allocation, the device address may be used in an `is_device_ptr` clause on a `target` construct or in other device memory functions. See Section 6.3.3 starting on page 265 for more details on using device addresses.

  There are several restrictions when using this function:

  - The behavior of the function is undefined if it is called from a `target` region.
  - Pointer arithmetic cannot be performed on a device address returned by this function.
  - Memory allocated by this function can be freed using the `omp_target_free()` function only.

- `void omp_target_free(void *device_ptr, int device_num)` - This function frees a memory block that was dynamically allocated in the address space of the device number specified by `device_num`. The value of the `device_ptr` should be a device address returned by an earlier call to the `omp_target_alloc()` function. The device number should be either the value returned by `omp_get_initial_device()` or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$. If `device_ptr` is `NULL`, the function has no effect. The behavior of the function is undefined if it is called from a `target` region.

- `int omp_target_is_present(void *ptr, int device_num)` - This function is used to test if a variable is present in a device data environment. The device number should be either the value returned by `omp_get_initial_device()` or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$.

  The function returns `true` if the host address stored in `ptr`, has a corresponding address in the device data environment of the device identified by `device_num`. Otherwise, `false` is returned. The effect of this function is undefined if called from within a `target` region.

- `int omp_target_memcpy(`

  ```
  void *dst,
  void *src,
  size_t length,
  size_t dst_offst,
  size_t src_offst,
  int dst_device_num,
  int src_device_num) -
  ```

  This function copies a contiguous memory block of `length` bytes from a source device to a destination device. The `dst_device_num` and `src_device_num` are device numbers that identify the source and destination devices. The device numbers should either be the value returned by `omp_get_initial_device()` or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$. The source device address is determined by adding the `src_offset` to the value of the `src` pointer variable. The destination device address is determined by adding the `dst_offset` to the value of `dst` pointer variable.

  The return value is zero upon successful completion of the copy and non-zero in the event of a failure. The `dst` and `src` pointers must be valid device pointers. The calculated source and destination addresses must be valid on the respective devices. The effect of this function is undefined if called from within a `target` region. This function contains a task scheduling point.

- `int omp_target_memcpy_rect(`

  ```
  void *dst,
  void *src,
  size_t element_size,
  int num_dims,
  const size_t *volume,
  const size_t *dst_offsets,
  const size_t *src_offsets,
  const size_t *dst_dimensions,
  const size_t *src_dimensions,
  int dst_device_num,
  int src_device_num) -
  ```

This function copies a rectangular sub-volume of memory from a multi-dimensional array in the address space of the source device to a multi-dimensional array in the address space of the destination device.

The `dst_device_num` and `src_device_num` are device numbers that identify the source and destination devices. The device numbers should either be the value returned by `omp_get_initial_device()` or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$.

The size of an element in the source and destination arrays is `element_size` bytes. The `volume`, `dst_offsets`, `src_offsets`, `src_dimensions`, and `dst_dimensions` are constant arrays of length `num_dims`.

The `volume` array specifies the length of each dimension for the sub-volume. The `src_dimensions` and `dst_dimensions` arrays specify the length of each dimension for the source and destination arrays, respectively. The sub-volume and the source and destination arrays are defined by their dimensions and the size of an element. The `src_offset` and `dst_offset` arrays specify a multi-dimension offset into the source and destination arrays, respectively.

The starting source device address is determined by the `src` device pointer variable and the `src_offset` array. The starting destination device address is determined by the `dst` device pointer variable and the `dst_offset` array.

The value of `num_dims` must be between 1 and an implementation-defined limit, which must be at least three. The `dst` and `src` pointers must be valid device pointers. The effect of this function is undefined if called from within a `target` region. This function returns zero if successful. Otherwise, it returns a non-zero value. If both `src` and `dst` are `NULL` pointers, the number of dimensions supported by the implementation for the specified device numbers is returned. This function contains a task scheduling point.

- ```
  int omp_target_associate_ptr(
        void *host_ptr,
        void *device_ptr,
        size_t size,
        size_t device_offset,
        int device_num) -
  ```

  This function associates a host memory block of `size` bytes with a corresponding memory block in the address space of a device. The device number,
  ```

specified in `device_num`, should either be the value returned by `omp_get_initial_device()`, or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$. The corresponding device address is determined by adding the `dst_offset` to the value of `dst` pointer variable.

Once associated, the host address range is mapped to the corresponding address range in the device data environment of the `device_num` device. The corresponding address range has an infinite reference count. See Section 6.3.1 starting on page 263 for more details on mapped variables. The value of `device_ptr` must be a valid device address. See Section 6.3.3 starting on page 265 for more details on device addresses.

This function returns zero if successful. Otherwise, a non-zero value is returned. Upon return from the function, the value of the memory in the corresponding address range is undefined. Once associated, the behavior when accessing the corresponding address range from the host device via an `is_device_ptr` clause or another device memory function is undefined until the association is removed by the `omp_target_disassociate()` function. A host address may correspond to only one device address on one device. Attempting to associate a host address to more than one device address results in a non-zero return value. The `omp_target_is_present()` function may be used to check if a host address is already associated with a device address. The effect of this function is undefined if called from within a `target` region.

- `int omp_target_disassociate_ptr(void ptr, int device_num)` - This function removes the association of a host memory block to a corresponding memory block in the address space of a device. The association would have been established by a previous call to the `omp_target_associate()` function. The device number should either be the value returned by `omp_get_initial_device()` or in the range $[0, \texttt{omp\_get\_num\_devices()} - 1]$.

  The host address is unmapped. The corresponding address range's reference count is set to zero and removed from the device data environment of the `device_num` device. See Section 6.3.1 starting on page 263 for more details on mapped variables. See Section 6.3.3 starting on page 265 for more details on device addresses.

The effect of this function is undefined if called from within a `target` region. If the host address in `ptr` does not have a corresponding device address associated with it by a previous call to `omp_target_associate_ptr()`, then the behavior of this function is undefined. Upon return from the function, the value of the memory in the corresponding address range is undefined.

2.3.5 Usage Examples of the New Runtime Functions

Now that all new runtime functions are covered, it is time to show some examples in what way these functions may be used.

The first example demonstrates how to query and change the loop schedule, as set through the `runtime` clause. In the next example, the nested parallelism code fragment listed in Figure 1.17 on page 25, is revisited. The relevant runtime functions are used to define a global thread ID. The third example illustrates how to get information on the thread affinity policy (or policies) that are in place.

Examples that use the device memory functions can be found in Section 6.12, starting on page 323. Examples using other heterogeneous runtime functions are shown in Section 6.5, starting on page 275.

Change the Loop Schedule Type at Runtime The `runtime` clause provides flexibility to select an appropriate schedule to distribute the work over the threads. There was one main issue with the original design, though. Recall that the schedule, as set by environment variable `OMP_SCHEDULE`, applied to *all* loops with this clause.

The new runtime functions provide greater flexibility and allow the application to customize the schedule to match the requirements on a per-loop basis. This is illustrated in the following example, which is an extended version of the code given in [2].

The function listed in Figure 2.22 uses the runtime function `omp_get_schedule()` to query the settings. At line 9, this function is called and the information is printed at lines $11 - 15$. In case a known value for the schedule type is returned, one of the four regular names is printed. Otherwise, "`implementation-defined`" is printed.

The runtime function also returns a value in the second argument, `schedule_modifier`. If the schedule type is `static`, `dynamic`, or `guided`, this variable contains the chunk size. It is undefined for schedule type `auto`, and implementation-dependent in case an implementation-specific schedule is used.

```
 1 void print_schedule_info(char *comment)
 2 {
 3   char *alt_schedule = "implementation-defined";
 4   char *schedules[4] = {"static","dynamic","guided","auto"};
 5
 6   omp_sched_t schedule_kind;
 7   int         schedule_modifier;
 8
 9   (void) omp_get_schedule(&schedule_kind, &schedule_modifier);
10
11   printf("Schedule details %s:\n", comment);
12   printf(" schedule kind  = %d\n", schedule_kind);
13   printf(" schedule type  = %s\n",
14    schedule_kind <= 4 ? schedules[schedule_kind-1]:alt_schedule);
15   printf(" schedule modifier = %d\n", schedule_modifier);
16 }
```

Figure 2.22: **A function to verify the loop schedule** – This function queries the current loop schedule and prints the information.

In Figure 2.23, this function is called twice. At line 1, the current settings for the schedule and chunk size are printed. These are applied to the parallel loop spanning lines $3 - 9$.

At line 11, the runtime function `omp_set_schedule()` is called. The first argument is the OpenMP variable `omp_sched_static`. This changes the schedule to `static`, regardless of the previous setting. Any value less than 1 for the second argument, enforces the default settings for the chunk size to be used. In this call, a value of zero is used to achieve this. The new settings are printed by the function call at line 13. This is what is used for the parallel loop at lines $15 - 21$.

The example code has been executed using 3 threads and a loop length of 7. The initial schedule is defined through environment variable `OMP_SCHEDULE` and set to `"dynamic,2"`. The full results are given in Figure 2.24.

The output confirms that the initial schedule type is `dynamic`, with a chunk size of 2. The assignment of iterations to threads for this first loop is just a snapshot. It can be expected that the results vary from run to run.

The results for the second loop demonstrate that the `static` schedule has indeed been used. Thread 0 executes the first 3 iterations. The remaining 4 iterations are

```
 1 (void) print_schedule_info("loop 1");
 2
 3 #pragma omp parallel for default(none) schedule(runtime)\
 4                         shared(n)
 5   for (int i=0; i<n; i++)
 6   {  printf("Iteration %d executed by thread %d\n",
 7           i, omp_get_thread_num());
 8   for (int j=0; j<i; j++) system("sleep 1");
 9 } // End of parallel for
10
11 (void) omp_set_schedule(omp_sched_static, 0);
12
13 (void) print_schedule_info("loop 2");
14
15 #pragma omp parallel for default(none) schedule(runtime)\
16                         shared(n)
17   for (int i=0; i<n; i++)
18   {  printf("Iteration %d executed by thread %d\n",
19           i, omp_get_thread_num());
20   for (int j=0; j<i; j++) system("sleep 1");
21 } // End of parallel for
```

Figure 2.23: **Example how to change the loop schedule** – This code fragment prints and changes the loop schedule.

distributed equally over the other 2 threads and are in consecutive order. These results are reproducible.

Nested Parallelism Revisited The new functions for nested parallelism provide a convenient way to query the environment. In this example, they are used to more easily construct a unique *global* thread ID.

Recall the example given in Figure 1.18 on page 26. Because the threads in each team start with thread ID 0 and are numbered consecutively, the value returned by function `omp_get_thread_num()` cannot be used to distinguish threads between different teams. Usually this is not an issue, but what if one would like to have a thread ID that is unique at a certain nesting level, or even across all nesting levels?

```
Schedule details loop 1:
 schedule type     = 2
 schedule type     = dynamic
 schedule modifier = 2
Iteration 0 executed by thread 2
Iteration 1 executed by thread 2
Iteration 2 executed by thread 0
Iteration 4 executed by thread 1
Iteration 6 executed by thread 2
Iteration 3 executed by thread 0
Iteration 5 executed by thread 1
Schedule details loop 2:
 schedule type     = 1
 schedule type     = static
 schedule modifier = 0
Iteration 5 executed by thread 2
Iteration 3 executed by thread 1
Iteration 0 executed by thread 0
Iteration 1 executed by thread 0
Iteration 2 executed by thread 0
Iteration 4 executed by thread 1
Iteration 6 executed by thread 2
```

Figure 2.24: **Example results from the program to query and set the loop schedule** – The loop length is set to 7, and 3 threads are used. Environment variable `OMP_SCHEDULE` is initialized to `"dynamic,2"`.

The following formulas may be used to construct a unique thread ID when the team size is *constant* at a specific nesting level. It can be different across nesting levels, though. Without this restriction the formulas do not apply and one must dynamically scan the tree representing the nesting structure in order to compute a unique thread ID. This is far beyond the scope of this example and is left as an exercise for the reader.

In the remainder, the thread ID at a specific nesting level is referred to as the *Level Thread ID*, or L_{TID} for short. This identifier is unique at each nesting level, but not across the levels. The thread ID that is unique across all nesting levels and thread teams is denoted by G_{TID} (*Global Thread ID*).

These are the formulas for L_{TID} and G_{TID}:

$$L_{TID} = T_{TID} + O_L(L) = T_{TID} + \sum_{k=1}^{L-1} \{AT_{L-k} \prod_{j=1}^{k} TS_{L-j+1}\} \tag{2.1}$$

$$G_{TID} = L_{TID} + O_G(L) = L_{TID} + \sum_{k=1}^{L-1} \prod_{j=1}^{k} TS_j \tag{2.2}$$

In formulas 2.1 and 2.2:

- L denotes the nesting level of the parallel region.
- T_{TID} is the team-level thread ID as returned by `omp_get_thread_num()`.
- $O_L(L)$ is the offset added to the team thread ID. Note that $O_L(1) = 0$, as one would expect.
- $O_G(L)$ is the global offset added to L_{TID}, the thread ID unique across one nesting level. Because $O_G(1) = 0$, the values for L_{TID} and G_{TID} are the same at level $L = 1$.
- TS_{level} is the value returned by `omp_get_team_size(level)`.
- AT_{level} is the value returned by `omp_get_ancestor_thread_num(level)`.

As an example, these formulas are applied to the nested parallelism example shown in Figure 1.18 on page 26. There, the team size for the second-level parallel region is 2 for each team. Because this is the second level, $L = 2$ and therefore $k = 1$. To compute the L_{TID} and G_{TID} values for the rightmost thread at this second level, these values are substituted in the formulas to get the following result:

$$L_{TID} = T_{TID} + AT_1 * TS_2 = 1 + 2 * 2 = 5$$

$$G_{TID} = 5 + TS_1 = 5 + 3 = 8$$

The C source function listed in Figure 2.25 returns the offset $O_L(L)$ at the current nesting level L as defined in Formula 2.1. This value can be added to the local thread ID T_{TID} to obtain the G_{TID} for the local thread.

```
 1 int level_offset(int *G_offset)
 2 {
 3   int team_size_prod;
 4   int global_prod;
 5
 6   int level_offset  = 0;
 7   int global_offset = 0;
 8   int cur_level     = omp_get_level();
 9
10   for (int k=1; k<cur_level; k++)
11   {
12     team_size_prod = 1;
13     global_prod    = 1;
14     for (int j=1; j<=k; j++)
15     {
16       team_size_prod *= omp_get_team_size(cur_level-j+1);
17       global_prod    *= omp_get_team_size(j);
18     }
19     team_size_prod *= omp_get_ancestor_thread_num(cur_level-k);
20
21     level_offset  += team_size_prod;
22     global_offset += global_prod;
23   } // End of loop over k
24
25   *G_offset = global_offset;
26
27   return(level_offset);
28 }
```

Figure 2.25: **Function to compute the offsets to the level and global thread ID** – This is a very straightforward implementation to compute the offsets that may be used to compute the L_{TID} and G_{TID} values as given in formulas 2.1 and 2.2. It is assumed that the team is constant within one nesting level. It may be different across nesting levels, though.

The function also computes the offset that can be added to the L_{TID} to obtain the unique G_{TID}. This offset is returned through the argument pointer `G_offset`.

The computations are deliberately implemented in a very straightforward way,

which may not be the most efficient way to do this, but hopefully makes things easier to understand.

At line 8, the current nesting level L is obtained. The for-loop spanning lines $10-23$ computes the sum of the various products. The products of the team sizes are computed at lines $12-18$. At line 19, this result is then multiplied with the appropriate thread ID of the ancestor thread. At lines $21-22$, the total sums for the level and global offsets are updated with their respective contributions.

With this function to compute the two offsets to the team thread ID, it is very easy to obtain the L_{TID} and G_{TID} values. The code given in Figure 1.17 on page 26 has been augmented to get these values. The relevant part of the new source code is shown in Figure 2.26.

A five column header is printed at lines $1-2$. The key lines here are $10-11$ and $22-23$. This is where the offsets are computed and added to variable `T_tid`, the team level thread ID, and `L_tid`. The three thread ID values, plus the current level and team size for the respective levels, are printed at lines $13-14$ and $25-26$.

Sample output is shown in Figure 2.27. At the first level, the values for the L_{TID} and G_{TID} are indeed the same. At the second level, there are $3x2 = 6$ threads. Although printed in arbitrary order, it is not difficult to see that each thread at the second level has a L_{TID} level ID in the $[0 \ldots 5]$ range. The $3 + 6 = 9$ global G_{TID} values are unique across both regions and in the $[0 \ldots 8]$ range.

Verify Thread Binding With the support for cc-NUMA, first introduced in OpenMP 4.0, one can now optimize data access for memory locality by informing the runtime system where threads should run. This is extensively covered in Chapter 4, starting on page 151. Here it is shown how to use some of the affinity-related runtime functions to verify that the settings are as intended. The same two nested parallel regions from Figure 2.26 are used, but this time the goal is to control and verify in which way the OpenMP threads are distributed across the system. In the example code in Figure 2.28, the runtime setting at each nesting level is verified. This includes the policy in place for the top-level master region.

At line 1, a variable of type `omp_proc_bind_t` is declared. It is used to store the return value of function `omp_get_proc_bind`. Character array `binding_choices` is declared and initialized at lines $2-3$. It is used to map the binding type to a descriptive name.

After printing a header at line 5, the binding information for the first-level parallel region is printed at lines $8-10$. Keep in mind that the value returned for the

```
 1 printf("%7s %7s %9s %7s %7s\n",
 2        "T_TID", "Level", "Team Size", "L_TID", "G_TID");
 3
 4 #pragma omp parallel num_threads(3)
 5 {
 6   int global_offset;
 7   int cur_level     = omp_get_level();
 8   int cur_team_size = omp_get_team_size(cur_level);
 9   int T_tid         = omp_get_thread_num();
10   int L_tid         = T_tid + level_offset(&global_offset);
11   int G_tid         = L_tid + global_offset;
12
13   printf("%7d %7d %9d %7d %7d\n", T_tid, cur_level,
14                                   cur_team_size, L_tid, G_tid);
15
16   #pragma omp parallel num_threads(2)
17   {
18     int global_offset;
19     int cur_level     = omp_get_level();
20     int cur_team_size = omp_get_team_size(cur_level);
21     int T_tid         = omp_get_thread_num();
22     int L_tid         = T_tid + level_offset(&global_offset);
23     int G_tid         = L_tid + global_offset;
24
25     printf("%7d %7d %9d %7d %7d\n", T_tid, cur_level,
26                                     cur_team_size, L_tid, G_tid);
27
28   }   // End of nesting level 2
29 }   // End of nesting level 1
```

Figure 2.26: **Nested parallelism example revisited** – The function shown in Figure 2.25 is used to compute and print the level thread ID L_{TID} and global ID G_{TID} at both parallel nesting levels. The team thread ID, nesting level, and team size are printed as well.

affinity policy is for the *next* nesting level. This means the value returned in array element `binding_choices[binding_setting]` is the affinity policy for the upcoming parallel region, *not* the current one.

T_TID	Level	Team Size	L_TID	G_TID
2	1	3	2	2
1	1	3	1	1
0	1	3	0	0
0	2	2	0	3
1	2	2	1	4
0	2	2	4	7
1	2	2	5	8
0	2	2	2	5
1	2	2	3	6

Figure 2.27: **Results for the revisited nested parallelism example** – The G_{TID} value at the first nesting level is indeed the same as the local L_{TID} value. At the second level, there are 6 threads and each thread has a unique ID in the $[0 \ldots 5]$ range.

This is repeated for the two parallel regions spanning lines $12-31$. In both cases, a `single` region is used to print the binding only once per parallel region. Because it is the same for all threads in the same team, printing this for all threads is not very meaningful.

Thread placement is achieved by setting environment variables `OMP_PLACES` and `OMP_PROC_BIND`. In practice, both must be set, but for the purpose of this example, only `OMP_PROC_BIND` must be defined. It is set to the following list: `"spread,close,master"`.[2]

The output is given in Figure 2.29 and confirms the outer level threads are spread across the system. The first printed line shows that execution is at level 0, and the next parallel region has the `spread` affinity policy. Likewise, the threads at each second-level parallel region are requested to be scheduled close to each other.

The last three lines are from the three innermost parallel regions and confirm that a subsequent parallel region has the `master` affinity policy. In this case, there is no such third-level region, though.

If there are deeper runtime nesting levels than specified through environment variable `OMP_PROC_BIND`, the remaining levels inherit the settings from the last level specified. For example, if this variable was set to `"spread"` only, the thread teams have affinity type "spread" at all nesting levels.

[2]For a definition of these keywords, refer to Section 4.6, starting on page 168.

```
 1 omp_proc_bind_t binding_setting;
 2 char            *binding_choices[5] = \
 3                 {"false", "true", "master", "close", "spread"};
 4
 5 printf("%13s %25s\n\n","Current Level","Binding Type/Level");
 6
 7 binding_setting = omp_get_proc_bind();
 8 printf("%7d %23s/%-3d\n",omp_get_level(),
 9                          binding_choices[binding_setting],
10                          omp_get_level()+1);
11
12 #pragma omp parallel num_threads(3)
13 {
14  #pragma omp single
15  { omp_proc_bind_t binding_setting = omp_get_proc_bind();
16    printf("%7d %23s/%-3d\n",omp_get_level(),
17                             binding_choices[binding_setting],
18                             omp_get_level()+1);
19  } // End of single region
20
21  #pragma omp parallel num_threads(2)
22  {
23     #pragma omp single
24     { omp_proc_bind_t binding_setting = omp_get_proc_bind();
25       printf("%7d %23s/%-3d\n",omp_get_level(),
26                                binding_choices[binding_setting],
27                                omp_get_level()+1);
28     } // End of single region
29
30  } // End of nesting level 2
31 } // End of nesting level 1
```

Figure 2.28: **Example to verify the thread affinity settings** – There is one master region with two nested parallel regions. At each level, the upcoming thread affinity setting, as defined by environment variable `OMP_PROC_BIND`, is verified.

```
Current Level        Binding Type/Level

      0                   spread/1
      1                    close/2
      2                   master/3
      2                   master/3
      2                   master/3
```

Figure 2.29: **Output from the code shown in Figure 2.28** – Prior to executing the program, environment variable `OMP_PROC_BIND` is set to `"spread,close,master"`. The output confirms these settings. Note that the "master" policy applies to a possible third-level parallel region, but this example has two levels only.

2.4 New Functionality

Compared to OpenMP 2.5, OpenMP 4.5 has significant new functionality. In this section, those new features that have a relatively small impact on the programming model and are often implemented as an extension to already existing functionality are presented and discussed.

In subsequent chapters, the new concepts and features that introduce entirely new concepts in OpenMP are covered extensively.

2.4.1 Changed Ownership of Locks

The ownership of locks has changed. Although this is not new functionality, it is important enough to mention. Whereas the ownership of a lock was tied to threads in OpenMP 2.5, as of OpenMP 3.0, locks are owned by task regions.

In many cases, this should not have any consequences, but in the example in Figure 2.30, there is a difference.

The lock variable `my_lock` is declared, initialized and acquired at lines $1-4$. This code is executed by the master thread and because it executes the `omp_set_lock()` function, it owns the lock.

Within the parallel region spanning lines $6-15$, the release of the lock at line 11 is guaranteed to be performed by the master thread again.

The task region where the lock is released is not same task region where it was acquired. Although this was allowed in OpenMP 2.5, this is no longer OpenMP compliant code as of version 3.0.

```
 1  omp_lock_t my_lock;
 2
 3  (void) omp_init_lock (&my_lock);
 4  (void) omp_set_lock  (&my_lock);
 5
 6  #pragma omp parallel
 7  {
 8     #pragma omp master
 9     {
10                 ......
11         (void) omp_unset_lock (&my_lock);
12                 ......
13     } // End of master region
14     ......
15  } // End of parallel region
16
17  (void) omp_destroy_lock (&my_lock);
```

Figure 2.30: **Non-conforming use of locks** – The master thread sets the lock and therefore owns it. In OpenMP 2.5 it was permitted to have the master thread release the lock within the parallel region. This is however no longer allowed, because these two actions occur in different task regions.

Although one may, of course, write code like this, a better and cleaner approach is shown in Figure 2.31. There are two different task regions again, and the lock is accessed in both, but this program is OpenMP-compliant. The reason is that the lock is acquired (line 10) and released (line 12) in the same task region.

2.4.2 Cancellation

There are two reasons for a parallel region to terminate prematurely. An error may have occurred, for example a memory allocation that fails, and the thread(s) can no longer continue execution. It doesn't always have to be a fatal situation however. There may be a good reason the threads no longer need to continue. An example is a parallel search. Once the target element has been found, all threads can stop searching.

Prior to OpenMP 4.0, it was impossible to end the execution of an individual parallel region or, in general, any OpenMP construct within a parallel region. This

```
 1  omp_lock_t my_lock;
 2
 3  (void) omp_init_lock (&my_lock);
 4
 5  #pragma omp parallel
 6  {
 7     #pragma omp master
 8     {
 9                 ......
10         (void) omp_set_lock (&my_lock);
11                 .....
12         (void) omp_unset_lock (&my_lock);
13     } // End of master region
14  } // End of parallel region
15
16  (void) omp_destroy_lock (&my_lock);
```

Figure 2.31: **Conforming use of locks** – This program is OpenMP-compliant, because the lock is initialized and released in the same task region.

implied that every OpenMP region either had to be executed to the very end, or not started at all.

Other than to abort program execution, the only alternative was for the application to detect a certain situation and if needed, take action after the parallel region finished. With the introduction of the *cancellation constructs* in OpenMP 4.0, an elegant way to terminate the execution of an OpenMP region is supported. Aside from gracefully handling certain types of errors, this feature is useful for specific parallel methods with a dynamic behavior, such as divide-and-conquer algorithms.

Cancellation is supported on the `parallel` construct, as well as the `sections` and `for` (`do` in Fortran) worksharing constructs. When using tasks, the `taskgroup` construct may be used for cancellation.

The syntax of the `cancel` and `cancellation point` constructs in C/C++ and Fortran is shown in Figure 2.32. Both constructs are stand-alone directives. They trigger an action if, or when, encountered at runtime. The idea is similar to exception handling, supported in some programming languages. A thread that wishes to terminate execution, needs to have the `cancel` directive in its execution path. This is most likely under control of an if-statement. The effect of the `cancel` direc-

#pragma omp cancel *construct-type-clause[[,]if-clause] new-line*
!$omp cancel *construct-type-clause[[,]if-clause]*
#pragma omp cancellation point *construct-type-clause new-line*
!$omp cancellation point *construct-type-clause*

Figure 2.32: **Syntax of the cancellation construct in C/C++ and Fortran** – The `cancel` directive requests the termination of the corresponding construct. For the other threads, this takes effect at the next cancellation point.

tive is to raise a flag to signal that cancellation has occurred. The `cancellation point` directive defines a point where the encountering thread checks the status of cancellation.

This is not the only place though. The request for cancellation is checked at the following *cancellation points*:

- A `cancel` region.
- A `cancellation point` region.
- A `barrier` region.
- An implicit barrier region.

Cancellation is controlled through environment variable `OMP_CANCELLATION`, described in Section 2.2. This variable needs to be explicitly set to `true` to enable cancellation. To reduce the runtime overhead, it is *disabled* by default. If a `cancel`, or `cancellation point`, directive is encountered, while this feature is not enabled, the constructs have no effect.

When a thread raises the cancellation flag, the event is not detected by another thread, until it has encountered one of the cancellation points listed above. As a consequence, cancellation may not take effect immediately.

This is illustrated in Figure 2.33, where time flows from top to bottom. A team of five threads executes an "enclosing construct." This is one of the constructs `parallel`, `for` (or `do`), `sections`, or `taskgroup` that supports cancellation. Events 1 to 6 are assumed to happen in the order depicted by their position on the timeline.

At event 1, thread three encounters a `cancellation point` directive. At this point, cancellation has not yet been requested. The `cancellation point` has no effect and this thread continues to execute the enclosing construct.

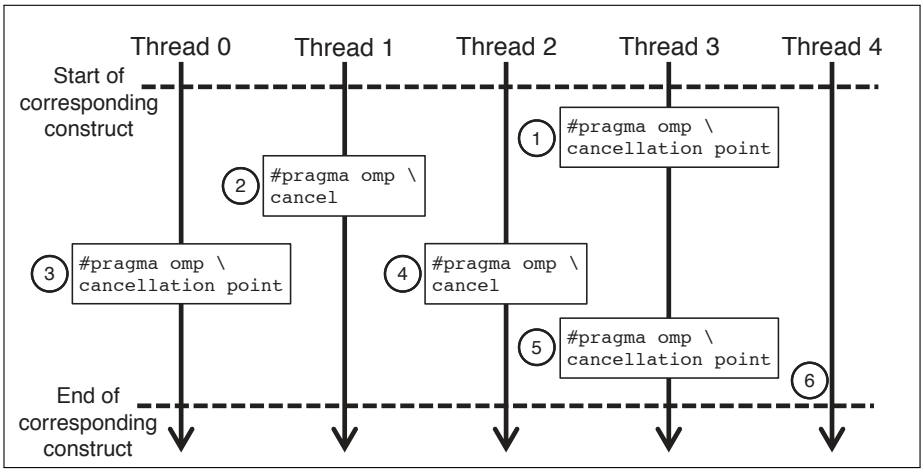

Figure 2.33: **Illustration of the cancellation feature** – In this diagram, time flows from top to bottom. Once cancellation is requested, threads only respond at the next cancellation point encountered at runtime.

Thread one encounters a `cancel` directive at event 2, and execution of the enclosing construct is cancelled. This raises a flag to indicate that cancellation has been requested. At the implied barrier or synchronization point at the end of the enclosing construct, thread one waits for all other threads to either complete execution of the enclosing construct, or be cancelled.

At event 3, thread zero encounters a `cancellation point` directive. Because the cancellation flag has already been set, it immediately cancels the execution of the enclosing construct. Again, the thread waits at the implied barrier, or synchronization point at the end of the enclosing construct, for all other threads, and then continues execution after the enclosing construct.[3]

Thread two encounters a `cancel` directive at event 4. Cancellation has already been requested and this thread immediately cancels execution of the enclosing construct. Thread three encounters a second `cancellation point` directive at event 5. This time (as opposed to event 1), cancellation is already requested and the thread cancels the execution of the enclosing construct.

[3]If the flag is set, the OpenMP runtime function `omp_get_cancellation()` returns the value `true`.

Thread four never encounters any cancellation points and completes execution of the construct.

To ensure timely cancellation, the user must insert `cancellation point` directives at suitable locations throughout the code, which is illustrated in the example in Figure 2.34.

Where exactly does a thread continue execution after the enclosing construct has been cancelled? This depends on the type of construct that is specified as a clause on the `cancel` directive. The clause can be any of these types:

- `parallel` - The thread cancels the current execution and the master thread continues after the implicit barrier at the end of the `parallel` construct. All tasks created in the parallel region are cancelled. In the case of a nested parallel region, cancellation only applies to the innermost parallel region.

- `for` (or `do` in Fortran), or `sections` - The thread cancels the current execution and jumps to the implicit barrier at the end of the worksharing construct. Task cancellation does not occur. If a `reduction` or `lastprivate` clause is specified on a worksharing construct and cancellation has occurred, the value of the corresponding variables are undefined.

  There are two restrictions. The worksharing loop construct (`for`, or `do`) may not contain an `ordered` construct. The `nowait` clause is not allowed on the worksharing constructs supported with cancellation.

- `taskgroup` - The task encountering the `cancel` directive sets the flag for the entire `taskgroup` region, but it continues execution to its end and then continues execution after the `task` region. The same holds for all tasks of the `taskgroup` that have already started execution. All other tasks belonging to the `taskgroup` are cancelled. If a task belongs to multiple (nested) `taskgroup` regions, and encounters a `cancellation point` construct, it checks if cancellation is requested in any `taskgroup` region it belongs to, and responds accordingly.

The use of the `cancel` and `cancellation point` directives is illustrated in Figure 2.34. This code fragment contains a nested parallel region. The outer parallel region spans lines $1 - 18$, and the inner parallel region spans lines $4 - 17$.

When a thread encounters the `cancel` construct at line 11, cancellation is requested for the innermost parallel region. If and when the condition at line 9

```
 1 #pragma omp parallel
 2 {
 3    // Do some work
 4    #pragma omp parallel
 5    {
 6      for (int i = 0; i < N; i++)
 7      {
 8        // Do some work
 9        if ( <some_condition> )
10        {
11           #pragma omp cancel parallel
12         }
13        // Do some more work
14        #pragma omp cancellation point
15        // Do even more work
16       }
17    } // End of inner parallel region
18 } // End of outer parallel region
```

Figure 2.34: **Cancellation of a parallel region** – Execution of the innermost parallel region is cancelled by the thread executing the cancel construct at line 11. Depending on where they are when this happens, the other threads either also cancel their work, or complete execution.

evaluates to `true`, the thread encountering the `cancel` construct terminates execution. At line 11, it is specified that the parallel region is affected. As a result, all remaining work is skipped and the thread waits in the implied barrier at the end of the inner parallel region (line 17). Those other threads that encounter the `cancellation point` at line 14, cancel the execution of the parallel region, and wait in the barrier. It a thread encounters line 14 before cancellation has been requested, the `cancellation point` has no effect. In case a thread is already past the cancellation point, it continues to execute and also waits in the barrier.

The execution of the outer parallel region is not affected by the cancellation. There is no support in OpenMP to request cancellation for an outer parallel region, or for both regions. If needed, the user needs to implement this manually.

Figure 2.35 illustrates the use of the `cancel` construct in a recursive algorithm employing tasks. Tasks are covered in great detail in Chapter 3, starting on page 103.

In this example, a binary tree, called `t`, is searched for the element `e`. Variable `present` is used to mark the termination of the search. Assuming it is initialized to `false`, it is changed to `true`, once the element has been found.

The search is parallelized using tasks that recursively process the left and right subtree of each element.[4]

The tasks are embedded in a `taskgroup` construct (line 7), because that is the granularity for cancellation with tasks. The `taskgroup` is embedded into a `master` construct to ensure that one thread only initiates the parallel search. Instead of `master`, a `single` construct has the same effect.

When the element is found (line 17), cancellation of the `taskgroup` is requested (line 19). Because the task executes to its end, the assignment to `present` may happen either after the `cancel` construct, or before. In this particular example, there is no need to use explicit `cancellation point` directives, because the generation of new tasks is terminated as soon as cancellation has been requested. All previously created, but not yet started, tasks are also terminated then.

If the element is not found, the initial value of variable `present` does not change. The algorithm terminates, because the recursion of `search` ends at leaf nodes, because then `t->left` and `t->right` are `NULL` and the `if` conditions (lines 22 and 27) evaluate to false.

2.4.3 User-Defined Reduction

Until OpenMP 4.0, there was no support to provide an application-specific reduction operation in case the standard reductions were not sufficient. Although one can always implement a reduction explicitly, this is not desirable. Through a feature called *User-Defined Reduction* (UDR) , a user has the option of defining a customized reduction operation and using it in a `reduction` clause to leverage the ease of use that OpenMP provides for such operations.

Recurrence calculations, such as the one shown in Figure 2.36, are common in scientific and engineering programs, but are also encountered in other disciplines. Recall from [2] that the `reduction` clause is provided to parallelize recurrence operations without the need for code modifications.

[4] A detailed example of a recursive algorithm parallelized with tasks can be found in Section 3.3, starting at line 118.

```
 1 void search_parallel(btree *t, element e, bool* present)
 2 {
 3   #pragma omp parallel
 4   {
 5     #pragma omp master
 6     {
 7       #pragma omp taskgroup
 8       {
 9         search(t, e, present);
10       }
11     } // End omp master
12   } // End omp parallel
13 }
14
15 void search(btree* t, element e, bool* present)
16 {
17   if (t->element == e)
18   {
19     #pragma omp cancel taskgroup
20     *present = true;
21   }
22   if (t->left)
23   {
24     #pragma omp task
25       {search(t->left, e, present);}
26   }
27   if (t->right)
28   {
29     #pragma omp task
30       {search(t->right, e, present);}
31   }
32 }
```

Figure 2.35: **Cancellation of a taskgroup in a recursive algorithm** – The binary tree is searched for element `e` and the parallel descent is cancelled if this element is found.

```
1 int sum = 0;
2 int *a, *b;  // Assume allocation and initialization occurred
3
4 #pragma omp parallel for default(none) firstprivate(n)\
5             shared(a,b) reduction(+:sum)
6 for (int i=0; i<n; i++)
7    sum += a[i] * b[i];
```

Figure 2.36: **Example of a recurrence calculation** – The `reduction` clause is used to parallelize the loop. Both the operation (`+`) and result variable (`sum`) are specified as part of the clause.

The reduction operator(s) and variable(s) are specified in the `reduction` clause. By definition, the result variable, like `sum` in this case, is shared in the enclosing OpenMP region.

This operation is implemented as follows: The thread performs the addition operation on a subset of the data. It uses a local, or private, variable to store the partial result. After completion of the operation, a thread updates the global result with its contribution. To avoid a data race, this update is protected through a lock.

The private copy of the reduction variable must be initialized with a value that is mathematically consistent with the reduction operation. The reduction operation must also be mathematically associative and commutative. This is because the order in which threads combine their value to construct the value for the shared result variable is non-deterministic.

A side effect of this is that floating-point results may vary (slightly) from run to run, and/or depend on the number of threads used. This is a numerical effect caused by round-off differences introduced when combining the result. It has nothing to do with the reduction operation as such, but could be a reason for numerical results to be somewhat different.

Figure 2.37 gives an overview of the pre-defined reduction operators supported in OpenMP. In this table, the second column lists the initialization value for the private instance of the reduction variable. This is the neutral element for the specific reduction operation. The third column is labeled "Combiner" and defines the arithmetic operation that needs to be performed to produce the result.

Variables `omp_priv`, `omp_in`, and `omp_out` used in this table are also needed in case of a UDR. This is discussed in more detail below.

Operator	Initializer	Combiner
+	omp_priv = 0	omp_out += omp_in
*	omp_priv = 1	omp_out *= omp_in
-	omp_priv = 0	omp_out += omp_in
&	omp_priv = ~0	omp_out &= omp_in
|	omp_priv = 0	omp_out |= omp_in
^	omp_priv = 0	omp_out ^= omp_in
&&	omp_priv = 1	omp_out = omp_in && omp_out
||	omp_priv = 0	omp_out = omp_in || omp_out
max	omp_priv = *Least(T)*	omp_out = omp_in > omp_out ? omp_in : omp_out
min	omp_priv = *Largest(T)*	omp_out = omp_in < omp_out ? omp_in : omp_out
+	omp_priv = 0	omp_out = omp_in + omp_out
*	omp_priv = 1	omp_out = omp_in * omp_out
-	omp_priv = 0	omp_out = omp_in + omp_out
.and.	omp_priv = .true.	omp_out = omp_in .and. omp_out
.or.	omp_priv = .false.	omp_out = omp_in .or. omp_out
.eqv.	omp_priv = .true.	omp_out = omp_in .eqv. omp_out
.neqv.	omp_priv = .false.	omp_out = omp_in .neqv. omp_out
max	omp_priv = *Least(T)*	omp_out = max(omp_in, omp_out)
min	omp_priv = *Largest(T)*	omp_out = min(omp_in, omp_out)
iand	omp_priv = not(0)	omp_out = iand(omp_in, omp_out)
ior	omp_priv = 0	omp_out = ior(omp_in, omp_out)
ieor	omp_priv = 0	omp_out = ieor(omp_in, omp_out)

Figure 2.37: **The pre-defined reduction operators supported by the reduction clause in C/C++ and Fortran** – The upper half of the table is for C/C++. The lower half of the table is for Fortran. This table lists the pre-defined reduction operators, along with their associated initializer and combiner expressions. `Least(T)` and `Largest(T)` are the least and largest representable values of the arithmetic type T.

While the `reduction` clause provides significant convenience to the user, certain applications have their own reduction operation that is not supported by OpenMP. Examples of that include the merging of lists or sets, the computation of a standard deviation over a set of elements, etc. It is always possible to explicitly code a reduction, but this adds to the complexity of the code, and it may not perform optimally.

```
 1 int *a; // Assume allocation and initialization occurred
 2 int result = INT_MAX;
 3
 4 #pragma omp declare reduction(my_abs_min : int :\
 5   omp_out = abs(omp_in) < omp_out ? abs(omp_in) : abs(omp_out))\
 6   initializer (omp_priv = INT_MAX)
 7
 8 #pragma omp parallel for default(none) firstprivate(n)\
 9             shared(a) reduction(my_abs_min : result)
10 for (int i=0; i<n; i++) {
11    if (abs(a[i]) < result) {
12       result = abs(a[i]);
13    }
14 }
```

Figure 2.38: **Example of a User-Defined Reduction** – The definition of a UDR used in combination with the `reduction` clause to compute the minimum absolute value of the array elements in parallel.

This is why the UDR was introduced in OpenMP 4.0. A UDR is defined through the `declare reduction` directive and consists of the following three components:

- Identifier - A name, or symbol, that identifies the reduction and the type(s) to which it applies.
- Combiner - A recipe to generate a result. It is expressed as an operation involving the two symbolic variables `omp_in` and `omp_out`, thus defining the actual reduction operation as applied on any given two arguments.
- Initializer - This is used to define the initial value for the private instances of a reduction variable.

The `declare reduction` directive must be used to define a UDR. Before introducing the formal syntax, the example in Figure 2.38 is presented and discussed. In this code, a UDR is defined to compute the minimum of the absolute values of an array.

At lines $4 - 6$, the `declare reduction` directive defines the UDR called `my_abs_min`. It returns a value of type `int`.

#pragma omp declare reduction*(reduction-identifier : typename-list : combiner) [initializer-clause] new-line*
!$omp declare reduction*(reduction-identifier : typename-list : combiner) [initializer-clause]*

Figure 2.39: **Syntax of the declare reduction directive in C/C++ and Fortran** – This directive defines a user-defined reduction. The various components are explained in the text. Figure 2.40 contains an explanation of the terminology.

Name	Description
reduction-identifier	A C, C++, or Fortran identifier or one of the pre-defined OpenMP reduction operators.
typename-list	The list with data type(s) to be used.
combiner	An expression in C/C++. In Fortran, it is an assignment statement, or subroutine name, followed by an argument list.
initializer-clause	This is of the form `initializer`*(initializer-expr)* with *initializer-expr* being one of the following: `omp_priv` = *initializer* (C/C++) `omp_priv` = *function-name(argument-list)* (C/C++) `omp_priv` = *expression* (Fortran) `omp_priv` = *subroutine-name(argument-list)* (Fortran)

Figure 2.40: **Terminology used with the declare reduction directive** – This table explains the various elements used to define a reduction.

The *combiner* is defined at line 5. Symbolic variable `omp_in` represents the variable used to store the partial result. This variable is private to each thread. Likewise, symbolic variable `omp_out` refers to the result of the combiner operation.

The expression at line 5 returns the absolute value of `omp_in` when it is less than the previous value of `omp_out`; otherwise, it returns the absolute value of the latter variable.

At line 6, another special variable, `omp_priv`, is set to the initial value each thread assigns to the local variable `omp_in`. In this case, it is `INT_MAX`, the largest integer value the system provides.

In this initialization expression, only special variables `omp_priv` and `omp_orig` are allowed. Although not used here, `omp_orig` represents the original value of the reduction variable. This variable is not allowed to be modified.

```
1 typedef std::vector< float > TVec;   // Assume some type here
2
3 #pragma omp declare reduction(+ : TVec : \
4         std::transform(omp_in.begin(),  omp_in.end(), \
5                        omp_out.begin(), omp_out.begin(), \
6                        std::plus< T >() ) ) \
7         initializer (omp_priv(omp_orig))
```

Figure 2.41: **Example of a User-Defined Reduction on C++ arrays** – The use of a UDR allows for the parallelization of the loop employing a C++ array type.

Now that the elements of the reduction operation are defined, it may be used in a regular `reduction` clause. This is shown at line 9, where the name `my_abs_min` is used in the clause.

The loop spanning lines $10 - 14$ does not need to be modified. At runtime, the reduction operation is executed the same way a pre-defined reduction is handled.

After this example, it is time to look at the more formal description. The syntax of the `declare reduction` directive is given in Figure 2.39. The terminology used is defined in Figure 2.40.

OpenMP defines the special variables `omp_orig`, `omp_priv`, `omp_in`, and `omp_out`. They are all of the same type and the only variables to be used when defining a UDR operation.

This might raise the question why the description of *typename-list* in Figure 2.40 shows that a list with multiple types is allowed. This is a convenience. It avoids the need to repeat the `declare reduction` directive for the data types to be supported. With a list, it is as if the definition is repeated for all the types in the list.

Neither the number of times the combiner is executed, nor the number of times and the order in which the initializer expression is evaluated, are implementation-defined. The results are undefined if a program makes any assumption about this.

Instead of the initializer expression, it is also possible to provide a function, or subroutine name in Fortran, with an argument list. In this case, one of the arguments must be the special variable `omp_priv`. For C++ programs, the `initializer` clause also accepts an expression that is not an assignment to `omp_priv`. This is demonstrated in Figure 2.41.

The code shown in Figure 2.41 implements a UDR called `+` to perform the reduction on arrays. The C++ data type `std::vector` represents the array, for which a

depend *(dependence-type: iteration-vector)*
depend *(dependence-type)*

Figure 2.42: **Syntax of the depend clause for the doacross loop in C/C++ and Fortran** – In the context of a `doacross` loop, this clause accepts two new keywords for the dependence-type. This can either be `sink`, followed by the iteration vector, or `source`, which does not have any further qualifiers. Refer to the main text for an explanation of these terms.

given type `T` is assumed. The code provides the `+` reduction operation for this data type. For the standard data types, the pre-defined reduction operation is used.

The combiner (lines $4-6$) makes use of the `std::transform` function. The initializer at line 7 specifies that every thread's private copy is constructed with the copy constructor, using the original reduction variable as the argument. The advantage is that by using this feature, the thread's private arrays in this example do not need to have a known size at compile time. Instead, the length is derived from the original list item.

2.4.4 The Doacross Loop

As of OpenMP 4.5, an additional type of parallel loop is supported. It is called the "doacross loop" [24].

A `doacross` loop has a loop-carried dependence. Such loops cannot be parallelized in a straightforward way, but through compiler transformations and/or additional synchronization, the code may be partially parallelized. It is important to note that these are *static* dependences. This technique cannot be used to handle runtime dependences.

In contrast with other newly introduced features, there is no specific construct to implement this functionality. Instead, it is supported through extensions on the `ordered` and `depend` clauses. The latter was introduced in OpenMP 4.0 for tasking. In OpenMP 4.5, the `depend` clause has been augmented to add functionality for tasks, but is also used to implement the `doacross` feature.

The two main new elements are called *sink* and *source*. The former takes an additional description, the *iteration vector*, to describe the dependences *relative* to the current loop variable. The syntax of this vector is $x_1\,[\pm d_1], x_2\,[\pm d_2], \ldots, x_n\,[\pm d_n]$. In this notation, x_i is the name of the loop variable of the i-th nested loop, d_i is

```
 1 #pragma omp parallel default(none) shared(n,a,b,c)
 2 {
 3   #pragma omp for ordered(1)
 4   for (int i=1; i<n; i++)
 5   {
 6      #pragma omp ordered depend(sink:i-1)
 7         a[i] = a[i-1] + b[i];
 8      #pragma omp ordered depend(source)
 9
10      c[i] = 2*a[i];
11
12    } // End of for-loop
13 } // End of parallel region
```

Figure 2.43: **Code fragment using a doacross loop** – The doacross loop is defined through the combination of the `ordered` and `depend` clauses.

a *constant* non-negative integer, and n is the value as specified in the `ordered(n)` clause on the loop directive.

Before we continue, it is probably best to show an example first. The parallel region shown in Figure 2.43 contains a `doacross` loop to parallelize the update of vector `a`.

The computation of `a[i]` at line 7 has a data dependence, but it can be partially parallelized. Another thing to note is that the computation of `c[i]` at line 10 depends on the recently computed value `a[i]`. Due to this dependence, this loop cannot be fully parallelized, but there is reduced parallelism that may be exploited by using the support for `doacross` loops. The `doacross` loop is within the parallel region and spans line $3 - 12$.

The `ordered` clause at line 3 should look familiar. Until OpenMP 4.5, this enforced serial execution of the ordered code block defined within the loop. The clause has been extended to support a number between parentheses to indicate the loop is a `doacross` loop. The number, which is 1 in this case, indicates the depth of the parallelism. In general, this number specifies the number of perfectly nested loops constituting the `doacross` loop. For example, if there are three loops with dependences, this number must be 3.

The `ordered` directives at line 6 and 8 are mandatory. They define the depen-

dence region. The `depend` clause at line 6 uses the one dimensional iteration vector to specify that there is a dependence of loop iteration `i` on `i-1` ($n = 1$, $x_1 = i$, the sign is negative, and $d_1 = 1$).

In general, *all* dependences in *all* dimensions must be specified on the `ordered` clause that has the `sink` keyword. If, for example, there are two loops and three dependences, there must be three `depend` clauses, and each clause must use an index vector with two components of the type $x_1 [\pm d_1], x_2 [\pm d_2]$, like `i-1,j+1`.

The `depend` clause with the `source` keyword at line 8 is mandatory and forces the runtime system to make the result `a[i]` available. This is to ensure the computation at line 10 uses the correct value for `a[i]`. If this clause is left omitted, the program deadlocks. The same is true if the dependences are not described correctly.

2.5 Concluding Remarks

In this chapter, a broad overview of where OpenMP stands today has been given. Many OpenMP 2.5 constructs have been enhanced, including the introduction of the `collapse` clause to better support nested loops, refinements on the `critical` construct, plus improved support for the `atomic` construct to support various types of updates.

But there is much more. Through the `doacross` construct, certain loops with dependences can now be parallelized. Another new feature is cancellation, to provide a more robust handling of errors during parallel execution. User-defined reductions allow the user to write application specific reductions.

In addition to all of this, major new functionality has been introduced. Tasking, affinity control, vectorization using SIMD instructions, and the support for heterogeneous architectures, all make OpenMP more suitable for a wider range of systems and applications. These four new features are so substantial that they deserve their own chapter. That is what the remainder of this book is about.

3 Tasking

Tasking was introduced in 2008 and provided a major addition to OpenMP 3.0 [1]. Thanks to this feature, algorithms with an irregular and runtime dependent execution flow may now be parallelized. Before tasking, this was not always possible, or was at least tedious and sometimes downright ugly.

A relatively simple example is a while-loop with independent chunks of work in the loop body. Although these chunks could be executed simultaneously, OpenMP did not provide the means to do so without changing the source code. In the best case, the loop could be manually transformed into a for-loop and then parallelized, but few users would like to do so.

Tasking provides an elegant solution to this problem. A queueing system dynamically handles the assignment of threads to the chunks of work that need to be performed. The threads pick up work from the queue until the queue is empty.

By design, the initial version of tasking was rather rudimentary. The idea was to decide on additional features through feedback from the users. That is indeed what has happened in subsequent releases of the specifications, and we are quite sure the current feature set is not final.

In this chapter, the tasking concepts, as well as all the features, are presented and discussed in detail. We start with a series of examples. Collectively they touch upon many of the features supported with tasking. In many cases, the features are only described briefly, but the follow-on sections after the examples go into much more detail.

3.1 Hello Task

Simply stated, an OpenMP *task* is a block of code contained in a parallel region that can be executed simultaneously with other tasks in the same region.[1]

Before we show the first example, more should be said about tasking in OpenMP. As with almost every feature in OpenMP, a task is part of a parallel region. Depending on the specific situation, this could pose a challenge. Without special precautions, the same task(s) may be executed by different threads. This is very different from the worksharing constructs, where it is well-defined how the work is distributed over the threads. As we shall see shortly, tasks are much more dynamic, and the same approach cannot be applied.

[1] This is a simplification. Subsequent sections contain the more formal coverage and definitions.

It may be needed that each thread executes all tasks, but it is more likely that different threads must execute different tasks. The problem is that, unlike a work-sharing construct, the work to be performed is not known upfront. This is why something special needs to be done.

The solution chosen by the OpenMP language committee makes sense, but at first glance may not seem intuitive. To guarantee each task is generated only once, the tasks could be embedded in a `single`, or `master` construct. There is a potential problem with the latter though, which is discussed in Section 3.1.3.

This is already a novel use of these constructs, but the thing that tends to confuse users new to tasking in OpenMP most, is that the tasks are not necessarily executed where they are defined in the source code. Tasks are *guaranteed* to be executed only at special and well-defined points in the code. Such points are implied with some constructs, but they are also under explicit control of the user.

The idea behind this approach is that one thread generates the tasks, while the other threads execute them as they become available. Eventually the generating thread may participate in the execution part, after it has completed generating the tasks. How this works is best illustrated with the example that follows next.

3.1.1 Parallelizing a Palindrome

Consider the following problem. We want to write a program that either prints `"race car "`, or `"car race "`, with no preference for either phrase.[2] At runtime, there is an equal chance of seeing either one of these phrases on the screen.

This could easily be solved using parallel sections, but let's assume these are not available. Instead, we like to use the concept of a task, briefly introduced above. The code fragment using tasks is shown in Figure 3.1.

The parallel region spans lines $1-10$ and has one single region only. This would not make any sense if it were not for the new task directives shown here. Each `#pragma omp task` directive defines a task. The code enclosed in the curly braces ({ and }) defines the task. The assumption is that tasks can execute independently. It is the responsibility of the user to ensure this produces correct results.

There are two tasks. One prints `"race "` and the other prints `"car "`. The order in which the tasks are executed is non-deterministic and most likely varies from run to run. For a correct parallel program, this should not affect the result. Note the trailing space in both print statements.

[2] Yes, the trailing space in the output matters here.

```
 1 #pragma omp parallel
 2 {
 3   #pragma omp single
 4   {
 5     #pragma omp task
 6       {printf("race ");} // Task #1
 7     #pragma omp task
 8       {printf("car ");}  // Task #2
 9   } // End of single region
10 } // End of parallel region
```

Figure 3.1: **Code fragment with two tasks** – There are two tasks. One prints the word "race ". The other one prints "car ". The order in which tasks are executed is runtime dependent and may vary across multiple runs.

How about this single region then? Here is how this code is executed. Assume there are two threads. One thread encounters the `single` construct and starts executing the code block. Both tasks are generated, but not yet executed. Meanwhile, the other thread falls through and waits in the implied barrier at the end of the `single` construct.

In the case of tasks, idle threads do not really wait in the barrier though. Instead, these threads are available to execute the tasks, as they are generated.

In this simple case, that means one thread generates the two tasks consisting of a single print statement. Any available thread executes the tasks, but not until it has reached the barrier. The executing thread(s) print `"race "` and `"car "`, but not necessarily in this order.

The thread generating the tasks also ends up in the barrier, once it has completed the generation phase. Therefore, theoretically both threads could print a word each, but both tasks are so short that it is more likely the second thread prints both. The order is non-deterministic however.

This code works, but it is not a real sentence yet. Let's modify the program to print either `"A race car "`, or `"A car race "`. The relevant program fragment is listed in Figure 3.2.

By inserting a print statement before the two tasks, it is guaranteed that `"A "` is printed first. Next, the tasks are generated and executed in the barrier that is implied with the `single` construct.

```
 1 #pragma omp parallel
 2 {
 3   #pragma omp single
 4   {
 5     printf("A ");
 6     #pragma omp task
 7       {printf("race ");} // Task #1
 8     #pragma omp task
 9       {printf("car ");}  // Task #2
10   } // End of single region
11 } // End of parallel region
```

Figure 3.2: **Augmented code fragment with two tasks** – The print statement at line 5 is executed by one thread only and before the tasks are generated. This ensures the sentence starts with "A ".

3.1.2 Parallelizing a Sentence with a Palindrome

Although longer, this is still not a full sentence. We want to append `"is fun to watch."` to it and also include a new-line symbol. That seems easy, doesn't it? The code fragment in Figure 3.3 adds a print statement before the end of the single region.

Although this seems straightforward, the output is not what we would expect to see. The six possible results are listed in Figure 3.4. Which one of these is printed at runtime depends on the number of threads used, the load on the system, and the implementation details of tasking.

The sentence always starts with `"A "`, but what happens next "depends." Remember that the tasks are executed by threads waiting in the barrier.

To simplify the discussion, the reference numbers in the first column of Figure 3.4 are used. The thread executing the single region is referred to as T_S.

In one scenario, assuming there are at least two threads, the other thread(s) could arrive in the barrier after thread T_S has executed the single region. In this case, either output 1 or 2 is printed.

It may also happen that one or more thread(s) arrive in the barrier before T_S has completed the single region. Then, one of the outputs $3-6$ is printed. In the case of 3 or 4, one task was executed after T_S finished. If this thread finishes after the other threads, outputs 5 or 6 are printed.

```
 1 #pragma omp parallel
 2 {
 3   #pragma omp single
 4   {
 5     printf("A ");
 6     #pragma omp task
 7       {printf("race ");} // Task #1
 8     #pragma omp task
 9       {printf("car ");}  // Task #2
10     printf("is fun to watch.\n");
11   } // End of single region
12 } // End of parallel region
```

Figure 3.3: **An incorrect sentence with a palindrome and two tasks** – This program does not produce the desired result. The print statement at line 10 is executed by one thread only. Although at the end of the single region, this line may be printed before the tasks get to print their word.

Reference ID	Text printed
1	`A is fun to watch.` `race car`
2	`A is fun to watch.` `car race`
3	`A race is fun to watch.` `car`
4	`A car is fun to watch.` `race`
5	`A race car is fun to watch.`
6	`A car race is fun to watch.`

Figure 3.4: **The six possible outputs from the program in Figure 3.3** – The first column contains a reference number used in the explanation. The second column lists the possible lines printed from this program.

Any of the above scenario's are possible. It depends on the number of threads used. With two threads the behavior is different compared to using three threads. Also the load on the system has an impact. If, for example, one thread gets delayed, the output may be different.

```
 1 #pragma omp parallel
 2 {
 3   #pragma omp single
 4   {
 5     printf("A ");
 6     #pragma omp task
 7       {printf("race ");} // Task #1
 8     #pragma omp task
 9       {printf("car ");}  // Task #2
10     #pragma omp taskwait
11       printf("is fun to watch.\n");
12   } // End of single region
13 } // End of parallel region
```

Figure 3.5: **A correct sentence with a palindrome and two tasks** – At line 10, the `taskwait` construct forces the pending child tasks to complete before execution resumes. In this way it is guaranteed the words are printed in the right order.

The tasking implementation details matter also. A work queueing system is used to handle the generation and execution of tasks. There are several different ways to assign threads to tasks, and this impacts the order of execution.

In a way, none of this matters though, because this program is not guaranteed to produce the output we want to see. What we need to do is to guarantee the two tasks are completed *before* the print statement at line 10 is executed. In this simple case, we could easily solve this by moving the last print statement out of the parallel region, but what if this is not possible? For such cases, the `taskwait` construct can be used. This feature is used in Figure 3.5 to achieve the desired result.

Execution proceeds as follows. Thread T_S enters the single region and executes the statement at line 5, printing `"A "`. Next, it generates the two tasks with the print statement. At some point in time, the other threads enter the implied barrier that is part of the `single` construct. Once the tasks become available, they will be executed by those threads.

The `taskwait` construct forces thread T_S to wait, until the two tasks have completed. In this case this means it must wait for `"race car "` or `"car race "` to have been printed. After that, it prints `"is fun to watch."`.

3.1.3 Closing Comments on the Palindrome Example

In this example the `single` construct is used to ensure each task is generated only once. Theoretically, the same could be achieved using the `master` construct, but there is a problem with that. In the barrier, all explicit tasks generated by the team must be executed to completion. The `master` construct has no implied barrier, however.

In this case, no harm is done, because then the tasks are executed in the barrier that is implied with the parallel region. This is, of course, specific to the example. In general, if the `master` construct is used with tasks, care needs to be taken when they are executed.

Generally, this is not the best approach anyhow. The master thread tends to be quite busy with the overall execution already. Why add more burden to it? Instead, we recommend using the `single` construct, possibly with the `nowait` clause. In the case of the latter, the same caveat about task completion holds.

Another option is to use parallel sections, but with only one single section. As with the `single` construct, the `nowait` clause can be used to eliminate the redundant barrier.

Another thing worth mentioning is that in this particular example, there is no data environment to worry about. This is very unlikely to be the case in more realistic applications. What you need to know about the data environment in tasks is extensively covered in the sections to follow, in particular in Section 3.5 on page 136.

3.2 Using Tasks to Parallelize a Linked List

The previous example introduced tasking at a very basic level and was somewhat crafted in the sense that other OpenMP constructs are more suitable and easier to solve the problem. The next example does not have such an easy solution though.

This program uses a linked list. The assumption is that each node in the linked list represents work independent of the other nodes. The independence opens up opportunities for parallelism, but unfortunately a linked list needs to be traversed sequentially, ruining the parallelism.

That is, unless we can generate a list with all the nodes first. The data elements from this list can then be processed simultaneously. This makes it a natural candidate for tasking.

```
 1 struct linked_list {
 2   int64_t value;
 3   struct linked_list *next;
 4 };
 5
 6 typedef struct linked_list my_data;
 7
 8 static int function_call_count = 0;
 9
10 int main(int argc, char *argv[])
11 {
12  my_data *previous, *ptr, *current, *head_of_list;
13
14  int64_t ntasks;
15  int64_t no_of_tasks_failed = 0;
16
17  (void) get_parameters(argc, argv, &ntasks);
18
19  }
```

Figure 3.6: **The first part of the sequential implementation of the linked list program** – The key data structures are defined and the input parameters are read.

3.2.1 The Sequential Version of the Linked List Program

This program is relatively lengthy, but for reasons that become clear soon, most of the code is presented and discussed. The initial part of the code is listed in Figure 3.6.

This first part is fairly straightforward. At lines $1-4$, the key list data structure, called `linked_list`, is defined. The list has two elements only: an integer value and a pointer to the next node in the list. To make things easier, an alias named `my_data` to reference the linked list is defined. Line 8 defines and initializes a static (global) variable called `function_call_count` to be used later on. Lines $12-15$ define other program variables needed. This includes four pointers to `my_data`. At line 17, a single input parameter is returned by the function call. This variable is called `ntasks` and defines the length of the list. As we shall see shortly, the number of tasks corresponds to the number of entries in the list, hence the name. Although

```
20  for (int64_t i=0; i<ntasks; i++) {
21      if ( (ptr = (my_data *) malloc(sizeof(my_data))) == NULL ) {
22         perror("ptr"); exit(-1);
23      }
24
25      if (i == 0 ) {
26         head_of_list = ptr;
27      } else {
28         previous->next = ptr;
29      }
30
31      ptr->value = i+1;
32      ptr->next  = NULL;
33      previous   = ptr;
34  }
```

Figure 3.7: **The second part of the sequential implementation of the linked list program** – By design, the linked list has `ntasks` entries. A for-loop with this length is used to construct the list.

seemingly a detail, variables `ntasks` and `no_of_tasks_failed`, are declared to be a 64-bit integer. This is because the number of tasks may easily be very large.

In the next block of code, the linked list is initialized. The code fragment is listed in Figure 3.7. By design, the list has `ntasks` entries and a for-loop is used to initialize these entries. This loop spans lines $20 - 34$. Memory for the next node in the list is allocated at lines $21 - 23$. Variable `ptr` contains the pointer to the new memory block. At line 26, the starting point of the linked list is saved in variable `head_of_list`. Upon subsequent loop iterations, the previous value of the pointer is saved (line 28). Lines $31 - 32$ assign a value to the data part of the list and the `NULL` pointer to the next node. The latter is because the end of the linked list needs to be terminated by the `NULL` pointer. At line 33, the list is updated with the new pointer.

After execution of this for-loop, a `NULL`-terminated linked list, consisting of `ntasks` nodes has been created. Variable `head_of_list` contains the start of the list.

The most interesting and relevant part of this example is shown in Figure 3.8. This is where the linked list is processed. Pointer variable `current` is used to keep track of the most recent node. At line 35, it is initialized to the start of the linked list.

```
35  current = head_of_list;
36  while (current != NULL) {
37    int64_t return_value;
38    return_value = do_work(current->value);
39    if ( return_value < 0 ) {
40       no_of_tasks_failed++;
41    }
42    current = current->next;
43  } // End of while loop
```

Figure 3.8: **The third part of the sequential implementation of the linked list program** – This is where the linked list is processed. At line 35, the most recent node location is set to the start of the list. The while-loop spans lines $36 - 43$ and executes a function called `do_work()` for each data element in the list. The parameter to the function call is the value of the data element. At line 42, the pointer advances to the next node in the list.

The key part of this program is the while-loop spanning lines $36 - 43$. This loop terminates in case the pointer to the next node is `NULL`. At line 38, a function called `do_work()` is called. It has one input parameter, `current->value`, that contains the value of the data element in the current node.

In case there was a problem executing `do_work()`, a negative value is returned. This is checked for at line 39 and if a problem has occurred, status variable `no_of_tasks_failed` is incremented by one (line 40). Note that it was initialized to zero at line 15 in Figure 3.6. After completion of the while-loop, the main program can use the value of variable `no_of_tasks_failed` to check for failures.

The details of function `do_work()` need to be discussed, because it requires some attention in the next section. The source code is listed in Figure 3.9. This function emulates a runtime error and uses a global variable, called `function_call_count`, to control the value returned to the caller.

At line 3, this variable is updated each time the function is called. By design, the first `MAX_ERRORS` calls to this function return a negative value, emulating a runtime error. After this, the function returns zero, indicating there were no errors. This is what the code at lines $5 - 9$ accomplishes. This block of code could be replaced by a single statement, but as shown in the next section, these lines need special care. This would be equally true for the one-liner. Note that the input parameter is not used, but has been included here to make the code look more realistic. In a

```
 1  int64_t do_work(int64_t input_value)
 2  {
 3    function_call_count++;
 4
 5    if ( function_call_count <= MAX_ERRORS ) {
 6       return(-1);
 7    } else {
 8       return(0);
 9    }
10 }
```

Figure 3.9: **The source code of function do_work()** – This function is a template to emulate an error that may have occurred during execution. For demonstration purposes, a global variable, called `function_call_count`, is updated.

real-world application, this value is most likely needed.

This concludes the description of the sequential version of this program. From now on, the focus is on the core part of the program, as shown in Figure 3.8. The challenge is going to be to parallelize it.

3.2.2 The Parallel Version of the Linked List Program

The `NULL` terminated while-loop is not straightforward to parallelize, because, unlike a for-loop, it is not known at runtime how often the loop body is going to be executed.

The main thing to realize here is that there is actually no need to know this. What if there was a dynamic queueing system to handle the assignment of threads to the chunks of work that need to be performed? If such a system is available, the threads can pick up work from the queue until the queue is empty. This is somewhat akin to the `dynamic` and `guided` loop iteration scheduling algorithms, available for parallel loops. The big difference is that the tasking concept is much more flexible and dynamic.

This is exactly the idea behind tasking. With tasking, it is up to the implementation to set up an internal (queueing) system that ensures the tasks are generated and executed. This system also decides upon the assignment of threads to the tasks in the queue. Therefore, we need to find a way to define the execution of the

```
 1 current = head_of_list;
 2 #pragma omp parallel firstprivate(current) \
 3                      shared(no_of_tasks_failed)
 4 {
 5   #pragma omp single nowait
 6   {
 7     printf("Thread %d executes the single region\n",
 8            omp_get_thread_num());
 9     while (current != NULL) {
10       #pragma omp task firstprivate(current) \
11                        shared(no_of_tasks_failed)
12       {
13         int64_t return_value;
14         return_value = do_work(current->value);
15         if ( return_value < 0 ) {
16              #pragma omp atomic update
17                no_of_tasks_failed++;
18         }
19       } // End of task
20       current = current->next;
21     } // End of while loop
22   } // End of single region
23 } // End of parallel region
```

Figure 3.10: **The parallel implementation of the linked list program** – The entire while-loop is embedded inside a parallel region, spanning lines $2-23$. This region consists of one single region (lines $5-22$). The thread executing this region creates the tasks defined at lines $10-19$. Tasks are executed in the implicit barrier at the end of the parallel region (line 23). For more details, refer to the main text.

while-loop in terms of tasks. Due to the sequential nature of scanning the nodes of the list, a `single` construct is ideally suited to generate the tasks.

The relevant code fragment is shown in Figure 3.10. As before, the current node in the linked list is stored in variable `current` and initialized to `head_of_list` (line 1). The entire while-loop is embedded inside a parallel region, spanning lines $2-23$.

The scoping is fairly straightforward. All threads get their private copy of pointer variable `current`. Thanks to the `firstprivate` clause, every thread has a copy, initialized to point to the first node in the list. Variable `no_of_tasks_failed` needs

to be updated by all threads and is therefore a shared variable. The same scoping is used for the tasks (lines $10-11$).

The parallel region consists of one relatively gigantic single region starting at line 5 and ending at line 22. Similar to what was seen in the previous example in Section 3.1.1, this approach is used to ensure one thread generates the tasks. For diagnostic purposes, the thread ID of the thread executing this single region is printed (lines $7-8$).

The two adjacent implied barriers at lines $22-23$ are not needed. We can easily eliminate the one at line 22 by using the `nowait` clause in line 5. As a result, all other threads wait in the implied barrier at the end of the parallel region at line 23.

The tasks are defined at lines $10-19$. Ignoring the details of the tasks for a moment, we know that this one thread executes the while-loop and scans through the linked list. This is exactly the same as we saw earlier with the sequential version. When this thread encounters the task region (lines $10-19$), it generates the appropriate code, plus data environment, to execute the specific task. At line 20, the pointer advances to the next node and at line 10 the next task can be generated. This is still a sequential operation. Meanwhile, the other threads wait in the barrier at line 23, but they are not really waiting. Instead, they start executing the tasks, generated by the thread executing the single region. Once this thread has finished, and if there is still work left to do, it joins the other threads and starts executing tasks from the queue.

Now that it is covered in what way these tasks are generated and executed, it is time to zoom in on the code that defines a task. At line 13, variable `return_value` is declared. It is private to the task and used to store the value returned by function `do_work()`.[3]

At this point, things get interesting, because we want to detect a possible error that has occurred when executing this function. For good reasons it is not allowed to jump out of a parallel region, because then other threads wait in the next barrier for one or more threads that never arrive there.

In Section 2.4.2 it is shown how the thread cancellation feature can be used for these situations, but in this example the more conventional approach is chosen. After the parallel region has completed, the value of variable `no_of_tasks_failed`

[3]This variable can be eliminated, but is used here to show the (simple) use of such a private variable.

```
 1 int64_t do_work(int64_t input_value)
 2 {
 3   int64_t TID, return_value;
 4
 5   TID = omp_get_thread_num();
 6
 7   #pragma omp critical
 8   {
 9     function_call_count++;
10
11     printf("TID = %-6ld Input: %-6ld function_call_count: %d\n",
12            TID, input_value, function_call_count);
13
14     if ( function_call_count <= MAX_ERRORS ) {
15        return_value = -(TID+1);
16     } else {
17        return_value = 0;
18     }
19   } // End of critical region
20
21   return(return_value);
22 }
```

Figure 3.11: **The thread-safe version of function do_work()** – This is the thread-safe version of the sequential code shown in Figure 3.9. The update at line 9 needs to be guarded against a data race and the read of this variable at line 14 has to be protected as well. This is achieved by using a critical section spanning lines $7 - 19$.

may be checked. If it is non-zero, an error has occurred and the program can take appropriate action.

Simply incrementing variable `no_of_tasks_failed` in case the function returns a negative value, is correct in the sequential version. For the parallel version, it is necessary to prevent that multiple threads simultaneously update this variable. The solution is straightforward. This is a perfect case to use an atomic update (line $16 - 17$) to avoid the data race. See also Section 2.1.6 for more details on the `atomic` construct.

There is one more thing left to do. Function `do_work()` needs to be made thread-safe. The modified, parallel, source code of this function is listed in Figure 3.11. It is the thread-safe version of the sequential code listed earlier in Figure 3.9.

At line 5, the thread ID is assigned to variable `TID`. This is used to ensure that each invocation of this function returns a unique negative number in case an error has occurred. As before, a negative value is returned the first `MAX_ERRORS` times this function is executed. After that, a value of zero is returned. In this way, the number of erroneous function calls is easily controlled and simulated.

In a sequential program, it is easy to keep track of the number of times the function is called. Global variable `function_call_count` is simply updated each time the function is called. In a multi-threaded environment, the update of a global variable requires care. Without any precaution, a data race occurs at line 9, because multiple threads are allowed to update the same variable at the same time. Typically, an atomic update is used to avoid this, but in this case there is also a potential data race between the write at line 9, and the read at line 14. This is why the entire code block is a critical region. It is assumed that in a real-world application quite some work is performed here. Otherwise, the cost of the critical region offsets, or even outweighs, any performance gain.

Last, but not least, the return value of this function needs to be handled. In the sequential version in Figure 3.9, the `return()` function was called at lines 6 and 8, using the appropriate argument. Since control cannot be transferred out of a critical region, calling the `return()` function is not possible. Instead, the desired value is assigned to variable `return_value` and this is used as the argument in the `return()` call at line 21.

Note that when setting the return value at line 15, a value of one is added to variable `TID`. Otherwise the return value for the master thread is zero, and not distinguishable from the return value set upon correct execution.

For demonstration purposes a print statement has been included in lines $11 - 12$. The thread ID, the value of the input parameter, and the number of times the function has been called, are printed. This program has been executed using 10 tasks and 20 threads. The results in Figure 3.12 show that the generating thread, 17, also executes two tasks. Not all threads execute the same number of tasks. The other eight tasks are distributed over six threads. Although there are sufficient threads to execute one task each, this is not what happens at runtime. Apparently, only seven threads were put to work. This demonstrates the dynamic behavior that comes with tasking.

```
Thread 17 executes the single region
TID = 13     Input: 1      function_call_count: 1
TID = 11     Input: 4      function_call_count: 2
TID = 17     Input: 10     function_call_count: 3
TID = 12     Input: 5      function_call_count: 4
TID = 11     Input: 8      function_call_count: 5
TID = 13     Input: 6      function_call_count: 6
TID = 14     Input: 2      function_call_count: 7
TID = 15     Input: 7      function_call_count: 8
TID = 17     Input: 9      function_call_count: 9
TID = 8      Input: 3      function_call_count: 10
```

Figure 3.12: **Example output from the parallel linked list program** – The program was executed for `ntasks=10` and using 20 threads. This output demonstrates that the generating thread, 17, also executes two tasks.

3.2.3 Closing Comments on the Linked List Example

This type of algorithm was notoriously hard to parallelize and one of the reasons to introduce the tasking concept in OpenMP.

What confuses users new to tasking, is that the parsing of the linked list is still a sequential process. Due to the pointer-chasing nature of scanning such a list, this cannot be avoided, but there is also no need to do so.

With tasks, as soon as an entry from the list has been created, the processing can start in a natural way.

3.3 Sorting Things Out with Tasks

Through nested parallelism, OpenMP has supported the parallelization of recursive algorithms, but this is a rather heavy and rigid mechanism. The typical cost of a parallel region is non-negligible. To make it even worse, each parallel region has an implied barrier and there is no way to omit it. Both drawbacks make it very difficult to efficiently parallelize, and load balance, fine-grained portions of work in recursive algorithms.

This example shows how a recursive divide-and-conquer algorithm can be parallelized using tasks. As with the other examples, this is a template for how to tackle similar problems in this category.

The popular quicksort algorithm has been selected [22]. It is conceptually simple, but has a structure that allows us to demonstrate various tasking features needed to implement the parallelism.

3.3.1 The Sequential Quicksort Algorithm

The quicksort algorithm uses a divide-and-conquer algorithm. Such an algorithm splits the work in two independent sections. In a recursive manner, each section is then split again. The sequential quicksort algorithm to sort an array is defined as follows:

1. Select an element, called a *pivot*, from the array.
2. In the *partitioning* phase, the array is re-ordered, such that all elements with values less than the pivot are placed before the pivot. All elements with values greater than the pivot are moved after it.[4] After this partitioning step, the pivot is guaranteed to be in its final position.
3. Recursively apply the above two steps to the elements to the left and right of the pivot.
4. The recursion ends when there is only one, or no, element left.

In the example discussed here, a specific implementation of the algorithm outlined above is presented. This is certainly not the only version. Among other choices, the selection of the pivot and partitioning phase are parameters affecting the performance. The partitioning algorithm used in this section is listed in Figure 3.13.

Function `choosePivot` is defined at lines $1-3$. It returns $\mathit{floor}(lo+hi)/2$.[5] If $lo+hi$ is an even number, it returns the array value in the center. Otherwise, it returns the integer value rounded down from $(lo+hi)/2$. For example, for $lo=3$ and $hi=6$, a value of 4 is returned.

The actual partitioning algorithm is defined at lines $10-24$. After the pivot value and corresponding index are defined (lines $10-11$), the pivot value is moved to `a[hi]`, the end of the array section. The value that was originally there has been moved by the `swap` function to where the pivot was.

[4] Equal values can go either way

[5] The Glossary contains the definition of this function.

```
 1 int64_t choosePivot(int64_t *a, int64_t lo, int64_t hi)
 2 {
 3  return( (lo+hi)/2 );
 4 }
 5
 6 int64_t partitionArray(int64_t *a, int64_t lo, int64_t hi)
 7 {
 8   int64_t pivotIndex, pivotValue, storeIndex;
 9
10   pivotIndex = choosePivot(a, lo, hi);
11   pivotValue = a[pivotIndex];
12
13   (void) swap(&a[hi], &a[pivotIndex]);
14
15   storeIndex = lo;
16
17   for (int64_t i=lo; i <= hi-1; i++)
18   {
19    if ( a[i] < pivotValue ) {
20       (void) swap(&a[i], &a[storeIndex]);
21       storeIndex++;
22    }
23   }
24   (void) swap(&a[storeIndex], &a[hi]);
25
26   return(storeIndex);
27 }
```

Figure 3.13: **Implementation of the partitioning phase** – This code implements the partitioning phase in the quicksort algorithm. In this case, the pivot element is defined to be $floor(lo + hi)/2$, the value in the center, or one element less, of the array. The value returned by the partitioning function is an index. On return, array elements with an index less than the return value are smaller than the pivot value. The array value at this index is the pivot value.

Variable `storeIndex` is initialized at line 15. It is used to point to the next available index in the left part of the array. This is where the values less than the pivot value are stored.

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	`storeIndex`
Initial values	8	6	3	4	4	5	3	9	2	2	
Before initial swap	8	6	3	4	**4**	5	3	9	2	**2**	
After initial swap	8	6	3	4	2	5	3	9	2	4	0
Source line 19 (i = 2)	**8**	6	**3**	4	2	5	3	9	2	4	0
Swap and update (i = 2)	3	6	8	4	2	5	3	9	2	4	1
Source line 19 (i = 4)	3	**6**	8	4	**2**	5	3	9	2	4	1
Swap and update (i = 4)	3	2	8	4	6	5	3	9	2	4	2
Source line 19 (i = 6)	3	2	**8**	4	6	5	**3**	9	2	4	2
Swap and update (i = 6)	3	2	3	4	6	5	8	9	2	4	3
Source line 19 (i = 8)	3	2	3	**4**	6	5	8	9	**2**	4	3
Swap and update (i = 8)	3	2	3	2	6	5	8	9	4	4	4
Before final swap	3	2	3	2	**6**	5	8	9	4	**4**	4
After final swap	3	2	3	2	4	5	8	9	4	6	4

Figure 3.14: **An example of the partitioning phase** – This shows how the array elements are moved around during the partitioning phase. The top row has the array index values. In this example, `lo = 0` and `hi = 9`. The pivot element is `4` and the pivot index is `4` as well. Array elements marked in bold are to be swapped next. In the last column the value of variable `storeIndex` is shown. The last row contains the resulting array. The elements that are moved to the left of the pivot are underlined.

The for-loop spanning lines $17 - 23$ moves all values less than the pivot value, if there are any, to the left. They are stored in a linear fashion, starting at `a[lo]`. After this loop has completed, $a[i] < pivotValue$ for $i = lo, \ldots, storeIndex - 1$. These values are not necessarily sorted yet. At line 24, the pivot value is put back and placed at the end of this sequence. Variable `storeIndex` separates the array values: $a[i] \leq pivotValue$ for $i = lo,\ldots,storeIndex$. Array elements for an index value exceeding `storeIndex` are equal or larger.

Figure 3.14 shows an example for an array consisting of 10 elements. As the partitioning algorithm proceeds, elements are swapped until those strictly less than the pivot value are to the left of the pivot.

Step by step, array elements are moved to their new position. Upon completion of this function, the elements are partially sorted. The index value returned partitions

```
 1 void seqQuicksort(int64_t *a, int64_t lo, int64_t hi)
 2 {
 3   if ( lo < hi ) {
 4     int64_t p = partitionArray(a, lo, hi);
 5
 6     (void) seqQuicksort(a, lo, p - 1); // Left branch
 7
 8     (void) seqQuicksort(a, p + 1, hi); // Right branch
 9   }
10 }
```

Figure 3.15: **The sequential quicksort implementation** – This is the core part of the algorithm. After the partitioning phase, array element `a[p]` is in the correct place. Elements to the left and right of it are each partitioned in a recursive manner.

the array. The corresponding array element is in its final position. All elements to the left are strictly less than the pivot value. The array elements with a higher index value are equal, or larger.

The next step is to repeat this process for the elements to the left and right of the index value returned. These two operations are independent and can be carried out simultaneously. This process is then applied repeatedly until all elements have been sorted. The key part of the sequential quicksort source code implementing this divide and conquer algorithm is listed in Figure 3.15.

The code is executed as follows. If `lo` is equal to, or exceeds `hi`, the function immediately returns, terminating the current execution path. Otherwise, at line 4, array `a` is partitioned within the range defined by parameters `lo` and `hi`. After this call, array element `a[p]` is in its final position and the array sections to the left (line 6) and to the right (line 8) are partitioned.

This pattern repeats recursively. Because the range decreases, at some point, the if-statement at line 3 evaluates to `false` and the execution terminates. Step by step, the function calls are skipped and when the program terminates, the array values have been sorted.

3.3.2 The OpenMP Quicksort Algorithm

As a first step towards a parallel implementation using tasking, the framework is set up. The still-to-be-written parallel function is called from here. To underline

```
1 #pragma omp parallel default(none) shared(a,nelements)
2 {
3   #pragma omp single nowait
4   {
5     (void) ompQuicksort(a, 0, nelements-1);
6   } // End of single section
7 } // End of parallel region
```

Figure 3.16: **The driver part for the OpenMP quicksort function** – As seen earlier, the function call is embedded in a parallel region. Since function `ompQuicksort` uses tasks, the `single` directive ensures only one thread generates the tasks. To avoid a redundant barrier, the `nowait` clause is used.

this is the OpenMP version, the function is called `ompQuicksort`. The relevant part of the calling program is listed in Figure 3.16. This driver part is straightforward. To ensure only one thread generates the tasks, a `single` directive is used to enforce this. The `nowait` clause at line 3 removes a redundant barrier.

Next, tasks need to be used to parallelize the sequential code shown in Figure 3.15. The main decision to be made is to identify which part(s) of the code should be defined as tasks. In this case, these are the calls at lines 6 and 8 of the sequential code. The modified source code using tasks is shown in Figure 3.17.

When comparing the two listings, it is easy to see that, other than changing the function name to distinguish the sequential and parallel versions, and the addition of two directives, the code has not changed.

As always, one must think about the scoping of the variables and it is good practice to specify this explicitly. Although it is the default, the variables that control the section of the array to be sorted are defined to be `firstprivate`.

The same could be done for the pointer to array `a`, but it is more natural to make it shared. This reflects that all tasks are working on the same block of memory. It is our responsibility to avoid data races. In this case, this is not an issue, because there is never a duplicate index value.

The first time function `ompQuicksort` is called, two tasks are created. They are working on two disjoint parts of the array to the left and right of the pivot element, which is in its final position.

The very first time the function is called, the entire array is scanned and partitioned. This is a sequential phase in the parallel algorithm, but after this initial

```
 1 void ompQuicksort(int64_t *a, int64_t lo, int64_t hi)
 2 {
 3   if ( lo < hi ) {
 4     int64_t p = partitionArray(a, lo, hi);
 5
 6     #pragma omp task default(none) shared(a) firstprivate(lo,p)
 7       { (void) ompQuicksort(a, lo, p - 1); } // Left branch
 8
 9     #pragma omp task default(none) shared(a) firstprivate(hi,p)
10       { (void) ompQuicksort(a, p + 1, hi); } // Right branch
11   }
12 }
```

Figure 3.17: **The OpenMP quicksort implementation** – The two (recursive) calls to `ompQuicksort` are defined to be tasks. The variables defining the array sections to be sorted are declared `firstprivate`. Although not the only choice, making array `a` shared is most straightforward.

split, there are two disjoint branches executing in parallel. Each branch subsequently splits itself into two more parallel function calls, generating two tasks. This splitting continues until there is only one element left in a particular branch. No more tasks are generated in this branch, ending the sorting process for this part of the array.

Unlike the example in Section 3.1.1, there is no `taskwait` construct needed, because each task executes independently. There is no need to wait for other tasks.

3.3.3 Fine-Tuning the OpenMP Quicksort Algorithm

The implementation shown in the previous section works fine. The code executes in parallel and the array is correctly sorted. There is an issue though. For a sufficiently large array, many tasks are generated. This in itself may already impact performance due to resource constraints, but in case of a divide-and-conquer method, the amount of work performed per task decreases, as the algorithm progresses. The cost of task management does not decrease, however and eventually this overhead dominates.

These performance issues are not restricted to this example. In many algorithms that use tasks, these need to be considered, especially if the amount of work

```
 1 void ompQuicksort(int64_t *a, int64_t lo, int64_t hi)
 2 {
 3   if ( lo < hi ) {
 4
 5    if ( (hi - lo + 1) < cutoff_seq ) {
 6       seqQuicksort(a, lo, hi); // Call sequential version
 7    } else {
 8       int64_t p = partitionArray(a, lo, hi);
 9
10       #pragma omp task default(none) shared(a) firstprivate(lo,p)
11         { (void) ompQuicksort(a, lo, p - 1); } // Left branch
12
13       #pragma omp task default(none) shared(a) firstprivate(hi,p)
14         { (void) ompQuicksort(a, p + 1, hi); } // Right branch
15    }
16   }
17 }
```

Figure 3.18: **The OpenMP quicksort implementation with escape hatch** – The code at lines $5-6$ ensures that a different function is executed in case the array length is below a certain threshold. The obvious choice is to call the sequential version of the algorithm then.

performed decreases over time. Let's first look at limiting the number of tasks generated.

In Figure 3.18, a solution is shown, where an escape hatch is introduced. In case the array length drops below a user-defined threshold `cutoff_seq`, the sequential version of the algorithm is executed.

This approach is simple and effective, but it has two drawbacks. One must write a sequential version of the algorithm. Admittedly, this is very similar to the OpenMP version, but there is also the issue of having to maintain two (nearly identical) sources.

Another problem to deal with is that one must choose the appropriate value for the threshold variable `cutoff_seq`. This is not trivial, because the optimal value depends on certain system and OpenMP implementation-dependent characteristics. The problem size may also influence the value for the cutoff. It is therefore highly recommended to conduct some experiments to find the optimal value. As a

guideline, the cutoff value can be set, such that the OpenMP version using a single thread performs the same as, or similar to, the sequential version.[6]

The above approach was the most common solution chosen at the time tasking was introduced in OpenMP 3.0, but in OpenMP 3.1 two more clauses were added to make it easier to fine-tune the performance of tasking.

The `final(`*`expr`*`)` clause can be used to avoid the runtime system continuing to generate tasks. It targets recursive algorithms and nested tasks, where the number of tasks tends to increase exponentially. This cannot only clog the runtime system, but especially if the amount of work decreases over time, the tasking overhead may offset any performance gains. The `final` clause takes an expression `expr`. If it evaluates to `true`, no more tasks are generated and the code is executed immediately. This is propagated to all of the child tasks.

This clause can be combined with the `mergeable` clause to avoid a separate data environment being created. This topic is covered in more detail in Section 3.7.

Effectively, the combination of these clauses, together with a suitable cutoff value, eliminates the need to have a separate sequential version. This is demonstrated in Figure 3.19.

The if-statement and call to the sequential version of the algorithm have been removed. Instead, the two clauses have been added on lines 7 and 11. They achieve the same functionality, but eliminate the need to have two separate source trees to maintain. As before, some experimentation is needed to determine the optimal value for the threshold variable `cutoff_tasks`.

There is one more thing that may be done to make this algorithm perform more efficiently. There is actually no need to have two tasks. The source for this approach is shown in Figure 3.20. The code is obviously nearly identical to the previous version. Only the tasking pragma and curly braces around the second call to execute the right branch have been removed. In this way, the tasking overhead is reduced, while there is still parallelism to be exploited.

As trivial as this change may seem, the order matters. With the approach shown, each invocation of the function generates a task. All these tasks are put on the runtime queue and executed by the threads. Meanwhile the sequential calls to the right branch are executed as well.

With tasking pragma on the right branch, there is a sequential path first. Only when this has finished, are the tasks generated and executed.

[6] Using bisection, this value can often be found relatively quickly.

```
 1 int64_t omp_quicksort(int64_t *a, int64_t lo, int64_t hi)
 2 {
 3  if ( lo < hi ) {
 4
 5    int64_t p = partition(a, lo, hi);
 6
 7    #pragma omp task final( (p - lo) < cutoff_tasks) mergeable \
 8                     default(none) shared(a) firstprivate(lo,p)
 9    { (void) omp_quicksort(a, lo, p - 1); } // Left branch
10
11    #pragma omp task final( (hi - p) < cutoff_tasks) mergeable \
12                     default(none) shared(a) firstprivate(hi,p)
13    { (void) omp_quicksort(a, p + 1, hi); } // Right branch
14
15    return(p);
16  }
17 }
```

Figure 3.19: **The OpenMP quicksort implementation using the final and mergeable clauses** – The test and call to the sequential version have been removed. Instead the `final` and `mergeable` clauses are used to achieve the same result. Just as with the previous version, finding the correct value for the threshold variable `cutoff_tasks` requires some experimentation.

3.3.4 Closing Comments on the OpenMP Quicksort Algorithm

Although this algorithm has been covered in quite some detail, there are some additional comments to be made.

On the algorithmic side, the most straightforward implementation has been selected, because it is already sufficiently complex to demonstrate the use of tasking to parallelize a divide-and-conquer type of algorithm. Without a doubt, there are more efficient sequential versions out there.

For one thing, the choice of the pivot is quite crucial. In our code, the middle element is selected, because that is good for load balancing. It is however not necessarily the optimal choice to get the best sequential performance.[7]

[7]This could be something to consider for the sequential version in Figure 3.18, if available.

```
 1 int64_t omp_quicksort(int64_t *a, int64_t lo, int64_t hi)
 2 {
 3  if ( lo < hi ) {
 4
 5    int64_t p = partition(a, lo, hi);
 6
 7    #pragma omp task final( (p - lo) < cutoff_tasks) mergeable \
 8                      default(none) shared(a) firstprivate(lo,p)
 9    { (void) omp_quicksort(a, lo, p - 1); } // Left branch
10
11    (void) omp_quicksort(a, p + 1, hi);     // Right branch
12
13    return(p);
14  }
15 }
```

Figure 3.20: **The OpenMP quicksort implementation with one task per call** – There is only one task per call now. Each time this function is called, a task is generated and put in the queue for the threads to work on. Note that issuing the task first is more efficient than the other way round.

Last, but not least, when trying this, be prepared to conduct several experiments. Not only should this be done to determine the threshold(s), but also to decide on the best thread affinity strategy, which is covered in Chapter 4.

3.4 Overlapping I/O and Computations Using Tasks

In this section, the code shown in Figure 1.11 on page 21 is revisited. The example shows the use of parallel sections to overlap I/O and computations. Under the assumption that the I/O can be processed in chunks, a pipeline is set up, such that the computational part waits for a read operation to be completed. Once completed, the data is processed and passed on to a post-processing function. This pipeline approach is demonstrated in Figure 3.21.

The pipeline works as follows. At the start, the first chunk of data is read. All subsequent activities need to wait for this operation to be completed. Once finished, the processing can start. Meanwhile, the next chunk of data can be read, resulting in potentially two activities executing concurrently. After the computation on the

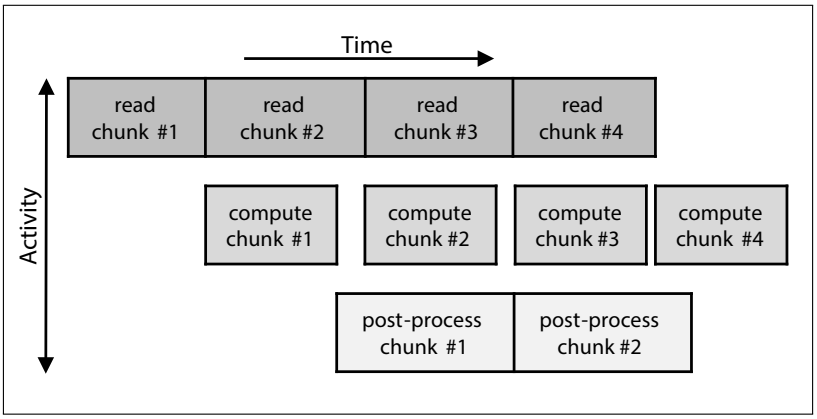

Figure 3.21: **A pipeline to overlap I/O and computations** – There are three activities: read a chunk of data, perform computations on this data and post-process the results. After a start-up phase, these activities can be executed simultaneously, but on different chunks of data.

first chunk of data has finished, the post-processing phase can start, while at the same time, the third chunk of data is read and processing of the second chunk of data starts soon after. At this point, there are three overlapping activities. This continues until the last chunk of data is read and the pipeline starts to wind down.

The code in Figure 1.11 demonstrates how these activities are parallelized using three parallel sections. Through various shared status arrays, the three phases are synchronized.

This approach works, but there is no good way to handle a load-imbalance. Another issue is that the communication of these phases through the status arrays is cumbersome. A polling mechanism is needed to check if the status of a phase has changed, and the OpenMP flush construct is required to ensure changes are made visible to the threads executing the other phases. Although less of a concern, the algorithm also requires either one, or otherwise at least three threads to work correctly.

3.4.1 Using Tasks and Task Dependences

As shown next, tasks provide a much more natural vehicle to implement such a pipeline. The code is listed in Figure 3.22.

```
 1 #pragma omp parallel default(none) \
 2                   shared(fp_read, n_io_chunks, n_work_chunks) \
 3                   shared(a, b, c) \
 4                   shared(status_read, status_processing) \
 5                   shared(status_postprocessing)
 6 {
 7   #pragma omp single nowait
 8   {
 9     for (int64_t i=0; i<n_io_chunks; i++) {
10
11       #pragma omp task depend(out: status_read[i]) \
12                        priority(20)
13       {
14          (void) read_input(fp_read, i, a, b, &status_read[i]);
15       } // End of task reading in a chunk of data
16
17       #pragma omp task depend(in: status_read[i]) \
18                        depend(out: status_processing[i]) \
19                        priority(10)
20       {
21          (void) compute_results(i, n_work_chunks, a, b, c,
22                                 &status_processing[i]);
23       } // End of task performing the computations
24
25       #pragma omp task depend(in: status_processing[i]) \
26                        priority(5)
27       {
28          (void) postprocess_results(i, n_work_chunks, c,
29                                     &status_postprocessing[i]);
30       } // End of task postprocessing the results
31
32     } // End of for-loop
33   } // End of single region
34 } // End of parallel region
```

Figure 3.22: **Overlapping I/O and computations using tasks** – Each phase has been turned into a task. By using task dependences, the correct order of execution is guaranteed. In addition to this, priorities are used to give hints to the runtime system.

As we've seen several times before, the entire parallel region consists of a gigantic single region, but that does not mean this code executes sequentially. On the contrary, this is how parallelism is created. To understand why that is, let's go through the core part of this code fragment line by line.

The for-loop spans lines $9 - 32$. The assumption is that `n_io_chunks` of data can be read independently. Since the loop is within a single region, one thread executes all the loop iterations. This thread encounters the tasks, generates the corresponding code, and puts them in the queue.

The queued tasks are made available for execution in, what is called a *task synchronization construct*, which in this case is the barrier implied at the end of the parallel region at line 34. In a task synchronization construct, any thread that is available, may be put to work. More on this can be found in Section 3.7, starting on page 141.

The first task spans lines 11 through 15. This is the task reading the chunk of data corresponding to iteration `i` of the loop. The data is read from the file associated with file pointer `fp_read` and stored in array elements `a[i]` and `b[i]`. Upon successful completion, a status value is written into array element `status_read[i]`. This value is meant to be checked later on to ensure correct execution.

The new element here is the use of the `depend` clause at line 11. It is used to define *task dependences*.[8] There is also another new clause, the `priority` clause at line 12. We'll get to that soon, but first look at task dependences in more detail.

The `depend` clause takes a dependence type, which can be `in`, `out`, or `inout`, with a variable. The variable is used to set up a dependence between two tasks, while the type defines the direction of the dependence.

In this case, the second task, `compute_results`, needs to be made dependent upon the first task, `read_input`. This is controlled through variable `status_read[i]`. By using the `in` type for the second task, and type `out` on the first task, it is guaranteed that the second task does not start before the first task. In a similar manner, a dependence between the second and third task is set up. Variable `status_processing[i]` is used to define the dependence. By using type `out` on the second task, and type `in` on the third task, the third task starts only upon completion of the second task. The loop variable `i` is included in the dependence variables to allow the runtime system to schedule multiple similar tasks at the same time.

[8] This is another use of the `depend` clause. In Section 2.4.4, it is shown how this clause is used to parallelize a `doacross` loop.

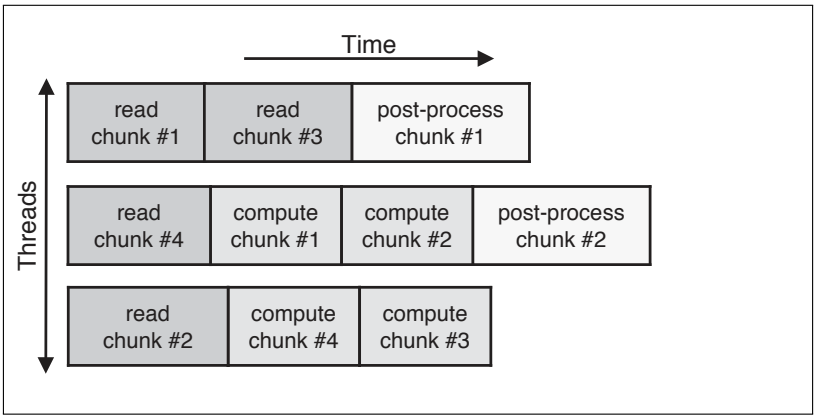

Figure 3.23: **The dynamic behavior of the pipeline** – It is assumed that three threads are used. As long as the dependences are obeyed, the loop iterations can be executed in an arbitrary order, as demonstrated here.

It is however worth noting that these three status arrays are not strictly necessary. Here they are used to verify correct execution afterwards, but as far as the dependences go, simply using the loop variable `i` instead is sufficient.[9]

The a-synchronous execution of the pipeline is illustrated in Figure 3.23. In the diagram, three threads are used, but this is only for the sake of the example. This code runs fine using any number of threads. The key thing is that for any given iteration, the three different phases must be executed in the correct order. This is achieved using the dependences. Aside from this, there are no restrictions on the order of execution and due to the more flexible scheduling, load balancing is less of an issue.

As mentioned above, in this code, *task priorities* are used as well. These are specified through the `priority` clause. This clause takes a non-negative scalar integer expression, specifying the priority. For example, at line 12, the priority of this task is set to 20. The second task has a priority of 10, and the third task has a priority of 5.

These priorities are hints to the runtime system to assist the scheduler to determine which tasks to choose to execute. The priority values are relative numbers only. In this example we want to emphasize that reading the data is most impor-

[9]Error handling can also be handled through cancellation, covered in Section 2.4.2.

tant (because everything else depends on it) and the post-processing part is least important. The priorities are suggestions only and correct execution of the program may not depend on them. In case there is a certain execution order, dependences must be used to enforce this.

Prior to program start up, environment variable `OMP_MAX_TASK_PRIORITY` must be set to the maximum value used. By default this variable is set to zero, that is, there are no higher priority tasks. Runtime function `omp_get_max_task_priority()` can be used to return the maximum value set through this environment variable.[10]

The solution shown in Figure 3.22 works fine. A major advantage over the original approach using parallel sections is that the explicit flush construct is no longer needed. Load balancing is also less of a concern and this code executes correctly using any number of threads.

The one possible drawback is that there are three tasks for each chunk of data read, and task scheduling overhead could become an issue. Especially if each task does not perform a sufficient amount of work. Features like the `final` and `if` clauses may be used to control this, but there is an alternative solution worth considering and discussed in the next section.

3.4.2 Using the Taskloop Construct

To reduce the tasking overhead, a solution may be to place all three function calls within a single task and apply similar constructs as before. There is, however, a more elegant and powerful construct, available since OpenMP 4.5. It is called the *taskloop construct* and is meant to simplify using tasks in a loop context. It is definitely much more than syntactic sugar and relieves the user from relatively low level coding details when using tasks.

The `taskloop` construct applies to a (nested) loop. The loop iterations are partitioned into tasks and executed as tasks by the runtime system. Many clauses from the tasking construct are inherited, but there are two new clauses worth mentioning.

Figure 3.24 shows the relevant code fragment using the taskloop construct. With one exception, the first few lines are the same as in the previous example. The status arrays, used to express the dependences, are gone. As mentioned earlier,

[10] There is no equivalent to set the priorities at runtime. This was considered to be too complex to implement.

```
 1 #pragma omp parallel default(none) \
 2                    shared(fp_read, n_io_chunks, n_work_chunks) \
 3                    shared(a, b, c)
 4 {
 5   #pragma omp single nowait
 6   {
 7     #pragma omp taskloop num_tasks(n_io_chunks/10) grainsize(50)
 8     for (int64_t i=0; i<n_io_chunks; i++) {
 9           (void) read_input(fp_read, i, a, b);
10           (void) compute_results(i, n_work_chunks, a, b, c);
11           (void) postprocess_results(i, n_work_chunks, c);
12     } // End of for-loop
13   } // End of single region
14 } // End of parallel region
```

Figure 3.24: **Overlapping I/O and computations using the taskloop construct** – Each triplet, with the functions called in sequence, is a task. This construct supports many clauses. To reduce the overhead and improve efficiency, the `num_tasks` and `grainsize` clauses can be used to fine-tune the performance. Note the absence of the dependence arrays.

this may be handled in a more simple way, but in any case, a variable to specify the dependences is no longer needed.

The new element is the use of the `taskloop` construct at line 7. Without the additional clauses, this creates a task for each triplet of the function calls inside the loop body. As illustrated in Figure 3.25, this ensures each sequence with the three calls occurs in the right order.

With the new approach, larger units of work are scheduled, and the overhead is reduced. As is typical with tasking, there is the risk that too many tasks are generated (`n_io_chunks` in this case) and the overhead of managing these tasks could slow down the execution. This is why the `num_tasks` and `grainsize` clauses are so convenient.

The `num_tasks` clause takes an integer expression. The runtime system generates as many tasks as this expression evaluates to, or fewer if there are less loop iterations. The `grainsize` clause takes an integer expression, the *grain-size*. The number of iterations assigned to a task is the minimum of *grain-size* and the number of loop iterations, but does not exceed twice the value of *grain-size*. These two

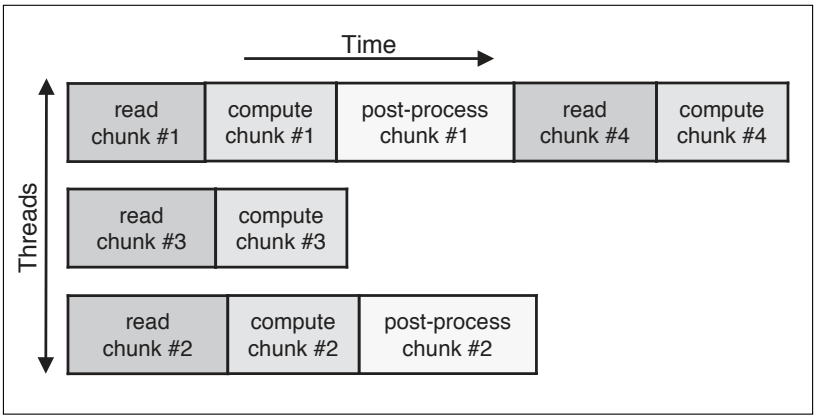

Figure 3.25: **The dynamic behavior of the pipeline using the taskloop construct** – Three threads are used. The loop iterations can be executed in any order, but for each iteration value, the triplet of functions is executed in order.

clauses provide powerful ways to fine-tune the efficiency when using the `taskloop` construct. They are discussed in more detail in Section 3.8, starting on page 146.

Their usage is shown at line 7. The number of tasks is restricted to be 1/10 of the maximum number of `n_io_chunks`. This number of 10% is not based upon actual measurements, or a recommendation. We merely want to demonstrate that this value may be an expression that depends on the runtime value(s) of other variables. The grain-size is set to 50, another arbitrary choice. Possibly, one would like to make this value dependent upon the value of certain program variables and that is supported too.

The effect of the `grainsize` clause is to increase the amount of work performed per task. In general this is a good thing to do, but if the workload is not equally balanced across the loop iterations, a too high value for the `grainsize`, may result in a load-balancing issue. That is ironic, because one of the strengths of tasking is that load-balancing tends to be less of an issue.

In any case, it is strongly recommend to conduct performance experiments to find the optimal values for these parameters. Don't be surprised if they turn out to be dependent upon several factors, and some differentiation depending upon the details of the computer system and OpenMP runtime implementation, may be needed.

3.4.3 Closing Comments on the Pipeline Example

Both versions that implement a pipeline to overlap I/O and computations, use tasks, but the behavior is different.

The first version has the maximum flexibility to schedule the tasks and is most robust in handling load-balancing issues. A downside is that it is easy to generate too many tasks and fine-tuning through the `final` and `if` clauses may be required. The second version uses the powerful `taskloop` construct. This relieves the user of several details to worry about, but load-balancing may be more of an issue. Through the use of the `num_tasks` and `grainsize` clauses, the overhead of tasking can be kept low.

Given the difference in the pros and cons it is difficult to give a recommendation. If the user has more insight into the application characteristics, this may help to select the optimal version.

3.5 The Data Environment with Tasks

So far, we have glossed over the data environment, or *scoping*, with tasking. It is important enough to elaborate upon, though.

Although some of the default scoping rules are intuitively clear, in general it is *strongly* recommended to use the `default(none)` clause and explicitly specify the data sharing attributes of the variable(s). Having said that, let's dig a little deeper into the default rules.

Some rules that apply to the parallel region carry over. For example, global and static variables are automatically shared. Local variables are private by default, by their very nature, as defined in C/C++ within the scope of curly braces (and unlike Fortran). If shared scoping is not inherited, orphaned task variables are `firstprivate` by default, while non-orphaned variables inherit the shared attribute. Variables are `firstprivate`, unless they are shared in the enclosing context.

This is illustrated in the example shown in Figure 3.26, where the scoping and values of the variables are as follows:

- Variable `a` is a global variable and shared. Because the initial value is not changed, it has a value of 1 in all tasks.
- Variable `b` is more challenging. It starts as a shared variable, but is made `private` at line 10. This makes the value undefined within the inner parallel

```
 1 int a = 1;
 2
 3 int main(int argc, char **argv)
 4 {
 5   int b = 2;
 6   int c = 3;
 7
 8   #pragma omp parallel shared(b)
 9   {
10     #pragma omp parallel private(b)
11     {
12       int d = 4;
13       #pragma omp task
14       {
15          int e = 5;
16
17          // Scope of a: shared,        value of a: 1
18          // Scope of b: firstprivate,  value of b: undefined
19          // Scope of c: shared,        value of c: 3
20          // Scope of d: firstprivate,  value of d: 4
21          // Scope of e: private,       value of e: 5
22
23       } // End of task
24     } // End of inner parallel region
25   } // End of outer parallel region
26 }
```

Figure 3.26: **An example of the default scoping rules for tasking** – This code demonstrates the automatic scoping for several variables. Although the rules are fairly straightforward, it is still recommended to explicitly scope variables.

region spanning lines $10 - 24$. Although by default it is made `firstprivate` within the task, the damage has been done and the value is undefined.

To make the value of `b` defined within the task, the solution is to scope it as `firstprivate(b)` at line 10.

- By virtue of the default scoping rules for parallel regions, variable `c` has the `shared` attribute within the outer and inner parallel regions. This is inherited

within the task and the value is 3.

- Variable `d` is a `private` variable inside the inner parallel region and therefore scoped as `firstprivate` within the task. Since the value is set to 4 at line 12, this is also the value inside the task.

- Variable `e` is declared within the task and therefore a local variable. This makes it a `private` variable with a value of 5.

The above example assumes a fairly deep understanding of the default scoping rules. A possible pitfall is that misinterpretation of a rule can lead to unexpected behavior and in general one should not count on a compiler to detect this.

Variable `b` is a good example how risky this is. By omitting the scoping on the task definition, an undefined value is used. In this simple case, a compiler might warn against this, but in a more complex code structure, this can easily be much more difficult, or even impossible, to detect.[11]

In summary, one can rely on the default rules, but using the `default(none)` clause can save a significant debugging effort at the expense of some more work in the development phase.

3.6 What is a Task?

Many aspects of tasking were discussed already and several examples of how to use tasks have been shown already. Hopefully these are sufficient to get started, but because several related topics were either skipped, or covered very briefly, a somewhat more formal description of the main constructs and features is given in this section.

To start with, what is a task exactly and how is it defined in an application? In Figure 3.27, the syntax of the task construct is shown. A task consists of the source code in the associated structured block, the data environment as defined through the implicit and explicit data scoping rules, plus the relevant ICVs. When a thread encounters a task, it packages the code and the data environment, such that the task is ready for execution. How and when the task is then executed, is discussed in more detail in Section 3.7.

[11] We know of at least one compiler that indeed detects this bug.

#pragma omp task *[clause[[,] clause]...]* *structured block*
!$omp task *[clause[[,] clause]...]* *structured block* **!$omp end task**

Figure 3.27: **Syntax of the task construct in C/C++ and Fortran** – This defines the structured block of code to be a task. A task consists of the code to execute, the data environment, plus the relevant ICVs.

private *(list)*	
firstprivate *(list)*	
shared *(list)*	
default(shared | **none)**	(C/C++)
default(shared | **firstprivate** | **private** | **none)**	(Fortran)
if *([* `task` *:] scalar-logical-expression)*	
final *(scalar-logical-expression)*	
mergeable	
depend *(dependence-type : list)*	
priority *(priority-value)*	
untied	

Figure 3.28: **The clauses supported by the task construct** – With the exception of the `untied` clause, the clauses specific to tasking have been introduced in the previous examples, but additional information is given in the text.

Tasks can be dynamically nested, for example in a recursive algorithm, but they may also be lexically nested within other tasks, parallel regions, and worksharing constructs.

The `task` construct supports various clauses. They are listed in Figure 3.28. The data scoping clauses `default`, `private`, `firstprivate` and `shared` are the same as used on other constructs, but there are several clauses specific to tasks. With the exception of the `untied` clause, all of these have been used and discussed in the examples in the previous section. Below they are described in a somewhat more formal way:

- The `if` clause takes the optional `task` keyword. Support for a keyword is a general feature of this clause and may be needed in case composite, or combined, constructs are used.

The scalar expression on this clause must evaluate to `true` or `false`. In the case of the latter, the encountering task is suspended and the new task is executed immediately. The parent task resumes, once the new task has completed.

This feature can be used by an implementation to improve the performance by avoiding queueing tasks that are too small.

The difference with the `final` clause below, is that, with the `if` clause, the child tasks are not affected. This is why it was *not* used in the example shown in Figure 3.19. Due to the nature of the quicksort algorithm, once the length of the array drops below the threshold, the length for child tasks is also below this value.

- The `final` clause is a feature to fine-tune the performance of tasks. The clause takes a scalar expression evaluating to `true` or `false`. If this evaluates to `true`[12], the task is considered to be final and no additional tasks are generated.

  This is like a deeper version of the `if` clause, where nested tasks are not affected. With the `final` clause, all child tasks are final tasks too.

- The `mergeable` clause was introduced to reduce the data requirements. Since each task comes with its own data environment, the program may require a significant amount of memory. The `mergeable` clause informs the compiler the data environments can be merged.

- The `depend` clause in the context of tasking is used to specify dependences between tasks.[13] The clause takes a keyword to describe the type of dependence, and a list of variables it applies to.

  The dependence type can be `in`, `out`, or `inout`. The latter is actually a leftover from the initial support for this feature. It is identical to type `out`.

  The two types `in` and `out` are strongly related. They define the direction of the dependence. Through the `out` type, it is specified that the variable(s) in the list are output variables from that task. Any task with dependency type

[12] Note the difference between the condition `false` and `true` for the `if` and `final` clause to be triggered.

[13] The `depend` clause is used on other constructs too.

`in` and the same variable, depends on this task and will not be executed until the task with the corresponding `out` type has completed.

The list item(s) may include array sections. See also Section 6.3.4 for more information on array sections.

- To indicate which task(s) are relatively important, the `priority` clause can be used. This is a hint to the OpenMP runtime system to help scheduling the execution of the various tasks.

  The clause takes a scalar expression as an argument. This must evaluate to a non-negative integer value and sets the priority for that task.

  The priorities are relative values. A task with a higher priority is considered to be more important than a task with a lower priority.

  Be aware that the value for the priority may not exceed the value set through environment variable `OMP_MAX_TASK_PRIORITY`, which is set to zero by default. The consequence is that this variable *must* be set when using priorities.

  The function `omp_get_max_task_priority()` returns this upper limit. There is no "set" counterpart to change the maximum priority during the execution of the program.

- By default, tasks are `tied`, but the `untied` clause changes this to `untied`. More details on tied and untied tasks can be found in Section 3.7.

The `if`, `final`, `mergeable`, and `priority` clauses are provided to optimize the performance of the tasking mechanism. These clauses are used to reduce the number of tasks generated, task scheduling overhead, memory requirements, and to express runtime scheduling preferences. An implementation is however free to ignore them.

3.7 Task Creation, Synchronization, and Scheduling

By design, task creation, scheduling, and execution details are meant to be a black box. This not only simplifies the life of the user, but also gives the implementation the freedom to adapt to the underlying architecture and evolve over time, without any impact on the portability of the code.

When a thread encounters a `task` construct, a new task is created and queued for execution by any thread in the team. Execution of the task could be immediate, or deferred until later, depending on the task scheduling details and thread availability.

Task synchronization construct	Description
barrier	either an implicit, or explicit barrier.
taskwait	wait on the completion of child tasks of the current task.
taskgroup	wait on the completion of child tasks of the current task, *and* their descendants.

Figure 3.29: **The three task synchronization constructs** – Completion of a task is guaranteed at a task synchronization point, but whether descendant tasks are affected also depends on the construct.

The thing that surprises users most the first time they consider using tasking, is that, although tasks are generated when the `task` construct is encountered, they are not necessarily executed then. More often than not, execution occurs "later," or in tasking terminology, the execution is *deferred*.

To be more precise, tasks are only guaranteed to be completed at program exit and at one of the three constructs listed in Figure 3.29. They are called *task synchronization constructs*.

The `barrier` construct can either be implied, at the end of the parallel region for example, or inserted explicitly. In both cases, the tasks are guaranteed to be completed then. The immediate consequence of this is that all tasks are completed at the end of a parallel region, since it has an implied barrier that may not be omitted.

The (lack of) implied barrier is why there is a difference between using the `single` construct versus the `master` construct. The latter has no implied barrier and tasks are not guaranteed to be completed upon exiting the master region.

There are also cases where the tasks need to completed, before the next barrier is encountered. For example, in the code shown in Figure 3.5 on page 108, the two tasks must be completed before the next (print) statement. This is where the `taskwait` construct comes to the rescue. If completion of the tasks needs to be enforced, this construct can be used. It is a stand-alone directive and ensures all child tasks of the current task are completed.

Another situation where the `taskwait` construct could be needed is when the `nowait` clause has been used. Since there is no longer an implied barrier, this clause has the side effect that there is no guarantee that all tasks have completed at the end of the construct to which the clause applies. Should this still be needed,

#pragma omp taskgroup *new-line* *structured block*
!$omp taskgroup *structured block* **!$omp end taskgroup**

Figure 3.30: **Syntax of the taskgroup construct in C/C++ and Fortran** – This defines the structured block of code to be a taskgroup and provides deep synchronization by ensuring the completion of all child tasks, as well as their descendants.

the `taskwait` construct can be used to enforce completion of all child tasks.

The `taskwait` construct works well, unless there are descendant tasks. Those are not affected and if deeper synchronization of tasks is needed, the `taskgroup` construct should be used. This guarantees that also all descendant tasks are completed. The syntax is given in Figure 3.30.

Now that we have seen how tasks are scheduled and what mechanisms are available to enforce the completion of tasks, it is time to look at *task scheduling*.

Although not required by the specifications, all current tasking implementations use a queueing system. With this, generated tasks are put in a queue and at some point, executed.[14] For the runtime system, managing this queueing system takes time, especially since it is common to have many tasks, and threads need to be assigned to tasks. All of this adds to the parallel overhead.

This is why several features to reduce this overhead are available, such as the `if` and `final` constructs. Both can be used to prevent the queueing system from being flooded with either too small tasks, too many tasks, or both.

To formalize this and support several performance-related features, there are four tasking related concepts in the specifications:

- *A task region* - A region consisting of all code encountered during the execution of a task.
- *An undeferred task* - This is a task for which execution is not delayed, or "deferred," with respect to its generating task region. The generating task region is suspended until execution of the undeferred task has completed.

[14]The details of such queueing systems are complex and beyond the scope of this book.

In case the `if` clause on the `task` construct evaluates to `false`, an *undeferred* task is generated and the encountering thread suspends the current task region. Execution resumes only after the generated task has completed.

- *An included task* - This is an undeferred task, executed immediately by the encountering thread.

  If the `final` clause evaluates to `true`, the generated task is a final task. All tasks encountered during the execution of a final task are also final and included.

- *A tied/untied task* - Upon resuming a suspended task region, a tied task *must* be executed by the same thread again. With an untied task, there is no such restriction and any thread in the team can resume execution of the suspended task.

The above means that, while the `if` clause is "shallow" and only affects the encountering task, the `final` clause is a "deep" feature. Once a task is final, all child tasks are final too.

When using tasks, *task scheduling points* are places in the application where a thread executing a task, may temporarily suspend the current task, and switch to execute a different task. It can either begin, or resume this other task. A task scheduling point is included at the following locations:

- The point immediately following the generation of an explicit task.
- After the point of completion of a `task` region.
- In a `taskyield` region.
- In a `taskwait` region.
- At the end of a `taskgroup` region.
- In an implicit and explicit `barrier` region.
- When using the support for accelerators:
  - The point immediately following the generation of a `target` region.
  - At the beginning and end of a `target data` region.

#pragma omp taskyield *new-line*
!$omp taskyield

Figure 3.31: **Syntax of the taskyield construct in C/C++ and Fortran** – This defines an explicit task scheduling point in the application.

- In a `target update` region.
- In a `target enter data` region.
- In a `target exit data` region.
- In the `omp_target_memcpy()` runtime function.
- In the `omp_target_memcpy_rect()` runtime function.

With the exception of the `taskyield` construct, all of these scheduling points are implied. They are there, but not visible. This is actually the reason the stand-alone `taskyield` construct was added. With this construct, the user can add an explicit task scheduling point, allowing the thread to suspend the current task and switch to another task. The syntax is given in Figure 3.31.

One of the clauses listed in Figure 3.28 on page 139 is the `untied` clause. By default, a task is "tied," which means that the *same* thread that executed the task must resume execution after the task has been suspended in a task scheduling point.[15] This could restrict the runtime scheduler and negatively impact performance.

Through the `untied` clause, this limitation is lifted and any thread may resume the execution of the suspended task. Clearly there are performance advantages to allow this kind of scheduling freedom, but there are some important caveats too.

First of all, `threadprivate` variables may not be used. Secondly, explicitly using the thread ID, for example through function `omp_get_thread_num()`, is not allowed and third, one must be careful using locks, including critical regions.

Last, but not least, untied tasks are an optional feature. An implementation is free to ignore the clause and make all tasks tied.

[15] This thread need not be the same thread that created the task.

#pragma omp taskloop *[clause[[,] clause]...]* *new-line* *for-loops*
!$omp taskloop *[clause[[,] clause]...]* *do-loops* **[!$omp end taskloop]**

Figure 3.32: **Syntax of the taskloop construct in C/C++ and Fortran** – The loop iterations are executed in parallel using tasks. At runtime, the iterations are distributed over the tasks.

private *(list)*	
firstprivate *(list)*	
lastprivate *(list)*	
shared *(list)*	
default(shared | **none)**	(C/C++)
default(shared | **firstprivate** | **private** | **none)**	(Fortran)
if *([* `taskloop` *:] scalar-logical-expression)*	
grainsize *(grain-size)*	
num_tasks *(num-tasks)*	
collapse *(n)*	
final *(scalar-logical-expression)*	
priority *(priority-value)*	
untied	
mergeable	
nogroup	

Figure 3.33: **The clauses supported by the taskloop construct** – The `nogroup`, `grainsize`, and `num_tasks` clauses are specific to this construct. All other clauses are supported on at least one other construct.

3.8 The Taskloop Construct

The `taskloop` construct has been added in OpenMP 4.5 and combines the ease of use of the parallel loop with the flexibility of tasking. In the example in Section 3.4, it was used to parallelize the loop in this code.

This construct simplifies using tasks in a (perfectly nested) loop. The syntax of the `taskloop` construct is given in Figure 3.32. The compiler and runtime system create and manage the tasks, while the user does not need to worry about the tasking details.

The `taskloop` construct supports the clauses listed in Figure 3.33. The data-sharing clauses are similar as on other constructs, and as with other constructs, the `if` clause supports the optional `taskloop` keyword. The `collapse`, `final`, `priority`, `untied`, and `mergeable` clauses are the same as on the `task` construct. There are, however, also three clauses that are unique to this construct:

- The `grainsize` clause takes a positive integer expression, the *grain-size*. The number of loop iterations assigned to a task is the minimum of this *grain-size* and the number of loop iterations, but does not exceed twice the value of *grain-size*.

  This clause can be used to adjust the granularity of the work performed by the tasks. It provides an easy way to avoid that the work performed per task is too small

  In absence of this clause, an implementation-dependent default is used.

- The `num_tasks` clause takes a positive integer expression. At runtime, as many tasks as this expression evaluates to, are generated, or fewer, if there are less loop iterations. This clause can be used to limit the number of tasks generated.

  In absence of this clause, an implementation-dependent default is used.

- With the `nogroup` clause, the `taskloop` construct is not embedded in an implied `taskgroup` construct.

The `nogroup` clause is needed in the following scenario. Since by default, the `taskloop` construct is embedded in an implicit `taskgroup` construct, there is a task synchronization point at the end. In case other tasks need to execute concurrently with the tasks executing the `taskloop` construct, this creates a problem. The `taskloop` loop could be executed first, followed by the other task(s). To make matters worse, task priorities are not handled properly either.

With the `nogroup` clause, there is no implied `taskgroup` construct and all tasks can run simultaneously, plus the priorities will be honored, if the implementation supports it.

```
 1 #pragma omp parallel firstprivate(n)
 2 {
 3   #pragma omp single
 4   {
 5
 6     #pragma omp taskloop firstprivate(n) nogroup priority(10)
 7     for (int i=0; i<n; i++)
 8     {
 9        <body of loop>
10     } // End of taskloop
11
12     #pragma omp task priority(50)
13     {
14        <body of task>
15     } // End of task
16
17   } // End of single region
18 } // End of parallel region
```

Figure 3.34: **An example of the nogroup clause on the taskloop construct** – This code demonstrates the use of the `nogroup` clause. Thanks to this clause, the task below the `taskloop` construct is executed simultaneously and if task priorities are implemented, even starts first.

The use of the `nogroup` clause is illustrated in Figure 3.34. Assuming task priorities are supported, and environment variable `OMP_MAX_TASK_PRIORITY` has been set to at least 50, the task defined at lines $12 - 15$ is executed first. Without the `nogroup` clause on the `taskloop` construct at line 6, this is not be the case and the isolated task at line 12 is executed last.

There is a second variant of the `taskloop` construct, called `taskloop simd`. The syntax of this construct is given in Figure 3.35. SIMD is explained in great detail in Chapter 5. On those processors supporting SIMD instructions, this construct allows the compiler to exploit them within the generated tasks. This construct supports both the clauses from the `taskloop` construct, as well as the clauses for the `simd` directives. The `collapse` clause is applied only once.

#pragma omp taskloop simd *[clause[[,] clause]...]* *new-line* *for-loops*
!$omp taskloop simd *[clause[[,] clause]...]* *do-loops* **[!$omp end taskloop simd]**

Figure 3.35: **Syntax of the taskloop simd construct in C/C++ and Fortran** – The loop iterations are executed in parallel using tasks. At runtime, the loop iterations are distributed over the tasks and the resulting loop uses SIMD instructions.

3.9 Concluding Remarks

This chapter has covered all aspects of tasking. Hopefully the examples, plus the additional information presented here, are sufficient to get started.

Tasks work through a queueing system, invisible to the user. A thread that encounters a tasks, generates the code to execute it, including the data environment plus ICVs, and puts it in a queue.

At task synchronization points, threads execute tasks that are waiting in the queue. These points are implied on several constructs, but through the `taskwait` and `taskgroup` constructs, the user can enforce completion of the tasks.

The runtime system is in charge of managing the queue(s), but the user has indirect, yet powerful, controls. The `if`, `final`, `mergeable`, and `priority` clauses allow the user to reduce the pressure on the queueing system, avoid tasks get too small, and help the runtime scheduler to decide which task(s) to execute. Especially if the performance is disappointing, these clauses may make quite a difference and some experimentation is highly recommended.

Task dependences provide a very powerful mechanism to create a chain of tasks, where execution happens in the right order, while leveraging the parallelism in the algorithm.

The taskloop construct makes it easier to leverage tasks in a loop and more efficiently handle a load-imbalance.

An often asked question is whether tasks replace other constructs. The answer is no. Tasks are extremely flexible and powerful, but sometimes other constructs are easier to use and get the job done as well.

5 SIMD – Single Instruction Multiple Data

According to Flynn's taxonomy [9], a Single Instruction Multiple Data (SIMD) processor exhibits data parallelism by providing instructions that operate on entire blocks of data, called *vectors*. This is in contrast with scalar instructions that operate on single data items, one at a time.

In the early 1970s, vector supercomputers were designed around this concept. Through a single instruction, they performed the same basic operation on an entire vector. Nowadays, this vector technology is mainstream and is more commonly known as "SIMD," but the underlying principles are the same. With a single instruction, multiple elements are updated concurrently.

Through the SIMD support in OpenMP, the user has an easy and portable way to express this instruction-level parallelism. SIMD is tightly integrated with the OpenMP threading model, supporting multi-level parallelism. In this chapter, the OpenMP SIMD constructs are covered in detail.

5.1 An Introduction to SIMD Parallelism

In OpenMP, vectorization is referred to as *SIMD parallelism*, or "SIMD" for short. SIMD provides data-parallelism at the instruction level: a single instruction operates upon multiple data elements concurrently. SIMD instructions use special *SIMD registers* containing multiple data elements. The width of these registers determines the *vector length*, which is the number of scalar data items that can be processed in parallel by a SIMD instruction.

Figure 5.1 illustrates the SIMD concept. Two arrays b and c are added together, and the result is stored into the array a. For the purpose of this example, the arrays have 16 elements only. The loop that implements this operation requires 32 load instructions, 16 add instructions, and 16 store instructions.[1] On a system with SIMD instructions with a vector length of four, it takes only 8 load instructions, 4 add instructions, and 4 store instructions.

The advantage is that the SIMD instructions are (almost) as fast as their scalar counterpart. In other words, the number of processor cycles needed to add 4 elements in SIMD mode is the same as the time it takes to execute one scalar add instruction. This means that, in this case, the processor time is reduced by a factor

[1] The few instructions needed to execute the loop, the "loop overhead," are not considered here.

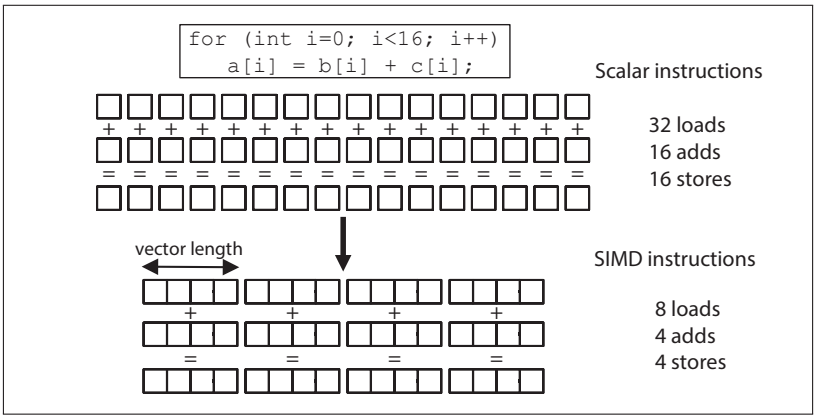

Figure 5.1: **An example of vectorization** – Vector instructions improve the performance by processing multiple data items concurrently.

of four. This is an upper limit, however. In practice, the improvement also depends upon the time it takes to move the data.

A compiler may be able to identify which operations are suitable for optimization using SIMD instructions, but it must prove that it is safe (or legal) to do so. Some of the challenges that a compiler encounters when trying to automatically generate SIMD instructions include: imprecise dependence information, data layout and alignment, conditional execution, packing and unpacking of scalar data items into vectors, calls to functions, and loop bounds that are not always an even multiple of the vector length.

In particular, C/C++ pointer variables pose a challenge to a compiler's dependence analysis. Different pointers may be aliases for the same memory block, resulting in seemingly independent operations having an implied dependence. Compilers must start with an assumption that a dependence exists between two accesses to memory and then attempt to prove that there is not one.[2]

Once it has been determined that a loop is safe to vectorize, there are some more issues to consider. To get the full benefit from SIMD, the starting address of the vectors may need to be aligned on the correct boundary. This means that the address in memory must be a multiple of the vector length in bytes.

[2]This is explained in more detail in the Glossary.

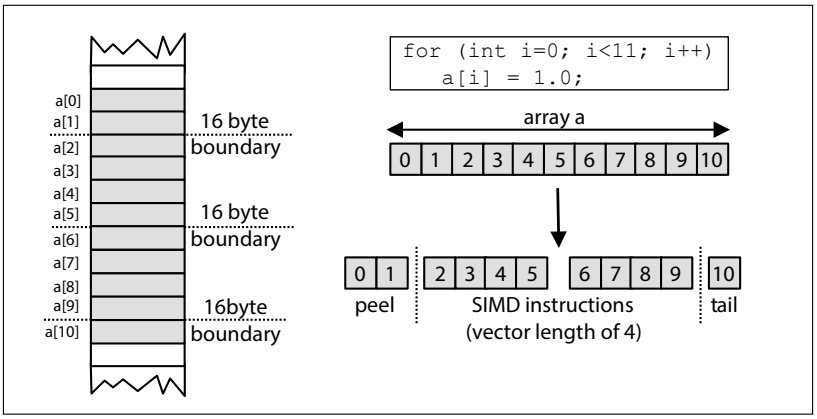

Figure 5.2: **Loop modifications needed to vectorize a loop using SIMD instructions** – To enforce alignment and ensure the remaining loop length is a multiple of the vector length, the compiler may need to peel off iterations and also treat the tail end separately.

If this is not the case, the compiler uses *loop peeling* to process the first element(s) separately, such that the first element processed with SIMD instructions starts on the correct boundary in memory. The vector is then processed in chunks that are a multiple of the vector length. If the number of loop iterations that remain after loop peeling is not a multiple of the vector length, the remaining items are processed in the *tail* part of the loop.

This is illustrated in Figure 5.2, where the array `a` with 11 elements is processed. This array is of type `float` and each element takes 4 bytes in memory. On a system with a SIMD register width of 16 bytes and with the memory-layout shown here, the first two elements need to be processed separately. The next 8 elements require two SIMD instructions. The remaining last element is handled separately.

The parts of the array processed separately are relatively inefficient. In most cases, the need to handle these situations can only be detected during program execution. In Section 5.2.5 it is shown which controls OpenMP provides to pass on more information to the compiler, which it can use to generate more efficient code.

#pragma omp simd *[clause[[,] clause]...] new-line* *for-loops*
!$omp simd *[clause[[,] clause]...]* *do-loops* **!$omp end simd**

Figure 5.3: **Syntax of the simd construct in C/C++ and Fortran** – The `simd` construct is the fundamental construct to express SIMD parallelism in a loop.

5.2 SIMD loops

The OpenMP compiler may transform a loop that is marked with the `simd` construct, into a *SIMD loop*. As a result, multiple iterations of the loop may be executed concurrently by a single thread. A SIMD loop of length n, consists of the logical iterations $0, 1, \ldots, n-1$. The numbering denotes the sequential order in which loop iterations are executed.

A *SIMD chunk* refers to the set of iterations that are executed concurrently by a SIMD instruction in a single thread. Within a chunk, each iteration is executed by a *SIMD lane*, which refers to the mechanism that a SIMD instruction uses to process one data element. The number iterations in a SIMD chunk is the vector length.

5.2.1 The Simd Construct

The `simd` construct applies to one or more subsequent loops. The structure of the loops on which the `simd` construct is placed must conform to the same restrictions that are required by the `for` (`do` in Fortran) construct. The `simd` construct syntax in C/C++ and Fortran is given in Figure 5.3. The clauses that are supported by the `simd` construct are listed in Figure 5.4.

The OpenMP specification describes the execution model of the `simd` construct as follows: "When an OpenMP thread encounters a `simd` construct, the iterations of the loop associated with the construct may be executed concurrently using the SIMD lanes that are available to the thread." Intentionally, this allows for much freedom in the implementation.

The code in Figure 5.5 sums the two arrays pointed to by `b` and `c` and stores the result in the array pointed to by `a`. All three arrays are of length n. For this operation, all loop iterations are independent and may be executed concurrently.

private *(list)*
lastprivate *(list)*
reduction *(reduction-identifier : list)*
collapse *(n)*
simdlen *(length)*
safelen *(length)*
linear *(list[:linear-step])*
aligned *(list[:alignment])*

Figure 5.4: **Clauses supported by the simd construct** – The semantics of the `private`, `lastprivate`, `reduction`, and `collapse` clauses have been extended to the `simd` construct and are described in the main text. The `simdlen` clause is described in Section 5.2.2. The `safelen` clause is described in Section 5.2.3, The `linear` clause is described in Section 5.2.4. The `aligned` clause is described in Section 5.2.5.

```
1 void simd_loop(double *a, double *b, double *c, int n)
2 {
3   int i;
4
5   #pragma omp simd
6   for (i=0; i<n; i++)
7      a[i] = b[i] + c[i];
8 }
```

Figure 5.5: **Example of the simd construct** – The vectors `b` and `c` are added and the result is stored in vector `a`.

This can be exploited by threads, for instance with the `for` construct, or with a SIMD loop (or with both).

Adding the OpenMP `simd` construct instructs the compiler to generate a SIMD loop. If, in fact, the pointer variable `b` or `c` is an alias for the pointer variable `a`, adding the `simd` construct to this loop would be a user error, the resulting behavior would be undefined, and incorrect results should be expected.

In this particular case, no further clauses are necessary. Similar to the `for` construct, the loop iteration variable `i` is `private`. However, with the `simd` construct, one private instance will be created per SIMD lane. The memory pointed to by `a`, `b`, and `c` (the arrays) is shared. The compiler is free to select the appropriate vector length that is suitable for the target architecture.

```
 1 void simd_loop_private(double *a, double *b, double *c, int n)
 2 {
 3   int i;
 4   double t1, t2;
 5
 6   #pragma omp simd private(t1, t2)
 7   for (i=0; i<n; i++)
 8   {
 9     t1 = func1(b[i], c[i]);
10     t2 = func2(b[i], c[i]);
11     a[i] = t1 + t2;
12   }
13 }
```

Figure 5.6: **Example using private variables in a simd construct** – The two function calls return intermediate results that must be stored in private variables to avoid a data race. Each SIMD lane has its own instance of a private variable.

A `simd` construct has a data environment with shared and private variables. The `private` and `lastprivate` clauses modify the `simd` construct's data environment. In Section 1.2.3, the concept of an "execution instance" was introduced. In the context of the `simd` construct, the execution instance is a SIMD lane, and one instance of each variable is created per SIMD lane.

The code fragment in Figure 5.6 illustrates the use of the `private` clause. The two function calls to `func1()` and `func2()` return intermediate results that are stored in the variables `t1` and `t2`, respectively. Because loop iterations are executed simultaneously by SIMD lanes, both variables must be privatized in order to avoid data races. The privatization is accomplished by listing the variables `t1` and `t2` in the `private` clause. Note that in this particular example, the same effect may be achieved by declaring both variables inside the loop.

The loop iteration variable `i` is privatized by default. However, in a `simd` construct, it is treated as `lastprivate`. After the loop, the original loop iteration variable contains the final value of the private loop iteration variable that was computed in the last sequential iteration.

```
 1 void simd_loop_collapse(double *r, double *b, double *c,
 2                         int n, int m)
 3 {
 4   int i, j;
 5   double t1;
 6
 7   t1 = 0.0;
 8   #pragma omp simd reduction(+:t1) collapse(2)
 9   for (i = 0; i<n; i++)
10       for (j = 0; j<m; j++)
11           t1 += func1(b[i], c[j]);
12   *r = t1;
13 }
```

Figure 5.7: **Example of the reduction and collapse clauses on the simd construct** – The two perfectly nested loops are collapsed into a single iteration space. The `reduction` clause is required to parallelize the accumulation into `t1`.

An instance of a private variable is not initialized and no assumption about its initial value can be made. Further, the lifetime of a private variable is restricted to the `simd` construct, and there is no method for accessing the value of a private variable after the region completes.

Variables that appear as list items in a `lastprivate` clause on a `simd` construct are private. As explained in Section 1.2.3, the final value of the private variable that is computed in the last sequential iteration is available after the construct in the original variable that the `lastprivate` variable corresponds to.

The `reduction` clause described in Section 2.4.3, on page 93 works similarly in the context of the `simd` construct. The `reduction` clause takes a list of variables and the reduction operator as its arguments. For each variable in the list, a private instance is used during the execution of the SIMD loop. The partial results are accumulated into the private instance. The reduction operator is applied to combine all partial results, such that the final result is returned in the original variable and made available after the SIMD loop.

The `collapse` clause described in Section 2.1.3, starting on page 43, works the same on the `simd` construct. The clause takes a positive integer number as its argument that indicates the number of loops that are collapsed into one iteration space. All the loop iteration variables in the associated collapsed loops are `lastprivate`.

```
1 unsigned int F(unsigned int *x, int n, unsigned int mask)
2 {
3    #pragma omp simd simdlen(32/sizeof(unsigned int))
4    for (int i=0; i<n; i++) {
5       x[i] &= mask;
6    } // End of simd region
7 }
```

Figure 5.8: **Example of the simdlen clause on the simd construct** – The `simdlen` clause is a hint to guide the compiler in the selection of a vector length for the loop.

The use of both the `reduction` and `collapse` clauses is illustrated in Figure 5.7. Through the `collapse` clause, the two loops are merged into a single loop. The `reduction` clause is used to correctly compute the variable `t1`.

The `collapse` clause should be used with caution in the context of vectorization, because it increases the complexity of the generated SIMD code. We recommend to verify that such a loop is vectorized as intended.

Many modern compilers provide a feature known as "compiler commentary," which provides information about the optimizations, or lack thereof, performed.[3] If vectorization is supported on the target processor, messages related to this may be helpful in determining how to effectively use the OpenMP `simd` constructs. We recommend checking the documentation of the compiler to determine if this feature is supported and, if so, how to enable it.

5.2.2 The Simdlen Clause

The `simdlen` clause takes a constant positive integer value as its argument. This value specifies the preferred number of iterations to be executed concurrently. It impacts the vector length used by the generated SIMD instructions.

The value in the clause is a preference. The compiler has the freedom to deviate from this choice and to choose a different length. In the absence of this clause, an implementation-defined default value is assumed for the vector length. The purpose of the `simdlen` clause is to guide the compiler. Perhaps the user has more informa-

[3]Success, or failure, of vectorization also depends on the compiler options used. Check the documentation of the compiler for relevant options to consider.

```
1 void dep_loop(float *a, float c, int n)
2 {
3   for (int i=8; i<n; i++)
4     a[i] = a[i-8] * c;
5 }
```

Figure 5.9: **Example of a loop-carried dependence** – The loop-carried dependence creates a limit on the vector length.

tion on the loop characteristics and knows that a specific length may be beneficial to performance. An incorrect choice may negatively impact the performance but will not lead to an incorrect result. Figure 5.8 is a code example that uses the `simdlen` clause on a `simd` construct.

5.2.3 The Safelen Clause

A limit on the vector length used by SIMD instructions is sometimes required. An example of this is when vectorizing a loop with loop-carried dependences. As shown in the code in Figure 5.9, an operation in a previous loop iteration must complete before an operation in the current loop iteration can execute. In this case, the read of `a[i-8]` on the current iteration i cannot execute until the write of `a[i]` on the previous $i - 8$ iteration is completed.

The *loop-carried dependence distance* is the number of loop iterations between a previous and current iteration, which in this example is 8. It is a distance between iterations, specifying how far a dependency is in the iteration space. When using SIMD parallelism, the vector length must be less than or equal to the distance of the smallest loop-carried dependence in the loop.

In this particular example, a vector length of up to 8 could be used. The maximum safe distance between loop iterations can be specified through the `safelen` clause.

The `safelen` clause takes a constant positive integer value as its argument. The value for `safelen` provides an *upper limit* on the vector length. This number specifies the length in which it is safe to vectorize the loop. The vector length ultimately selected by the compiler is still implementation-defined, but it does not generate SIMD code using a vector length that exceeds the `safelen` value.

```
1 void simd_loop_safelen(double *a, double *b, double *c, int n,
2                         int offset)
3 {
4   int i;
5   #pragma omp simd safelen(16)
6   for (i=offset; i<n; i++)
7     a[i] = b[i-offset] + c[i];
8 }
```

Figure 5.10: **Example of the safelen clause on the simd construct** – The `safelen` clause ensures that a vector length of up to 16 is correct.

The code fragment in Figure 5.10 illustrates the use of the `safelen` clause. The accesses to the elements of array `b` use the variable `offset`. Unless the compiler can determine the value of this variable, it may select a vector length that generates incorrect results. In this case, the user has more knowledge of the value. The value 16 is specified in the `safelen` clause at line 4. This means that the user guarantees that the loop can be safely vectorized using a vector length of 16 or less.

If the clause is not specified, the assumed value for `safelen` is the number of loop iterations. Note the difference between the `safelen` and `simdlen` clauses. The `safelen` clause is required for correctness while the `simdlen` clause indicates a preference.

5.2.4 The Linear Clause

Scalar data elements are packed into vectors, operated on collectively as a vector by SIMD instructions, and then unpacked. When the scalar data elements are accessed in a linear fashion, it is easy to see how the vector is constructed. For example, assuming that alignment restrictions have been satisfied, a single SIMD load or store instruction may be used to read and write consecutive scalar data elements as vectors.

In situations in which the scalar data elements are arranged consecutively in memory, the accesses to the scalar data elements are said to occur with a stride of one (*unit stride*). If the scalar data elements are not arranged consecutively in memory, but instead are at a regular offset from each other, then the data elements may be accessed with a stride greater than one. In this case, the stride is equal to the offset between the scalar data elements.

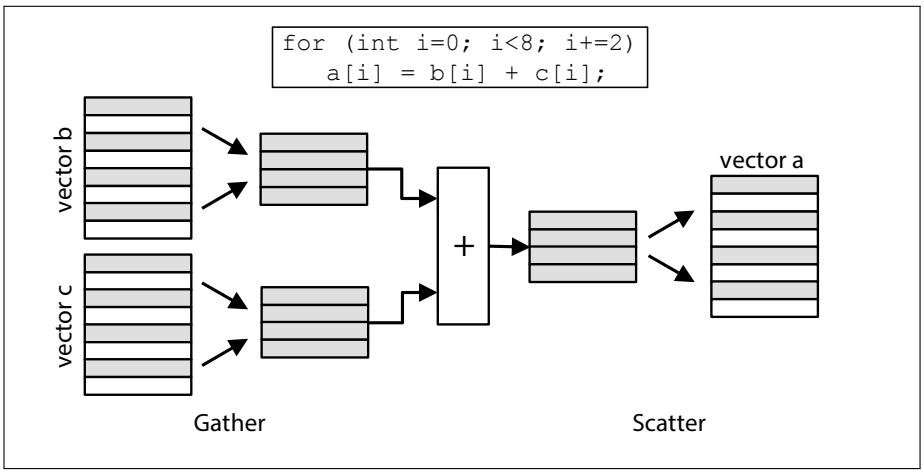

Figure 5.11: **Gather and scatter scalar data elements in and out of a vector** – Scalar data elements are gathered into vectors, operated on as a vector, and then scattered back to their destination locations.

To construct a vector, a *gather* operation reads scalar data elements from memory linearly but with a stride greater than one. Likewise, a *scatter* operation writes the scalar data elements in a vector back to memory linearly with a stride greater than one. The gather and scatter vector operations are illustrated in Figure 5.11. Some architectures have SIMD load and store instructions that support gather and scatter operations.

The best situation is when the vectors are accessed with a linear stride of one. The next best scenario is when the access pattern is linear but with a stride that is greater than one and gather and scatter instructions may be used. The worst case scenario is when no linear access pattern can be determined and the scalar data elements must be individually packed into and unpacked from vectors.

The `linear` clause is provided for the user to indicate the linear behavior of a variable in a loop. The compiler may use this information to determine the most efficient way to pack and unpack scalar data elements into vectors.

The `linear` clause is a data-sharing clause that provides a *superset* of the functionality of the `private` clause. On a loop or `simd` construct, the syntax of the clause is `linear` *(list[:linear-step])*. The `linear` clause accepts a list of variables as its argument. An optional, colon-separated, *linear-step* (stride) is supported.

```
1 void simd_loop_linear(double *a, double *b, double *c, int n,
2                        int offset)
3 {
4   int i, j = 0;
5
6   #pragma omp simd linear(j:1)
7   for (i=offset; i<n; i+=2)
8     a[i] = b[j++] + c[i];
9 }
```

Figure 5.12: **Example of the linear clause on the simd construct** – The `linear` clause defines how the `j` index variable relates to the loop variable `i` that is used as an index into some of the arrays.

The `linear` clause may appear on the `simd` or `for` (`do` in Fortran) constructs. In C, a variable that appears in the clause must have an integral or pointer type. In C++, the variable must either have an integral or pointer type or be a reference to an integral or pointer type. In Fortran, the variable must have an `integer` type. If a *linear-step* expression is specified, it must be invariant during the execution of the associated loop.

The semantics for the `private` clause apply: every variable listed in the clause is private in the associated construct. For the `simd` construct, it is private to a SIMD lane. In addition, this clause asserts that a variable has a linear relationship with the iteration space of the loop to which the clause applies. The value of the private variable in each loop iteration is defined as the value of the original variable, before the construct was entered, plus the logical number of the loop iteration times the *linear-step* or 1 if no *linear-step* is given. After the loop, the original variable has the final value of the private variable from the sequentially last iteration.

A common use case of the `linear` clause is a loop, where in addition to the loop variable, additional variables are used to index into arrays. If such variables have a linear relationship with the loop variable, this clause may be used. A simple example that demonstrates the use of this clause is shown in Figure 5.12.

Arrays `a` and `c` are accessed through the loop variable `i`, but array `b` is indexed through another variable, `j`. The variable `j` has a linear relationship with the loop iteration variable `i`, which is incremented by 2 in each iteration, while `j` is incremented by 1. Variable `i` takes the values `offset, offset+2, offset+4,...`.

The sequence for `j` is `0, 1, 2, ...`. By asserting the linear properties of the variable `j`, the compiler can more easily determine the access pattern for the array `b` and thus generate a SIMD instruction that reads `b` as a vector. Notice that a gather operation may be use to read the array `c` and that a scatter operation may be used to write to the array `a`.

The user may ask if the `linear` clause is necessary because a compiler can very often easily determine the linear behavior of a variable in a loop. The `linear` clause may also appear on `declare simd` directive, where it is used to specify the linear behavior of a function parameter (see Section 5.2.2). It is perhaps more useful in this capacity where the compiler may not be able to automatically determine the linear behavior of a function parameter.

5.2.5 The Aligned Clause

Data alignment is important for good performance. If a data element is not aligned on an address in memory that is a multiple of the size of the element in bytes, an extra cost is incurred when accessing the element. For example, on some architectures it may not be possible to load or store from a memory address that is not aligned to the size of the object being accessed. Instructions that do this are sometimes referred to as non-aligned loads and stores. Even if an architecture has non-aligned load and store instructions, they may execute with a higher cost to performance.

In many cases, alignment is handled by the compiler, but in certain scenarios this is impossible. As an example, consider processing a stream of data at the bit level. The program interprets this stream in terms of 32-bit integers. Depending on where in the stream the processing begins, the data may or may not be aligned.

Often, there are ways to enforce the correct alignment. For example, if two arrays are used in a loop, their relative alignment may be improved by adjusting the dimensions. There are also specific functions to enforce a certain alignment. An example is the `posix_memalign()` function from the POSIX standard. It allocates a memory block on a specific boundary. Other options are to consider vendor-specific features. We recommend checking the vendor's documentation for the details.

The alignment issues are the same when using SIMD instructions to vectorize loops. Figure 5.2 on page 223 illustrates the difference between aligned and unaligned data access in a vector loop. As explained there, in the case of a misalignment, iterations need to be "peeled off" to enforce alignment.

If vectorization is requested through the use of the OpenMP `simd` construct, but the resulting performance is poor, or the compiler commentary indicates issues in the vector code generation, then adjusting the data alignment may improve the performance.

The `aligned` clause is supported on both the `simd` construct, as well as the `declare simd` directive. This clause requires a list of variables as its argument, and an optional, colon-separated, *alignment*. The alignment must be a constant positive integer, and an implementation-defined default value is assumed if it is not given in the clause. In C, a variable that appears in the clause must have an array or pointer type. In C++, a variable that appears in the clause must have array, pointer, reference to array, or reference to pointer type. In C/C++, all the variables in the list are guaranteed to point to an object that is aligned in memory at an address that is a multiple of the number of bytes specified by the alignment. In Fortran, all the variables in the list are guaranteed to be aligned in memory at an address that is a multiple of the number bytes specified by the alignment.

If the alignment assumption is invalid and one or more variables do not have the alignment attributes specified, the behavior is implementation-dependent. It is highly recommended to avoid this type of error.

In the example code shown in Figure 5.13, the `align` clause appears on the `simd` construct at line 5. It asserts that the value of the pointer variable x is always aligned to a 16-byte boundary. Assuming that the size of the `float` data type is 4 bytes and knowing the alignment of the address in x is 16 bytes, the compiler has the information it requires to generate aligned 128-bit vector load and store instructions. It is important to note that the code is dependent on the assertion that value of `x` is always aligned to at least a 16-byte boundary.

5.2.6 The Composite Loop SIMD Construct

One of the main reasons for OpenMP to include support for SIMD parallelism is to cleanly define the interaction between vectorization through SIMD and thread-level parallelism, such as with the loop construct. Before support for SIMD was introduced in OpenMP 4.0, vendor-proprietary constructs had to be mixed with OpenMP. This not only violated portability, but there were also unwanted side-effects on performance.

```
1 void f_aligned(float *x, float scale, int n)
2 {
3   int i;
4
5   #pragma omp simd aligned(x:16)
6   for (i=0; i<n; i++)
7     x[i] = x[i]*scale;
8 }
```

Figure 5.13: **Example of the aligned clause on the simd construct** – The `aligned` clause asserts that the value of the pointer variable `x` is always aligned on a 16 byte boundary. With this information, the compiler may generate wider and more efficient vector load and store instructions.

#pragma omp for simd *[clause[[,] clause]...] new-line* *for-loops*
!$omp do simd *[clause[[,] clause]...]* *do-loops* **!$omp end do simd**

Figure 5.14: **Syntax of the loop simd construct in C/C++ and Fortran** – The composite construct combines thread and SIMD parallelism. Chunks of loop iterations are distributed to the threads in a team. The chunks of iterations are then executed with a SIMD loop. A clause that may appear on either the loop construct or the `simd` construct may appear on the composite clause.

In OpenMP, the `simd` construct may be combined with the loop construct, resulting in the `for simd` (`do simd` in Fortran) composite construct.[4] In the same way as the OpenMP `simd` construct, the `for simd` construct applies to the subsequent loop, and the same restrictions outlined earlier must be adhered to, in addition to possible restrictions from the `for` or `do` construct. The syntax for the `for simd` construct in C/C++ and the `do simd` construct in Fortran is given in Figure 5.14.

The composite construct addresses the following question: which is performed first, vectorization and then thread-level parallelism, or vice versa?

With the `for simd` construct, chunks of loop iterations are first distributed across the threads in a team by a method that is determined by any clauses that apply

[4]The difference between a composite and a combined construct is explained in Section 6.5.2 starting on page 283.

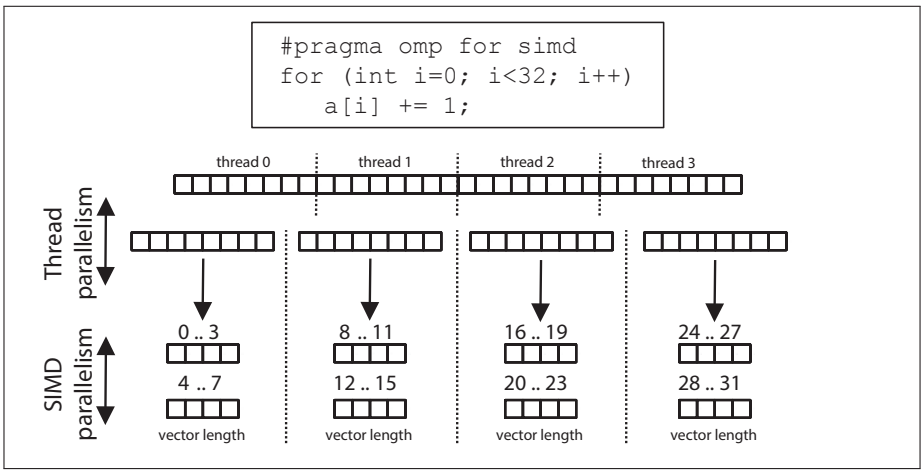

Figure 5.15: **Combining thread and SIMD parallelism** – Thread and SIMD parallelism are used to execute a loop.

to the `for` construct. Then, the chunks of loop iterations may be converted into SIMD loops in a way that is determined by any clauses that apply to the `simd` construct. The process is the same when using the `do simd` construct in Fortran. The distribution of loop iterations across threads and then SIMD loops is illustrated in Figure 5.15.

When using the composite construct, the user must be aware that both the number of threads, as well as the scheduling of the worksharing construct, may influence the efficiency of the resulting SIMD loop. Furthermore, the user must consider the effect of a private variable in the context of a composite construct. There is a private instance of the variable per SIMD lane.

Each loop has a certain amount of work associated with it. If the number of threads is increased, the amount of work performed per thread is reduced. Adding SIMD parallelism to this does not necessarily improve the efficiency. Especially if the SIMD loops that each thread works on reduce in length as the thread count increases. Most likely, some experimentation to find the optimal combination may be required.

The loop schedule of the worksharing construct affects the opportunities for vectorization. For good performance this must be taken into account. If the `static` schedule is used with a small chunk size, or if the `dynamic`, or `guided`, schedule is

```
 1 void func_1(float *a, float *b, int n)
 2 {
 3   #pragma omp for simd schedule(static, 5)
 4   for (int k=0; k<n; k++)
 5   {
 6     // do some work on a and b
 7   }
 8 }
 9
10 void func_2(float *a, float *b, int n)
11 {
12   #pragma omp for simd schedule(simd:static, 5)
13   for (int k=0; k<n; k++)
14   {
15     // do some work on a and b
16   }
17 }
```

Figure 5.16: **SIMD loops without and with the simd schedule modifier** – The `simd` schedule modifier in `func_2()` guarantees that a preferred implementation-defined vector length is respected when distributing the loop

used, the SIMD efficiency may not be optimal. The reason is that the number of iterations per thread may be too small for SIMD to be beneficial. The compiler may not be able to generate the most efficient SIMD code because of loop tails and poor memory access patterns. Ideally, the selected chunk size for the threads is a multiple of the vector length.

The code in function `func_1()` in Figure 5.16 shows an example of a problematic chunk size. Each thread gets assigned a loop with only 5 iterations. This loop is vectorized, but it is very short, and depending on the vector length of the target architecture, this may lead to inefficiencies.

To address this issue, the `simd` schedule modifier is used at line 12 in Figure 5.16. For the loop in function `func_2()`, the modifier results in an adjustment of the chunk size. If `chunk_sz` denotes the chunk size, the formula to adjust this for a given vector length is given by: $ceiling(\texttt{chunk\_sz}/\texttt{simd\_len}) \times \texttt{simd\_len}$, with `simd_len` the vector length of the target architecture. For example, if the vector length is eight `float` elements, the resulting chunk size is increased from 5 to 8.

```
 1 double compute_pi(int n)
 2 {
 3   const double dH = 1.0 / (double) n;
 4   double dX, dSum = 0.0;
 5
 6   #pragma omp parallel for simd private(dX) \
 7                         reduction(+:dSum) schedule(simd:static)
 8   for (int i=0; i<n; i++) {
 9     dX = dH * ((double) i + 0.5);
10     dSum += (4.0 / (1.0 + dX * dX));
11   }
12   // End parallel for simd region
13
14   return dH * dSum;
15 }
```

Figure 5.17: **Example that uses the composite for simd construct** – The numerical solution for integration has a compute-intensive loop that is parallelized and vectorized.

An example that combines many of the topics covered in this section is shown in Figure 5.17. The number π can be approximated by computing the integral $\int_0^1 \frac{4}{1+x^2}\,\mathrm{d}x$ through numerical integration. The `compute_pi()` function implements a simple numerical solution to this problem.

In Figure 5.17, the combined use of a `parallel` construct and the composite `for simd` construct is illustrated by placing the `parallel for simd` directive before the loop that spans lines $8-11$. Variable `dX` must be private, because it is modified in every loop iteration and the value differs for every SIMD lane. The computation of the summation at line 9 is a reduction operation that requires the `reduction` clause at line 7. This is an algorithm without load balancing issues and the `simd:static` schedule is most efficient.

5.2.7 Use of the Simd Construct with the Ordered Construct

The `ordered` clause is described in detail in Section 1.6.4. In short, the `ordered` region within a parallel loop is guaranteed to be executed in the original sequential order of the loop iterations. The code outside of this region is executed in parallel.

```
 1 extern int x;
 2 extern void global_update(int, int);
 3 #pragma omp declare simd
 4 extern int vec_work(int);
 5
 6 void F(int *a, int *b, int n)
 7 {
 8   #pragma omp simd
 9   for (int i=0; i<n; i++)
10   {
11     // vectorize this part of the loop
12     a[i] = vec_work(b[i]);
13
14     // execute this part of the loop in sequential order
15     #pragma omp ordered simd
16     {
17       global_update(x, a[i]);
18     } // End ordered simd region
19   } // End simd region
20 }
```

Figure 5.18: **An ordered region within a SIMD loop** – The `ordered` region is executed with one SIMD lane in the original sequential order of the loop iterations.

Since OpenMP 4.5, the `simd` clause is supported on the `ordered` construct. Following the semantics of the loop construct, this clause has the effect that any encountering thread uses only one SIMD lane to execute the `ordered` region in the sequential order of the loop iterations. An example that uses the `simd` clause on the `ordered` construct is shown in Figure 5.18.

On line 12, the call to the function `vec_work()` is executed with SIMD parallelism. The call to the function `global_update()` at line 17 is executed in the original sequential loop iteration order by one SIMD lane at a time. This approach is efficient if the `ordered` region contains a small portion of the code that is not critical to the overall runtime.

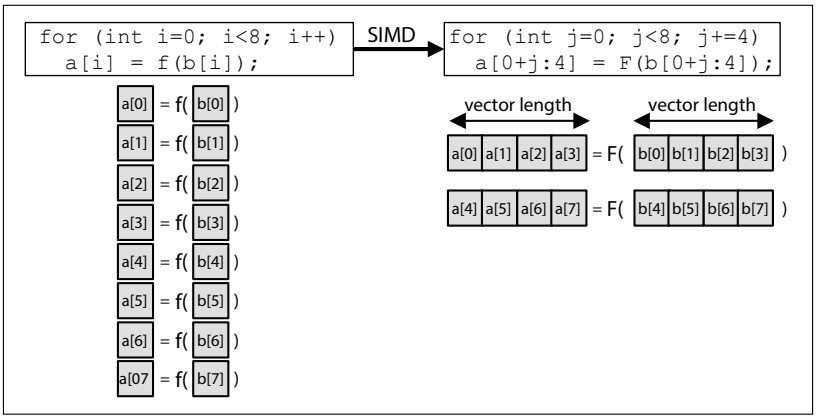

Figure 5.19: **Illustration of function calls with SIMD** – The scalar function `f()` is modified and renamed to `F()` in this example. This function supports vector input arguments and returns an entire vector.

5.3 SIMD Functions

Function calls inside a SIMD loop obstruct the generation of efficient SIMD instructions. In the worst case scenario, the call to the function must be done using scalar data elements, which will most likely negatively impact the efficiency of the generated code.

To fully exploit SIMD parallelism, a function called from within a SIMD loop must be to a SIMD equivalent version of the function. This means that the compiler must generate a special version of the function with SIMD parameters and code that uses SIMD instructions.

The concept of a SIMD function is illustrated in Figure 5.19. The compiler generates a SIMD version of the function `f()`. The function's scalar parameters are converted to vector parameters. Scalar operations in the function body are replaced with corresponding vector operations. If the function `f()` is called outside the context of a simd region, the scalar variant is used.

This `declare simd` directive and its clauses are used to tell the compiler to generate one or more SIMD versions of a function. These specialized versions of a function may then be called from SIMD loops.

#pragma omp declare simd *[clause[[,] clause]...] new-line* *function declaration or definitions*
!$omp declare simd*[(proc-name)] [clause[[,] clause]...]*

Figure 5.20: **Syntax of the declare simd directive in C/C++ and Fortran** – The `declare simd` directive is used to declare that one or more versions of a SIMD function should be generated.

simdlen *(length)*
linear *(list[:linear-step])*
aligned *(list[:alignment])*
uniform *(argument-list)*
inbranch
notinbranch

Figure 5.21: **Clauses supported by the declare simd directive** – The `simdlen`, `linear`, `aligned`, and `uniform` clauses, discussed in Section 5.3.2, declare SIMD attributes for function parameters. The `inbranch` and `notinbranch` clauses, described in Section 5.3.3, specify that a SIMD function variant is always called under a conditional branch and never called under a conditional branch.

5.3.1 The Declare Simd Directive

The `declare simd` directive is used to declare that a SIMD variant of a function may be called from a simd region. The syntax for this directive in C/C++ and Fortran is shown in Figure 5.20. The clauses that may appear on the `declare simd` directive are listed in Figure 5.21.

When the `declare simd` directive is placed before a function declaration, it asserts that a SIMD variant of the function, with characteristics that are described by the clauses on the directive, may be called from within a loop in a simd region. The user may think of this like a special type of SIMD function prototype. If the `declare simd` directive is placed in the same translation unit as the definition of the function, then a SIMD variant of the function is generated by the compiler.

The compiler may generate multiple versions of a SIMD function and select the appropriate version to invoke at a specific call-site in a `simd` construct. The user may tailor a SIMD function variant by using clauses on the directive. For example, different SIMD versions of a function may be specialized for either different SIMD instruction widths or other function parameter attributes.

```
 1 #pragma omp declare simd
 2 double my_func(double b, double c)
 3 {
 4   double r;
 5   r = b + c;
 6   return r;
 7 }
 8
 9 void simd_loop_function(double *a, double *b, double *c, int n)
10 {
11   int i;
12   #pragma omp simd
13   for (i=0; i<n; i += 2)
14   {
15     a[i] = my_func(b[i], c[i]);
16   }
17   // End simd region
18 }
```

Figure 5.22: **Example of the declare simd directive** – The function `my_func()` is called from within a `simd` construct. The `declare simd` directive declares that the compiler should generate a SIMD version of the function.

There are two restrictions. If a SIMD function has a side effect, the resulting behavior is undefined.[5] Furthermore, in C++, a function that appears under a `declare simd` directive is not allowed to throw any exceptions.

An example using the `declare simd` directive is shown in Figure 5.22. At line 1, the `declare simd` directive is used to inform the compiler that the function `my_func()` may be called from a simd region. This instructs the compiler to generate at least one additional SIMD version of the function `my_func()`. Because of the `simd` construct at line 12, SIMD instructions are used to execute the loop that spans lines $13 - 16$. The call to `my_func()` at line 15 will be to a SIMD variant of the function that was declared at line 1.

[5] In the Glossary it is explained what a side effect is.

Note that in some examples in this section, the function call and the function definition are presented as if they were both in the same file (or translation unit). This is done to keep the examples simple.

Because the compiler can see the definition of the function and the place where the function is called, the user may wonder why the `declare simd` directive is needed. Depending on the compiler used, it may not be.

The `declare simd` directive is more critical when a function is defined in one file and called in another file. In that case, the compiler is instructed to generate a SIMD function variant even though it may not see the call to the function in a place where SIMD instructions are used.

5.3.2 SIMD Function Parameter Attributes

The `uniform`, `linear`, `simdlen`, and `aligned` clauses are used to specify attributes for SIMD function parameters. Except for the `simdlen` clause, the variables that appear in these clauses must be parameters of the function to which the directive applies.

When a parameter is listed in the `uniform` clause, it indicates that the parameter is passed an argument with the same value for all concurrent calls to the function in the execution of a single SIMD loop. This means that all SIMD lanes observe the same value for the parameter. If the arguments from consecutive loop iterations are passed as a vector to a SIMD variant of a function, then each element in the vector has the same value.

The `linear` clause has a different meaning when it appears on a `declare simd` directive. In this context, it is not a data-sharing clause as described in Section 5.2.4. Instead, it indicates that an argument passed to a function parameter has a linear relationship across the concurrent invocations of a function. Each SIMD lane observes the value of the argument in the first SIMD lane plus the offset of the SIMD lane from the first SIMD lane times the *linear-step*. For example, if the linear step is 2, there are 8 simd lanes, and the first simd lane observes the value 10, then the second lane observes the value 12, the third lane observes the value 14, and so on.

The example in Figure 5.23 shows the combined use of the `uniform` and the `linear` clauses on a function that is marked for vectorization with the `declare simd` directive. At line 14, the function `cosScaled` is called with the loop-invariant base address of the array `b`. The variable `c` is loop-invariant and the loop variable

```
 1 #include <math.h>
 2 #pragma omp declare simd uniform(ptr, scale) linear(idx:1)
 3 double cosScaled(double *ptr, double scale, int idx)
 4 {
 5   return (cos(ptr[idx]) * scale);
 6 }
 7
 8 void simd_loop_uniform_linear(double *a, double *b, double c,
 9                                int n)
10 {
11   int i;
12
13   #pragma omp simd
14   for (int i=0; i<n; i++) {
15     a[i] = cosScaled(b, c, i);
16   }
17   // End simd region
18 }
```

Figure 5.23: **Example of the linear and uniform clauses on the declare simd directive** – The SIMD variant of the `cosScaled` function may assume that `ptr` and `scale` are loop invariant and that `idx` is incremented by 1 each time through the loop from where the function is called.

`i` is an offset into the `b` array. In the function `cosScaled`, the `ptr` parameter is a pointer and the `scale` parameter is a scalar.

The `uniform(ptr, scale)` clause informs the compiler to generate a SIMD function that assumes both of these parameters are passed arguments that are loop-invariant. The `idx` parameter appears in a `linear` clause, indicating to the compiler that the `idx` parameter is passed an argument that is incremented by 1 each time through the SIMD loop from where the function is called. By listing `ptr` in a `uniform` clause and `k` in a `linear` clause, the compiler can generate a unit stride SIMD load instruction for the write to `ptr[idx]`.[6]

Figure 5.24 is another example that uses the `uniform` and `linear` clauses to optimize the access to a multi-dimensional array within a function that is called from a SIMD loop. If the index in the first dimension is always the same, and the

[6] In [26], the vector code generated by the Intel compiler for a similar example is discussed.

```
1 #pragma omp declare simd uniform(x, y, d1, i, a) linear(j)
2 void saxpy_2d(float *x, float *y, float a, int d1, int i, int j)
3 {
4   y[(d1*i)+j] = a*x[(d1*i)+j] + y[(d1*i)+j];
5 }
```

Figure 5.24: **Example using the uniform and linear clauses for multi-dimension array access** – Access in the first dimension is uniform. The loop that calls the `saxpy_2d()` function indexes linearly through the second dimension via the variable `j`.

```
 1 #pragma omp declare simd simdlen(16)
 2 char F(char x, char y, unsigned char mask)
 3 {
 4    return (x + y) & mask;
 5 }
 6
 7 void img_mask(char *img1, char *img2, int n, unsigned char *m)
 8 {
 9   #pragma omp simd simdlen(16)
10   for (int i=0; i<n; i++) {
11     img1[i] = F(img1[i], img2[i], m[i]);
12   } // End simd region
13 }
```

Figure 5.25: **Example of the simdlen clause on the declare simd directive** – Use the `simdlen` clauses on the `declare simd` directive to generate a SIMD variant of the function `F()` that has a vector length of 16.

outer (SIMD) loop progresses over the second dimension, then the index variable of the first dimension may appear in a `uniform` clause to assert that this variable has an invariant value.

A `simdlen` clause may appear on a `declare simd` directive and has a similar meaning as it does for the `simd` construct. The constant value that appears in the clause specifies a length for vectorized arguments. The clause is treated as a hint on the `simd` construct; on the `declare simd` directive, it is not. The compiler generates a SIMD version of the function, which expects its vector arguments to have a length specified in the clause. The function variant may be called from

```
 1 #pragma omp declare simd linear(src,dst) \
 2                     aligned(src,dst:16) simdlen(32)
 3 void copy32x8(char *dst, char *src)
 4 {
 5    *dst = *src;
 6 }
 7
 8 #pragma omp declare simd uniform(x,y) linear(i) \
 9                           aligned(x,y:64) simdlen(16)
10 float saxpy(float a, float *x, float *y, int i)
11 {
12   return a * ((x[i]) + (y[i]));
13 }
```

Figure 5.26: **Example of the aligned clause on the declare simd directive** – Use the `aligned` clauses on the `declare simd` directive to generate SIMD variants of the functions `copy32x8()`, and `saxpy()` that expect the addresses passed in pointer arguments to have a specific byte alignment.

a SIMD loop whose corresponding `simd` construct has a `simdlen` clause with the same value. An example that does this is shown in Figure 5.25.

When a pointer variable appears in an `aligned` clause on a `declare simd` directive, it declares that the value of the pointer variable argument has the specified byte alignment. The SIMD version of the function may then use aligned vector memory accesses for the pointer variable. When the SIMD function is called from a simd region, the object pointed to by the pointer variable argument must be aligned to the specified byte boundary. This can be ensured by using the `aligned` clause on the `simd` construct that encloses the call to the SIMD function. Examples that use the `aligned` clause combined with other clauses on `declare simd` directives are shown in Figure 5.26.

The `linear` clause supports a *modifier* that provides additional capabilities for C++ and Fortran. The following description focuses on the semantics of the modifier in C++.[7] The syntax of the `linear` clause with a modifier is `linear`*(modifier(list)[:linear-step])*. The modifier may be `uval` or `ref`. The `uval`

[7]See the OpenMP 4.5 specifications for more details on using the `linear` clause with a modifier in Fortran.

```
 1 #pragma omp declare simd linear(ref(x)) linear(uval(c))
 2 void increment(int& x, int& c)
 3 { x += c; }
 4
 5 void Fref(int *a, int n)
 6 {
 7   #pragma omp simd
 8   for (int i=0; i<n; i++) {
 9     increment(a[i], i);
10   } // End simd region
11 }
```

Figure 5.27: **Example of the ref and uval modifiers in the linear clause** – The `ref` modifier declares that the address of `x` is linear. The `uval` modifier declares that the address of `c` is uniform, and its value is linear.

or `ref` modifier can be used only if the function parameter has a reference type. The modifier defines how the address and the value of a variable are observed across SIMD lanes as follows:

- `uval(x)`: The storage address of `x` is uniform. The value in the storage location `x` is linear.
- `ref(x)`: The storage address of `x` is linear.

A simple C++ example that uses the `ref` and `uval` modifiers in the `linear` clause on a `declare simd` directive is shown in Figure 5.27.

5.3.3 Conditional Calls to SIMD Functions

The `inbranch` and the `notinbranch` clauses declare whether or not a SIMD variant of a function is called conditionally from a simd region. There are no arguments for these clauses.

A conditional branch is a point in a program in which the execution control may be transferred to another point within the program. An example is an `if-then-else` statement. The `if` part is the branch (instruction). If the condition evaluates to `false`, execution is transferred to the `else` part.

In the presence of conditional branches, the generation of SIMD instructions using the full vector length may not be possible. Because a conditional branch

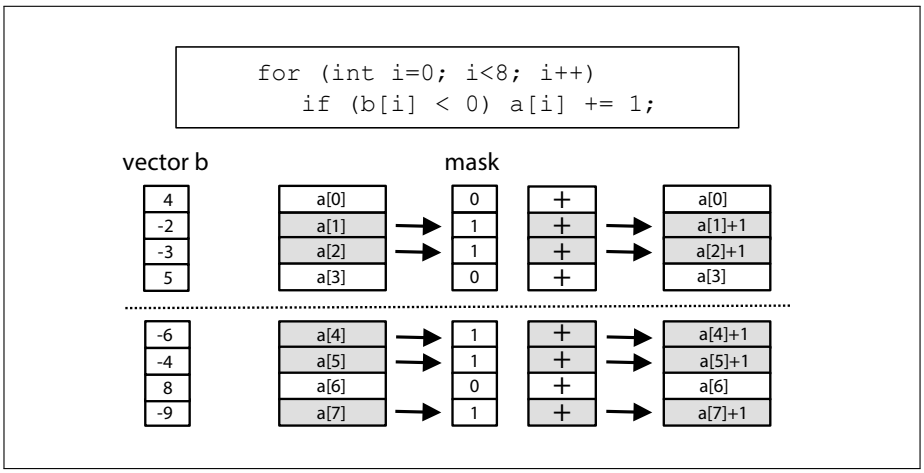

Figure 5.28: **Conditional control flow converted to masked vector instructions** – A vector mask predicate is used to enable or disable an operation on a given vector element. If the indexed mask is 1, the operation occurs. When it is 0, the operation is masked off.

creates an uncertainty regarding the number of consecutive elements available, it is more difficult for the compiler to generate SIMD instructions that use the full vector.

Depending on the target architecture, a compiler may generate *masked* vector code. Masked vector instructions use a bit vector, called "the mask," to ensure that the vector operation is applied only to those elements for which the mask bit is set to `true`. This concept is illustrated in Figure 5.28.

The `inbranch` clause asserts that the function is always called from within a conditional branch in a SIMD loop. With the `inbranch` clause present, the compiler must restructure the code to handle the possibility that a SIMD lane may not execute the code in the function. One approach to doing this is to pass the vector mask as an extra argument to the SIMD function variant.

The `notinbranch` clause may be used when the function is never called from within a conditional branch in a SIMD loop. The `notinbranch` clause enables the compiler to be more aggressive at optimizing the code in the function to use SIMD instructions. It can do this, because it does not need to consider the conditional execution of the instructions that execute in a SIMD lane.

```
 1 #pragma omp declare simd inbranch
 2 float do_mult(float x)
 3 {
 4   return (-2.0*x);
 5 }
 6
 7 #pragma omp declare simd notinbranch
 8 extern float do_pow(float);
 9
10 void simd_loop_with_branch(float *a, float *b, int n)
11 {
12   #pragma omp simd
13   for (int i=0; i<n; i++) {
14     if (a[i] < 0.0 )
15       b[i] = do_mult(a[i]);
16
17     b[i] = do_pow(b[i]);
18   } /* --- end simd region --- */
19 }
```

Figure 5.29: **Example of the inbranch and notinbranch clauses on the declare simd directive** – The `inbranch` clause tells the compiler to generate a SIMD variant of the function `do_mult()` that must be called conditionally within a SIMD loop. The `notinbranch` clause on the declaration of the `do_pow()` function tells the compiler that there is a SIMD variant of the function that must be called unconditionally within a SIMD loop.

If neither clause is specified, then the SIMD version of the function may or may not be called from within a branch, and the code the compiler generates must handle either situation.

The code in Figure 5.29 illustrates the use of the `inbranch` and `notinbranch` clauses. At line 1, the `inbranch` clause is used on the `declare simd` directive to tell the compiler to generate a SIMD variant of the function `do_mult()` that assumes it is always called conditionally. At line 15, the function `do_mult()` is called only when the condition computed at line 14 evaluates to 1 (`true`). The compiler generates a call to the SIMD variant of the function declared at line 1. If the compiler uses masking, the function is called unconditionally with an extra argument that passes the condition mask.

```
1 #pragma omp declare simd linear(pixel) uniform(mask) inbranch
2 #pragma omp declare simd linear(pixel) notinbranch
3 #pragma omp declare simd
4 extern void compute_pixel(char *pixel, char mask);
```

Figure 5.30: **Example of multiple declare simd directives for a function** – Multiple SIMD versions of the function are generated. The invocation of a specific version of the function is determined by where it is called.

At line 7, the `notinbranch` clause is used on the `declare simd` directive to tell the compiler that there is a SIMD variant of the function `do_pow()` that is optimized to assume it is never called conditionally. At line 17, the function `do_pow()` is called unconditionally. The compiler generates a call to the SIMD variant of the function declared at line 7, which does not pass a vector mask to the function.

5.3.4 Multiple Versions of a SIMD Function

As shown in Figure 5.30, multiple consecutive `declare simd` directives with different clauses may appear before the declaration of a function. When the declaration is in the same file as the definition of the function, the SIMD function variants are generated for each `declare simd` directive.

The multiple consecutive `declare simd` directives must appear on a function declaration that is visible when the function is called in a `simd` construct. This enables the compiler to select the best SIMD function variant for the function call.

5.4 Concluding Remarks

This chapter has covered how to use the OpenMP SIMD constructs to exploit SIMD instructions, which are present in several contemporary microprocessors. The performance improvements from using SIMD instructions can be substantial. In [15], the authors proposed a draft version of what became the OpenMP SIMD constructs and evaluated the performance improvement for selected benchmarks. In [14], the authors discuss vectorization for a specific architecture.

The SIMD constructs in OpenMP may be used as a stand-alone feature to vectorize loops, but in many cases, we expect the SIMD constructs to be used to further improve the performance of an application already parallelized using OpenMP

threads. This is achieved by adding the `simd` constructs presented here to the time-consuming loops.

The capabilities of modern compilers to successfully vectorize code differ. Check the compiler's documentation for the relevant options to consider and to see if it supports a compiler commentary feature.

The SIMD loop body should not contain complex nested conditional branches, and the `if` clause should be avoided. When possible, align objects on byte boundaries that match an architecture's vector length. Use the `aligned` clause to indicate a variable's alignment. Use the `safelen` clause to vectorize loops with dependences, but care needs to be taken to use it the correct way. Use the `schedule(simd:static)` on the `for simd` and `do simd` constructs to balance thread and SIMD parallelism.

Utilize the `linear`, `uniform`, `simdlen`, and `aligned` clauses on the `declare simd` directive to generate specialized SIMD function variants. Add the `inbranch` and `notinbranch` clauses to the `declare simd` directive to generate SIMD versions of functions that may be called conditionally or unconditionally depending on the function call site. Be careful when using multiple `declare simd` directives for the same function. Each directive may result in another version of the function, which can increase the size of a program.

The `simd` and `for simd` (`do simd` in Fortran) constructs are combined with other OpenMP constructs. The `taskloop simd` construct is discussed in Chapter 3. The `distribute parallel for simd`, `distribute parallel do simd`, and `distribute simd` constructs are discussed in Chapter 6. This underlines the strength of OpenMP. In a consistent, portable, and integrated way, multiple levels of parallelism are supported.